

UNIT - 1

Introduction

1. Brief History of Python

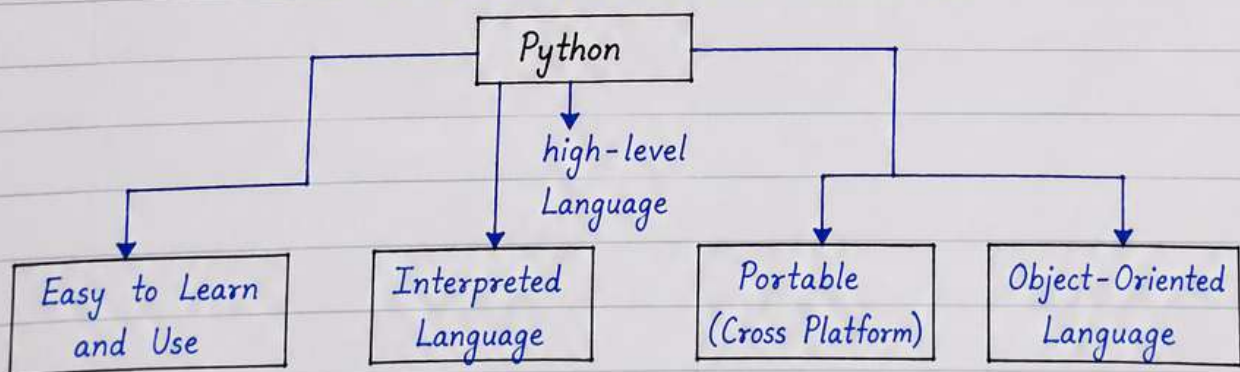
- Python एक high-level, interpreted और general-purpose programming language है।
- इसे Guido van Rossum द्वारा बनाया गया था।
- इसका विकास कार्य दिसम्बर 1989 में शुरू हुआ और पहला release 20 फरवरी 1991 को हुआ।
- Python का उद्देश्य code को आसान, readable और efficient बनाना था।
- इसका नाम "Python" BBC के comedy show "Monty Python's Flying Circus" से लिया गया है, जो Guido को बहुत पसंद था।
- Python को शुरुआती समय में scripting language के रूप में डिज़ाइन किया गया था।
- आज Python का उपयोग Web Development, Data Science, AI, Machine Learning, Automation, Networking, आदि क्षेत्रों में व्यापक रूप से किया जाता है।

2. Python Versions

Python के कई versions release हुए हैं। उनमें से प्रमुख versions निम्नलिखित हैं -

Version	Release Year	Key Features
Python 1.0	1994	Initial release with basic features.
Python 2.0	2000	New features like List comprehensions, Garbage Collection.
Python 2.7	2010	Last major release of Python 2 series.
Python 3.0	2008	Major update, not backward compatible. Better structure and new features.
Python 3.6	2016	Improved performance, f-strings, new dict structures.
Python 3.11	2022	Faster performance, better error messages, improved features.

* Note : वर्तमान में Python 3.x series का उपयोग अधिक किया जाता है क्योंकि यह modern, secure और efficient है।



3. Installing Python

Python को install करने के लिए python.org की official website से latest version download किया जाता है।

Steps :

- ① python.org पर जाएँ।
- ② "Downloads" पर click करें।
- ③ अपने OS (Windows / macOS / Linux) के लिए latest Python version download करें।
- ④ Download complete होने के बाद installer file run करें।
- ⑤ Important : "Add Python to PATH" का option select करें।
- ⑥ "Install Now" पर click करें और installation complete होने दें।
- ⑦ Installation के बाद "Close" पर click करें।

Verify Installation :

Command Prompt (CMD) खोलें और निम्न command लिखें -

`python --version` (या) `python -V`

अगर Python सही से install हुआ है तो version number दिखाई देगा।

4. Environment Variables

Environment Variables वे values होती हैं जो system में programs और processes के behavior को control करती हैं।

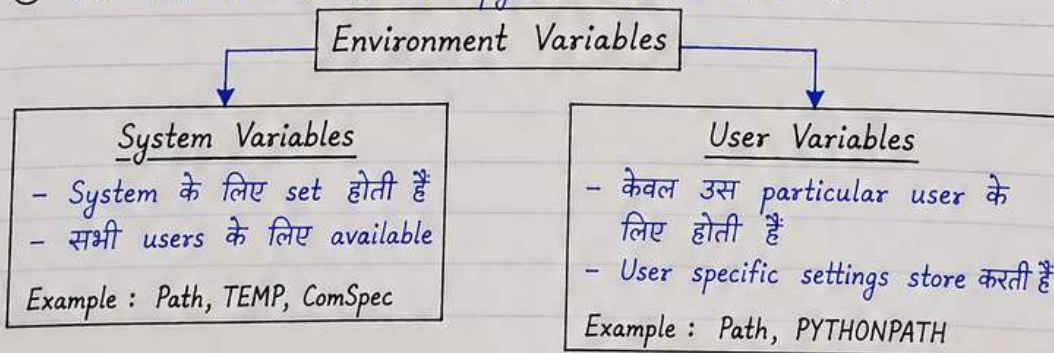
Python के लिए सबसे important variable है PATH.

PATH Variable :

PATH variable में Python installation directory का path add करने से हम किसी भी location से Python को run कर सकते हैं।

PATH Variable set करने के Steps (Windows) :

- ① Start menu में "Edit the system environment variables" search करें और open करें।
- ② "Environment Variables" बटन पर click करें।
- ③ System variables में "Path" को select करें और "Edit" पर click करें।
- ④ "New" पर click करके Python installation folder का path add करें।
(Default : C:\PythonXX\ ; XX = version)
- ⑤ "OK" पर click करें।
- ⑥ सभी windows को "OK" करके बंद करें।
- ⑦ अब CMD में कहीं से भी "python" लिखकर run करें।



Executing Python from the Command Line, IDLE

1. Python को Command Line से Execute करना

- Python को command line (Terminal / CMD) से चलाने के लिए system में python interpreter को command prompt द्वारा access करके program को run किया जाता है।
- इसके लिए Python का executable file python.exe होता है, जो command line से direct execute होता है।

2. Command Line से Python चलाने की विधि

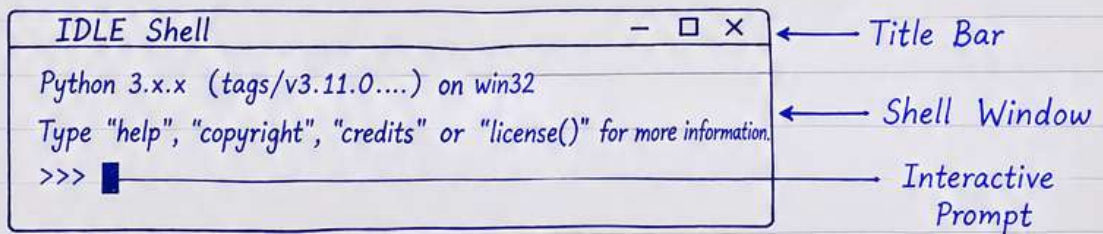
- ① Start → Run → cmd (या Terminal) open करें।
- ② कमांड prompt पर python लिखकर Enter दबाएँ।
- ③ Python interpreter start हो जाएगा और वहाँ आप Python code लिखकर तुरंत execute कर सकते हैं।

उदाहरण:

```
C:\> python
Python 3.x.x (tags/v3.11.0....)
>>> print("Hello Python")
Hello Python
```

3. IDLE (Integrated Development and Learning Environment)

- IDLE Python का official GUI tool है, जिसके द्वारा code लिखना, edit करना और run करना आसान होता है।
- इसके मुख्य भाग :



4. Python Command Line vs IDLE

बिंदु	Command Line	IDLE
उपयोग	तेज और हल्का	आसान और user friendly
इंटरफेस	Text based	GUI based
उद्देश्य	Program execute	Code लिखना, test करना
किसके लिए	Advanced Users	Beginners के लिए बेहतर

5. महत्वपूर्ण कमांड्स

- python → Python interpreter start करना।
- python -V → Python version देखना।
- exit() → Python से बाहर निकलना।

* Note : Command line से Python run करने के लिए python लिखना और IDLE से code लिखने व debug करने के लिए उपयोगी हैं।

1. Editing Python Files

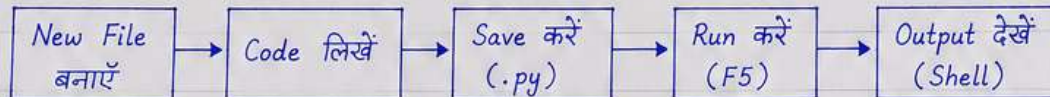
- Python program लिखने के लिए हम किसी text editor या IDE का उपयोग करते हैं।
- Python files सामान्यतः .py extension के साथ save की जाती हैं।
- File edit करना, save करना और run करना बहुत आसान होता है।

Commonly Used Editors / IDEs :

Editor / IDE	Use	Features
IDLE	Python का built-in IDE	Simple, easy to use, beginners के लिए best
VS Code	Advanced code editor	Auto-complete, extensions, debugging, terminal support
PyCharm	Professional IDE	Smart coding assistance, debugging, project management
Notepad++	Lightweight text editor	Syntax highlighting, multiple tabs, plugin support

File Edit करने के Steps (IDLE में) :

- ① IDLE खोलें।
- ② File → New File पर click करें।
- ③ Editor window में अपना Python code लिखें।
- ④ File → Save As पर click करके file को .py extension के साथ save करें।
- ⑤ Run → Run Module (F5) दबाएँ, output shell में प्राप्त होगा।



* Note : हमेशा अपने program को सही नाम और .py extension के साथ save करें।

2. Python Documentation

- Python Documentation आधिकारिक रूप से Python language की पूरी जानकारी प्रदान करती है।
- यह beginners से लेकर advanced users तक सभी के लिए उपयोगी होती है।
- Documentation में modules, functions, classes, methods और examples की पूरी जानकारी होती है।

Python Documentation के प्रमुख भाग :

- ① Tutorial - Python सीखने के लिए step-by-step guide
- ② Library Reference - Standard library modules और functions की जानकारी
- ③ Language Reference - Python language की syntax और rules
- ④ Examples - Code examples और sample programs
- ⑤ How-to Guides - Common tasks को करने के तरीके

Documentation Access करने के तरीके :

1. <https://docs.python.org/3/>
 - Official Python Documentation website
 - यहाँ search box में किसी भी topic को search करके जानकारी प्राप्त कर सकते हैं।

* Note : Documentation पढ़ने की आदत डालने से coding skills और problem solving skills दोनों में सुधार होता है।

1. Getting Help

- Python में help प्राप्त करने के कई तरीके होते हैं।
- जब हम किसी module, function, class या method के बारे में जानना चाहते हैं तो हम help system का उपयोग करते हैं।
- यह documentation, usage और examples के बारे में जानकारी देता है।

Help प्राप्त करने के तरीके :

- ① help() function का उपयोग
- ② built-in function dir() का उपयोग
- ③ help() function (Interactive Mode में)
- ④ Online Python Documentation
- ⑤ IDLE Help Menu

① help() function का उपयोग :

help() function किसी module, function या object के बारे में detailed information दिखाता है।

```
C:\> python
>>> help(print)
Help on built-in function print in module builtins:
print(*args, sep=' ', end='\n', file=None, flush=False)
Prints the values to a stream, or to sys.stdout by default.
```

help() से उस function का syntax, parameters और description प्राप्त होता है।

② dir() function :

dir() किसी module या object के सभी attributes और methods की list देता है।

```
>>> import math
>>> dir(math)
```

यह math module के सभी functions और attributes की list return करता है।

③ Interactive Mode में help() :

Python shell में help() बिना argument के लिखने पर help system खुल जाता है।

```
>>> help() → Help Utility Window → topic search करें → जानकारी प्राप्त करें
```

④ Online Documentation : <https://docs.python.org/3/>

⑤ IDLE Help Menu : Help Menu → Python Docs / About IDLE

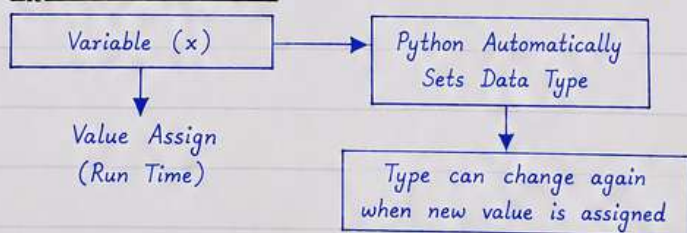
2. Dynamic Types

- Python एक Dynamically Typed language है, इसका मतलब है कि variables का data type run time में automatically तय होता है।
- हमें variable declare करते समय उसका type specify करने की जरूरत नहीं होती।
- Variable का type उस value के अनुसार बदल सकता है जो उसे assign की जाती है।

उदाहरण :

```
x = 10          # int
print(type(x))  # <class 'int'>
x = 3.14        # float
print(type(x))  # <class 'float'>
x = "Hello"     # str
print(type(x))  # <class 'str'>
x = [1, 2, 3]   # list
print(type(x))  # <class 'list'>
```

Type बदलने का Flow :



मुख्य बातें :

- Python में Static type declaration की आवश्यकता नहीं होती।
- यही कारण है कि Python beginners के लिए आसान भाषा मानी जाती है।
- Dynamic typing से code लिखना तेज और flexible होता है।

1. Python Reserved Words

- Python में कुछ words को Reserved Words या Keywords कहा जाता है।
- इन words का उपयोग Python syntax में special meaning के लिए होता है।
- हम इन words को variable, function, class या किसी identifier के नाम के रूप में उपयोग नहीं कर सकते।
- Python 3.x में कुल 35 Reserved Words होते हैं।

False	None	True	and	as	assert	async	await
break	class	continue	def	del	elif	else	except
finally	for	from	global	if	import	in	is
lambda	nonlocal	not	or	pass	raise	return	try
while	with	yield					

* Note : Reserved Words को हम बदल (rename) नहीं सकते, क्योंकि इनका उपयोग Python interpreter के द्वारा syntax और operations को define करने के लिए किया जाता है।

2. Naming Conventions

- Naming Conventions से तात्पर्य है कि हम variables, functions, classes, modules आदि के नाम किस नियम के अनुसार रखें।
- सही naming code को readable और maintainable बनाता है।
- Python में नामकरण के लिए निम्नलिखित प्रमुख नियम हैं:

① Meaningful Names (सार्थक नाम)

हमेशा ऐसा नाम रखें जो उसके काम को दर्शाए।

✓ total_marks × x, y, z

② Use of Letters, Digits and Underscore

नाम में केवल letters (A-Z, a-z), digits (0-9) और underscore (_) का उपयोग करें।

✓ age_10 ✓ student1
× age@10 × stu#dent

③ First Character Rule

नाम की शुरुआत letter (A-Z, a-z) या underscore (_) से होनी चाहिए, digit से नहीं।

✓ _value ✓ total
× 1value × 2total

④ Case Sensitivity (Case Sensitive)

Python में upper case और lower case अलग-अलग माने जाते हैं।

total, Total और TOTAL
तीनों अलग variables हैं।

⑤ No Spaces Allowed

नाम में space नहीं होना चाहिए। इसके स्थान पर underscore (_) का उपयोग करें।

✓ total_marks
× total marks

⑥ Avoid Python Reserved Words

Reserved Words का उपयोग नाम के रूप में नहीं करें।

× class, def, return, if ...
(यह Reserved Words हैं)

⑦ Recommended Style

Variable और function names के लिए lower_case_with_underscores
Class names के लिए CamelCase (First letter capital)
Constant names के लिए UPPER_CASE का उपयोग करें।

✓ student_name
✓ calculate_sum()
✓ StudentInfo
✓ PI_VALUE

* Summary :

Reserved Words Python language के special keywords होते हैं और इन्हें identifier के रूप में उपयोग नहीं किया जा सकता। सही Naming Conventions follow करने से हमारा code clean, readable, maintainable और professional बनता है।

UNIT- 2 BASIC PYTHON SYNTAX

1. Python Reserved Words

- Python में कुछ words को Reserved Words या Keywords कहा जाता है।
- इनका use Python syntax में special meaning के लिए होता है।
- हम इन words को variable name, function name, class name या किसी identifier के रूप में use नहीं कर सकते।
- Python 3.x में कुल 35 Reserved Words होते हैं।

False	None	True	and	as	assert	async	await
break	class	continue	def	del	elif	else	except
finally	for	from	global	if	import	in	is
lambda	nonlocal	not	or	pass	raise	return	try
while	with	yield					

* Note: Reserved Words को हम बदल (rename) नहीं सकते, क्योंकि इनका उपयोग Python interpreter के द्वारा syntax और operations को define करने के लिए किया जाता है।

2. Naming Conventions (नामकरण के नियम)

- Python में variables, functions, classes, modules आदि के नाम निर्धारित करने के लिए कुछ नियम follow किए जाते हैं।
- सही naming code को readable, maintainable और professional बनाती है।
- Python में नामकरण के लिए निम्नलिखित प्रमुख नियम हैं:

① Meaningful Names (सार्थक नाम)

हमेशा ऐसा नाम रखें जो उसके काम को दर्शाए।

✓ total_marks, student_age
✗ tm, x1

② Use of Letters, Digits and Underscore

नाम में केवल letters (A-Z, a-z), digits (0-9) और underscore (_) का उपयोग करें।

✓ age10, student_1
✗ age@10, stu#dent

③ First Character Rule

नाम की शुरुआत letter (A-Z, a-z) या underscore (_) से होनी चाहिए, digit से नहीं।

✓ _value, total
✗ 1value, 2total

④ Case Sensitivity (Case Sensitive)

Python में upper case और lower case अलग-अलग माने जाते हैं।

total, Total और TOTAL
तीनों अलग variables हैं।

⑤ No Spaces Allowed

नाम में space नहीं होना चाहिए। इसके स्थान पर underscore (_) का उपयोग करें।

✓ total_marks
✗ total marks

⑥ Avoid Python Reserved Words

Reserved Words का उपयोग नाम के रूप में नहीं करें।

✗ class, def, return, if...
(यह Reserved Words हैं)

⑦ Recommended Style

- Variable और function names के लिए lower_case_with_underscores
- Class names के लिए CamelCase (First letter capital)
- Constant names के लिए UPPER_CASE का उपयोग करें।

✓ student_name
✓ CalculateSum()
✓ MAX_VALUE

* Summary: Reserved Words Python के special keywords होते हैं और इन्हें identifier के रूप में use नहीं किया जा सकता। सही Naming Conventions follow करने से हमारा code clean, readable, maintainable और professional बनता है।

1. String Values

- Python में String, characters (अक्षरों) का sequence होता है।
- String को single quotes (' '), double quotes (" ") या triple quotes (''' ''' या "" "" """) के द्वारा लिखा जा सकता है।
- String immutable होती है, अर्थात् एक बार create होने के बाद इसके characters को बदला नहीं जा सकता।

String Declaration के तरीके :

Example	Explanation
'Hello'	Single quoted string
"Hello"	Double quoted string
'''Hello'''	Triple quoted string (multi-line string के लिए उपयोगी)
"""Hello"""	Triple double quoted string

उदाहरण :

```
s1 = 'Python'
s2 = "Programming"
s3 = '''This is
multi-line
string'''
s4 = """This is also
multi-line
string"""
```

Indexing in String :

String के हर character का एक index होता है।

Index 0 से शुरू होता है।

s = 'Python'

P	y	t	h	o	n
index → 0	1	2	3	4	6

Negative Indexing : दायें से बायें की ओर -1 से शुरू होता है।

P	y	t	h	o	n
-7	-6	-5	-4	-3	-1

2. String Methods

- Python में कई built-in methods होती हैं जिनका उपयोग String पर अलग-अलग operations करने के लिए किया जाता है।
- नीचे कुछ महत्वपूर्ण String methods दिए गए हैं :

Method	Description	Example	Output
lower()	सभी characters को lower case में बदलता है।	'PYTHON'.lower()	'python'
upper()	सभी characters को upper case में बदलता है।	'python'.upper()	'PYTHON'
title()	हर word के पहले letter को capital बनाता है।	'hello world'.title()	'Hello World'
strip()	String के start और end से spaces हटाता है।	' Python '.strip()	'Python'
len()	String की length (कुल characters की संख्या) लौटाता है।	len('Python')	6
replace(old, new)	पुराने substring को नए substring से replace करता है।	'I love Python'.replace('love', 'like')	'I like Python'
find(sub)	Substring का first occurrence का index लौटाता है। न मिलने पर -1।	'Python Programming'.find('Pro')	7
split(sep)	String को separator के आधार पर list में तोड़ता है।	'a,b,c'.split(',')	['a', 'b', 'c']
join(iterable)	Iterable (जैसे list) के elements को String में जोड़ता है।	'-'.join(['a', 'b', 'c'])	'a-b-c'
startswith(sub)	क्या String दिए गए substring से start होती है? (True/False)	'Python'.startswith('Py')	True
endswith(sub)	क्या String दिए गए substring से end होती है? (True/False)	'Python'.endswith('on')	True
count(sub)	Substring कितनी बार आता है, यह बताता है।	'banana'.count('a')	3

उदाहरण :

```
s = ' Hello Python '
print(s.strip())      → 'Hello Python'
print(s.lower())      → ' hello python'
print(s.replace('Python', 'World')) → ' Hello World '
print(s.split())      → ['Hello', 'Python']
print('-'.join(['A', 'B', 'C']))  → 'A-B-C'
```

Note :

- * String immutable होती है, इसलिए कोई भी method original string को change नहीं करती, एक नई string return करती है।

1. The format() Method

- Python में `format()` method का उपयोग string formatting के लिए किया जाता है।
- यह method string में placeholder `{}` के स्थान पर values को insert करता है।
- Placeholder को index number या key name के द्वारा refer किया जा सकता है।
- यह method बहुत flexible और readable होता है।

Syntax :

"string with {} placeholders".format(value1, value2, ...)

या

"string with {key} placeholders".format(key=value)

Examples :

Code	Explanation	Output
① "My name is {}".format("Pankaj")	एक value insert की	My name is Pankaj
② "{} + {} = {}".format(2, 3, 5)	तीन values insert की	2 + 3 = 5
③ "{0} is {1} years old".format("Ravi", 20)	index number का उपयोग	Ravi is 20 years old
④ "{name} lives in {city}".format(name="Ram", city="Kanpur")	key name का उपयोग	Ram lives in Kanpur
⑤ "{x:.2f}".format(12.34567) # float formatting	float value को 2 decimal में	12.35
⑥ "{:<10} {:>10} {:^10}".format("left", "right", "center")	alignment (left, right, center)	left right center ----- ----- -----

* Note :

- * f-string (formatted string literal) Python 3.6+ में और भी आसान तरीका है।

ex. name = "Pankaj" age = 20

f"My name is {name} and I am {age} years old."

→ My name is Pankaj and I am 20 years old.

Advantage of format()

- Readable
- Reusable
- Supports both index & key
- Alignment & formatting options

2. String Operators

- Python में strings पर कई operators काम करते हैं।
- इनकी मदद से strings को combine, repeat, compare आदि किया जा सकता है।

Operator	Name	Description	Example	Output
+	Concatenation	दो strings को जोड़ता है	"Hello" + "World"	'Hello World'
*	Repetition	String को n बार repeat करता है	"Hi" * 3	'HiHiHi'
in	Membership	एक string दूसरी string में है या नहीं	'ell' in "Hello"	True
not in	Non-membership	एक string दूसरी string में नहीं है	'xyz' not in "Hello"	True
==	Equality	दो strings बराबर हैं या नहीं	"abc" == "abc"	True
!=	Inequality	दो strings बराबर नहीं हैं	"abc" != "def"	True
<	Less than	पहली string दूसरी से छोटी है (lexicographically)	"apple" < "banana"	True
>	Greater than	पहली string दूसरी से बड़ी है	"zoo" > "apple"	True
<=	Less than or equal	छोटी या बराबर	"cat" <= "cat"	True
>=	Greater than or equal	बड़ी या बराबर	"dog" >= "dog"	True

Examples :

```
s1 = "Python"
s2 = "Programming"
print(s1 + " " + s2) → 'Python Programming'
print(s1 * 2) → 'PythonPython'
print('thon' in s1) → True
print('java' not in s1) → True
print('A' < 'B') → True
```

Note :

- * String comparison lexicographical order में होता है, यानी dictionary order के अनुसार।
- * in और not in operators search के लिए बहुत useful होते हैं।
- * + और * सबसे अक्सर उपयोग होने वाले operators हैं।

1. Numeric Data Types

- Python में numbers को store करने के लिए अलग-अलग Numeric Data Types होते हैं।
- ये types immutable होते हैं, अर्थात इनका value change नहीं किया जा सकता।
- Python 3.x में मुख्यतः तीन Numeric Types होते हैं: int, float और complex।

Data Type	Description	Example	Type() Output	Size (Approx.)
int	Integer numbers (पूरी संख्या) Positive, Negative या Zero	10, -25, 0, 999	<class 'int'>	Variable (Depends on value)
float	Floating point numbers (दशमलव संख्या)	10.5, -3.14, 0.0, 1.2e3	<class 'float'>	24 bytes (Approx.)
complex	Complex numbers (a + bj के रूप में)	3+4j, -2.5j, 10+0j	<class 'complex'>	32 bytes (Approx.)

Notes :

- * int की size value के अनुसार बढ़ती या घटती है।
- * float IEEE 754 standard के अनुसार store होता है।
- * complex number में real part (a) और imaginary part (b) होता है, $j = \sqrt{-1}$ ।

Examples :

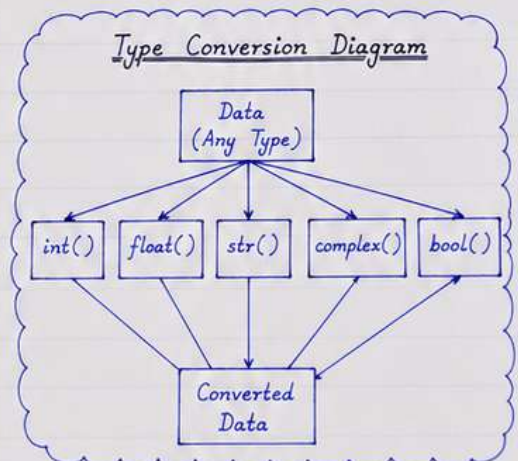
```
>>> type(10)
<class 'int'>
>>> type(3.14)
<class 'float'>
>>> type(2+5j)
<class 'complex'>
```

2. Conversion Functions

- Conversion functions का उपयोग एक data type को दूसरे data type में बदलने के लिए किया जाता है।
- Python में कुछ built-in functions होती हैं जो data conversion में मदद करती हैं।

(A) Common Conversion Functions

Function	Description	Example	Output
int(x)	x को integer में convert करता है	int(25.9)	25
		int('100')	100
float(x)	x को float में convert करता है	float(10)	10.0
		float('3.14')	3.14
complex(x)	x को complex में convert करता है	complex(5)	(5+0j)
		complex('2+3j')	(2+3j)
str(x)	x को string में convert करता है	str(50)	'50'
		str(3.14)	'3.14'
bool(x)	x को boolean में convert करता है	bool(0)	False
		bool(25)	True



(B) Type Conversion Examples

```
>>> int(3.99)      → 3
>>> int('45')     → 45
>>> float('7.25')  → 7.25
>>> str(100)       → '100'
>>> str(3.5)       → '3.5'
>>> complex('4+6j') → (4+6j)
>>> bool(0)        → False
>>> bool(100)      → True
```

Important Points :

- * int() decimal value को truncate करता है।
- * float(), int() string को valid number होने पर ही convert करते हैं, वरना ValueError देते हैं।
- * bool(0), bool(0.0), bool(''), bool([]), bool({}) हमेशा False return करते हैं।
- * बाकी सभी values True मानी जाती हैं।

Note : सही data type का चयन program की efficiency और memory usage के लिए बहुत महत्वपूर्ण होता है।

1. Simple Output

- Python में output स्क्रीन पर print() function द्वारा display किया जाता है।
- print() function text (string), variables, numbers या expressions को output के रूप में दिखाता है।
- यह function end में newline (\n) add करता है।

Syntax :

```
print(value1, value2, ..., sep = ' ', end = '\n')
```

Default Values

sep = ' ' (space)
end = '\n' (newline)

Examples :

Code	Explanation	Output
① print("Hello World")	String को print करता है।	Hello World
② print(10)	Integer number को print करता है।	10
③ print(3.14)	Float number को print करता है।	3.14
④ print("Python", 2024)	Multiple values (default sep = space)	Python 2024
⑤ print("A", "B", "C", sep = '-')	Separator '-' के साथ output देता है।	A-B-C

Important Points :

- * print() automatically next line पर चला जाता है।
- * end parameter के द्वारा newline को change किया जा सकता है।
ex. print("Hello", end = ' ') → cursor next line पर नहीं जाएगा।
- * sep parameter के द्वारा values के बीच separator change किया जा सकता है।

More Examples :

```
>>> print("Hello", end = ' ')  
>>> print("World")  
Output → Hello World  
>>> print(10, 20, 30, sep = ', ')  
Output → 10, 20, 30
```

2. Simple Input

- Python में user से data प्राप्त करने के लिए input() function का उपयोग किया जाता है।
- input() function हमेशा string के रूप में data return करता है।

Syntax :

```
variable = input(prompt)
```

Note

prompt user को बताता है।
कि कौन-सा input देना है।

Examples :

Code	Explanation	User Input (मान लें)	Output (Stored in Variable)
① name = input("Enter your name: ")	User से name input लो	Pankaj	name = 'Pankaj'
② age = input("Enter your age: ")	User से age input लो	20	age = '20'
③ num = input("Enter a number: ")	User से number input लो	55	num = '55'

Type Conversion Example :

```
num = input("Enter a number: ")  
print("Type of num:", type(num)) # String  
num = int(num)  
print("After conversion:", type(num)) # Integer
```

Important Points :

- * input() हमेशा string return करता है।
- * Numerical calculations के लिए type conversion करना आवश्यक होता है।
- * Prompt के बिना input() लेने पर cursor अगली line में चला जाता है।

Complete Example :

```
name = input("Enter your name: ")  
age = int(input("Enter your age: "))  
print("Hello", name)  
print("Next year your age will be", age + 1)
```

User Input (मान लें)

Pankaj
20

Output

Hello Pankaj
Next year your age will be 21

1. The % Method

- Python में % operator का उपयोग string formatting (old style formatting) के लिए किया जाता है।
- इसमें format string के अंदर % के साथ placeholders होते हैं और values को tuple के रूप में दिया जाता है।
- यह method Python 2.x से use में था और आज भी backward compatibility के लिए available है।

Syntax :

"format string" % (value1, value2, ...)

Placeholders :

Placeholder	Data Type	Example	Meaning
%d	Integer	"Age: %d" % 20	Integer number
%f	Float	"Price: %f" % 99.50	Float number
%.2f	Float (precision)	"PI: %.2f" % 3.14159	Float with 2 decimal places
%s	String	"Name: %s" % "Pankaj"	String
%c	Character	"Grade: %c" % 'A'	Single character
%%	Percent Sign	"Discount: 50%% off"	Prints % sign

Examples :

- ① "Hello %s" % "Pankaj"
- ② "Age = %d years" % 20
- ③ "PI value = %.2f" % 3.14159
- ④ "Name: %s, Age: %d" % ("Pankaj", 20)
- ⑤ "Total = %d, Price = %0.2f" % (2, 199.956)

Output :

- Hello Pankaj
- Age = 20 years
- PI value = 3.14
- Name: Pankaj, Age: 20
- Total = 2, Price = 199.96

2. The print() Function

- print() function Python में output display करने के लिए use होता है।
- यह function values, variables, expressions आदि को screen पर दिखाता है।
- print() function end में newline add करता है, जिससे अगला output अगली line में आता है।

Syntax :

print(value1, value2, ..., sep=' ', end='\n')

Parameters :

- value1, value2, ... : print करने के लिए values या variables
- sep : values के बीच separator (default space)
- end : line के अंत में add करने वाला character (default newline)

Default Parameters

sep = ' ' (space)
end = '\n' (newline)

Examples :

Code	Output
print("Hello")	Hello
print(10)	10
print("Pankaj", 20)	Pankaj 20
print("A", "B", "C")	A B C
print("Hello", end=' ')	Hello World
print("World")	(same line)
print(1, 2, 3, sep='-')	1-2-3

More Examples :

- * print("Hi", "Python", sep='***')
Output → Hi***Python
- * print("Line 1")
print("Line 2")
Output → Line 1
Line 2
- * print("End", end='!')
- print("Done")
Output → End!Done

Note : * Python 3 में f-string (formatted string literal) का use ज्यादा होता है, लेकिन % method और print() function still useful हैं।
* print() function debugging और output display के लिए बहुत important है।

Unit - 3 Language Components

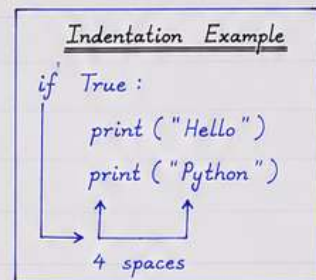
Topic : Indenting Requirements

- Python एक block structured language है, जिसमें indentation (indent) का उपयोग code blocks को define करने के लिए किया जाता है।
- Python में { } (curly braces) का उपयोग नहीं होता, उसकी जगह indentation का उपयोग किया जाता है।
- Indentation से Python interpreter को पता चलता है कि कौन सा statement किस block के अंदर आता है।
- यदि indentation सही नहीं होगा तो IndentationError प्राप्त होता है।

1. Indentation क्या होता है?

किसी भी statement को दाईं side (left margin से अंदर की ओर) लिखना indentation कहलाता है।

Python में आमतौर पर 4 spaces को एक indentation level माना जाता है। हम tab key का भी उपयोग कर सकते हैं, लेकिन consistency जरूरी है।



2. Indenting के नियम (Rules)

- समान block के सभी statements का indentation level एक जैसा होना चाहिए।
- किसी new block के statements को पिछले line से अधिक indent करना होता है।
- Block समाप्त होने पर indentation कम (dedent) करना होता है।
- Mixing of spaces and tabs एक ही block में नहीं करना चाहिए।
- आमतौर पर 4 spaces per indentation level का उपयोग best practice माना जाता है।

3. Indentation का उपयोग

→ Python में indentation का उपयोग निम्न कार्यों के लिए होता है -

- ① Control Statements (if, else, elif, for, while, etc.)
- ② Functions और Class के अंदर code define करने के लिए
- ③ Loop और Conditional block को define करने के लिए
- ④ Code को readable और logically structured बनाने के लिए

Why Important ?

Indentation code की structure और logic को साफ और readable बनाता है।

<u>Correct Indentation Example</u>	<u>5. Incorrect Indentation Example</u>
<pre>x = 10 if x > 5: print("x is greater than 5") else: print("x is less than or equal to 5") for i in range(3): print(i) print("Inside loop")</pre>	<pre>x = 10 if x > 5: print("x is greater than 5") else: print("x is less than or equal to 5") for i in range(3): print(i) print("Inside loop")</pre> Same level पर write नहीं होना चाहिए यह Loop के अंदर होना चाहिए था

6. Common Indentation Errors

- ① IndentationError: unindent does not match any outer indentation level
- ② unexpected indent
- ③ inconsistent use of tabs and spaces in indentation

Best Practices

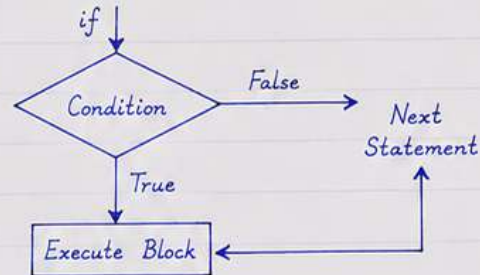
- ✓ हमेशा 4 spaces का उपयोग करें।
- ✓ एक ही code block में spaces और tabs को mix न करें।
- ✓ Code को properly align और indent करने की आदत डालें।

Topic : The if Statement

- if statement Python में decision making या conditional execution के लिए use किया जाता है।
- यह statement किसी condition को evaluate करता है और condition True होने पर block के statements को execute करता है।
- अगर condition False होती है तो if block skip कर दिया जाता है।

1. Basic Syntax

```
if condition:  
    statement(s)  
next statement
```



2. How it Works ?

- Python condition ko check करता है।
- यदि condition True होती है, तो उसी indentation level (block) के सभी statements execute होते हैं।
- यदि condition False होती है, तो if block को skip कर दिया जाता है। और control next statement पर चला जाता है।

Important :

Python indentation (spaces) के द्वारा block define करता है, इसलिए proper indentation बहुत जरूरी है।

3. Examples

	Example Code	Explanation	Output
①	<pre>x = 10 if x > 5: print("x is greater than 5") print("End of program")</pre>	x की value 10 है और condition True है, इसलिए if block execute होगा।	x is greater than 5 End of program
②	<pre>x = 3 if x > 5: print("x is greater than 5") print("End of program")</pre>	x की value 3 है और condition False है, इसलिए if block skip होगा।	End of program

4. Indentation Example (Very Important)

<u>Correct Indentation</u> (✓)	<u>Incorrect Indentation</u> (X)
<pre>age = 18 if age >= 18: print("You are eligible to vote.") print("Thank You!") (Proper indentation)</pre>	<pre>age = 18 if age >= 18: print("You are eligible to vote.") print("Thank You!") (Improper indentation) → IndentationError</pre>

Note :

Python में code की readability और structure indentation पर depend करता है।

5. Points to Remember

- if statement अकेले भी use किया जा सकता है।
- condition के बाद colon (:) लगाना अनिवार्य है।
- if block के सभी statements का indentation समान होना चाहिए।
- if statement case sensitive होता है (True और true अलग होते हैं)।

Use :

if statement का use उन situations में होता है जहाँ हमें किसी condition के True होने पर कुछ action करना होता है।

Topic : Relational and Logical Operators

- Python में operators का उपयोग values और expressions को evaluate करने के लिए किया जाता है।
- Relational Operators दो values के बीच relation check करते हैं और result True या False return करते हैं।
- Logical Operators conditions को combine करते हैं।

1. Relational Operators

- Relational Operators का उपयोग दो values के बीच comparison करने के लिए होता है।
- Result हमेशा Boolean (True या False) मिलता है।

Relational Operators Example

$10 > 5 \rightarrow \text{True}$
 $10 == 5 \rightarrow \text{False}$
 $10 != 5 \rightarrow \text{True}$
 $5 <= 5 \rightarrow \text{True}$

<u>Operator</u>	<u>Name</u>	<u>Meaning</u>	<u>Example</u>	<u>Result</u>
$==$	Equal to	दोनों values बराबर हैं।	$5 == 5$	True
$!=$	Not equal to	दोनों values बराबर नहीं हैं।	$5 != 3$	True
$>$	Greater than	बाएँ value दाएँ value से बड़ी है।	$7 > 4$	True
$<$	Less than	बाएँ value दाएँ value से छोटी है।	$3 < 6$	True
$>=$	Greater than or equal to	बाएँ value बड़ी या बराबर है (right value से)	$6 >= 6$	True
$<=$	Less than or equal to	बाएँ value छोटी या बराबर है (right value से)	$4 <= 9$	True

2. Logical Operators

- Logical Operators multiple conditions को combine करने के लिए उपयोग होते हैं।
- Result हमेशा True या False होता है।
- Python में तीन Logical Operators होते हैं: and, or, not.

(A) and Operator

- and operator तब True return करता है जब दोनों conditions True हों।
- किसी भी एक condition False होने पर result False होता है।

A	B	A and B
True	True	True
True	False	False
False	True	False
False	False	False

Example :

$x = 10, y = 5$

$(x > 5) \text{ and } (y < 10) \rightarrow \text{True and True} \rightarrow \text{True}$

$(x < 5) \text{ and } (y > 10) \rightarrow \text{False and False} \rightarrow \text{False}$

(B) or Operator

- or operator तब True return करता है जब कम से कम एक condition True हों।
- दोनों conditions False होने पर ही result False होता है।

A	B	A or B
True	True	True
True	False	True
False	True	True
False	False	False

Example :

$(x > 5) \text{ or } (y > 10) \rightarrow \text{True or False} \rightarrow \text{True}$

$(x < 5) \text{ or } (y < 3) \rightarrow \text{False or False} \rightarrow \text{False}$

(C) not Operator

- not operator एक ही condition को reverse (उल्टा) कर देता है।
- True को False और False को True कर देता है।

A	<u>not A</u>
True	False
False	True

Example :

$x = 10$

$\text{not } (x > 5) \rightarrow \text{not True} \rightarrow \text{False}$

$\text{not } (x < 5) \rightarrow \text{not False} \rightarrow \text{True}$

Key Points :

- * Relational operators comparison के लिए उपयोग होते हैं।
- * Logical operators multiple conditions को combine करते हैं।
- * Result हमेशा Boolean (True/False) होता है।

Topic : Bit Wise Operators

- Bitwise Operators bits पर काम करते हैं, न कि पूरे numbers पर।
- ये operators integers के binary representation पर perform करते हैं।
- इनका उपयोग low level programming, system programming और performance critical applications के लिए किया जाता है।
- Python में Bitwise Operators निम्नलिखित हैं: $\&$, $|$, $\wedge$, $\sim$, $\ll$, $\gg$

1. Bitwise Operators की सूची

Operator	Name	Meaning	Example	Result
$\&$	AND	दोनों bits 1 होने पर 1	$5 \& 3$	1
$ $	OR	दोनों bits में से कोई एक 1 हो तो 1	$5 3$	7
$\wedge$	XOR	दोनों bits अलग हैं तो 1	$5 \wedge 3$	6
$\sim$	NOT	सभी bits को उल्टा करता है (1's complement)	~ 5	-6
$\ll$	Left Shift	Bits को बायीं ओर shift करता है	$5 \ll 1$	10
$\gg$	Right Shift	Bits को दायीं ओर shift करता है	$5 \gg 1$	2

2. Bitwise Operators कैसे काम करते हैं? (उदाहरण सहित)

(A) AND Operator ($\&$)

उदाहरण : $A = 5$, $B = 3$

Binary में,

$A = 5 \rightarrow 0000\ 0101$

$B = 3 \rightarrow 0000\ 0011$

$A \& B \rightarrow 0000\ 0001 \rightarrow 1$

Rule :

$1 \& 1 = 1$
बाकी सभी स्थिति में 0

(B) OR Operator ($|$)

$A = 5 \rightarrow 0000\ 0101$

$B = 3 \rightarrow 0000\ 0011$

$A | B \rightarrow 0000\ 0111 \rightarrow 7$

Rule :

$1 | 1 = 1$
 $1 | 0 = 1$
 $0 | 1 = 1$
 $0 | 0 = 0$

(C) XOR Operator ($\wedge$)

$A = 5 \rightarrow 0000\ 0101$

$B = 3 \rightarrow 0000\ 0011$

$A \wedge B \rightarrow 0000\ 0110 \rightarrow 6$

Rule :

$1 \wedge 1 = 0$
 $1 \wedge 0 = 1$
 $0 \wedge 1 = 1$
 $0 \wedge 0 = 0$

(D) NOT Operator ($\sim$)

$A = 5 \rightarrow 0000\ 0101$

$\sim A \rightarrow 1111\ 1010$

(In Decimal) $\rightarrow -6$

Python में $\sim A = -(A+1)$ होता है।

(E) Left Shift ($\ll$)

$A = 5 \rightarrow 0000\ 0101$

$A \ll 1 \rightarrow 0000\ 1010 \rightarrow 10$

$A \ll 2 \rightarrow 0001\ 0100 \rightarrow 20$

Left shift में दाएँ ओर 0 fill होते हैं।

(F) Right Shift ($\gg$)

$A = 5 \rightarrow 0000\ 0101$

$A \gg 1 \rightarrow 0000\ 0010 \rightarrow 2$

$A \gg 2 \rightarrow 0000\ 0001 \rightarrow 1$

Right shift में बाएँ ओर sign bit (0) fill होता है।

3. Example (सभी एक साथ)

$a = 5$, $b = 3$

$a \& b = 1$

$a | b = 7$

$a \wedge b = 6$

$\sim a = -6$

$a \ll 1 = 10$

$a \gg 1 = 2$

Note :

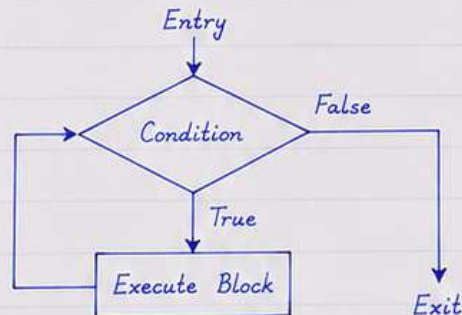
Bitwise Operators का उपयोग कभी-कभी memory management, encryption, graphics, और low level programs में किया जाता है।

Topic : The while Loop

- while loop Python में एक entry controlled loop होता है।
- यह loop condition के True होने तक बार-बार execute होता है।
- जैसे ही condition False होती है, loop terminate हो जाता है और control loop के बाहर चला जाता है।
- while loop उन situations में use किया जाता है जहाँ हमें नहीं पता होता कि loop कितनी बार चलेगा।

1. Syntax

```
while condition :  
    statement(s)  
    (update statement)
```



2. How it Works ?

- सबसे पहले condition check होती है।
- अगर condition True है, तो loop के अंदर का block execute होता है।
- फिर update statement के द्वारा condition को change किया जाता है।
- इसके बाद दोबारा condition check होती है।
- यदि condition False हो जाती है, तो loop stop हो जाता है और control loop के बाहर चला जाता है।

Important :

- while loop में कम-से-कम condition एक बार check जरूर होती है।
- infinite loop से बचने के लिए update statement जरूरी है, जो condition को False कर सके।

3. Flow of while Loop

Start → Check Condition → (True) Execute Block → Update Condition → Check Condition

(False) ↓
Exit

4. Example Program

1 से 5 तक numbers print करना

```
i = 1  
while i <= 5 :  
    print(i)  
    i = i + 1
```

Output :

1
2
3
4
5

Example : Sum of first n natural numbers

```
n = 5  
i = 1  
sum = 0  
while i <= n :  
    sum = sum + i  
    i = i + 1  
print("Sum =", sum)
```

Output :

Sum = 15

5. Infinite Loop (सावधानी).

```
i = 1  
while i <= 5 :  
    print(i)  
    # i = i + 1 (अगर यह line  
    नहीं लिखी तो loop कभी  
    खत्म नहीं होगा)
```

Result :

यह loop
infinite
रूप से
चलता रहेगा।

6. Uses of while Loop

- जब हमें पहले से नहीं पता कि loop कितनी बार चलेगा।
- Menu driven programs में।
- User input validate करने के लिए।
- File processing, game loops, data reading आदि में।

Note :

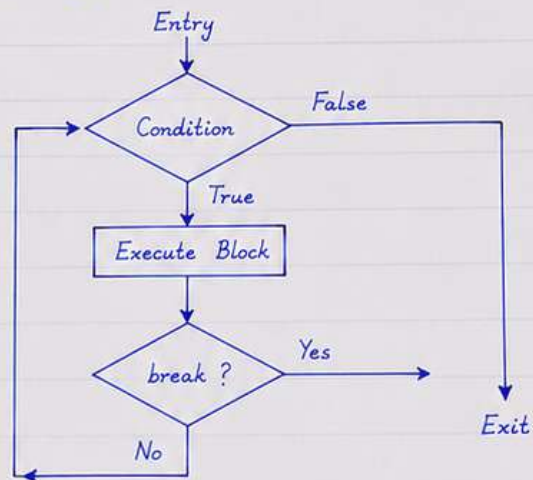
- while loop एक powerful tool है repeated tasks करने के लिए।
- हमेशा condition और update statement को सही तरह से design करें।
- गलत use करने पर infinite loop की संभावना रहती है।

Topic : break and continue

- Python में break और continue loop control statements होते हैं।
- इनका उपयोग loop के execution को control करने के लिए किया जाता है।
- break loop को terminate करता है जबकि continue current iteration को skip करके next iteration पर चला जाता है।

1. break Statement

- break statement का उपयोग loop को तुरंत terminate करने के लिए होता है।
- जब break execute होता है, तब loop का control loop के बाहर चला जाता है।
- break के बाद loop के अंदर का कोई भी code execute नहीं होता।



Example : Print numbers from 1 to 10,
but break the loop when $i == 5$

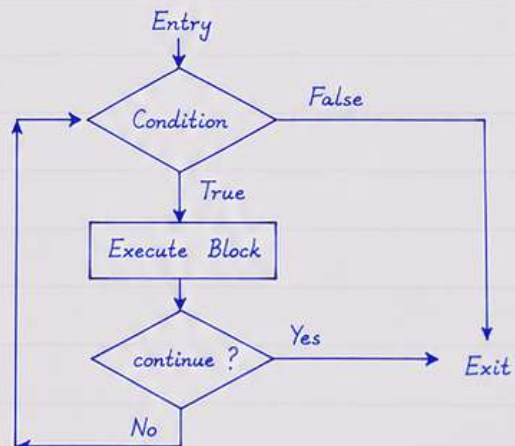
```
for i in range(1, 11):
    if i == 5:
        break
    print(i)
```

Output :

1
2
3
4

2. continue Statement

- continue statement का उपयोग current iteration को skip करने के लिए होता है।
- जब continue execute होता है, तब loop का control अगले iteration पर चला जाता है।
- continue के बाद loop के शेष statements उस iteration के लिए execute नहीं होते।



Example : Print numbers from 1 to 10,
but skip number 5

```
for i in range(1, 11):
    if i == 5:
        continue
    print(i)
```

Output :

1
2
3
4
6
7
8
9
10

Key Points :

- * break loop को पूरी तरह terminate करता है।
- * continue केवल current iteration को skip करता है।
- * break और continue केवल loop के अंदर ही use होते हैं (for, while)।

break vs continue		
Point	break	continue
कार्य	loop को terminate करता है	current iteration को skip करता है
Control कहाँ जाता है ?	loop के बाहर	loop के अगले iteration पर
आगे का code	loop के बाद का code execute होगा	उसी loop का अगला iteration execute होगा
उदाहरण	search में item मिलने पर loop रोकना	odd numbers को skip करके even print करना

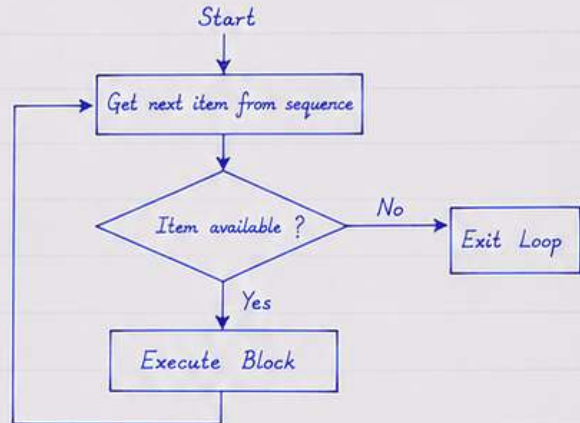
Topic : The for Loop

- for loop Python में एक controlled loop होता है जो किसी sequence (जैसे list, tuple, string, range आदि) के items पर iterate करता है।
- यह loop निश्चित बार चलता है, क्योंकि हमें पता होता है कि items कितने हैं।
- for loop का उपयोग तब करते हैं, जब हमें किसी sequence के प्रत्येक element पर कुछ operation करना हो।

1. Syntax

```
for variable in sequence :  
    statement(s)  
    (indented block)
```

- variable → हर iteration में next item को hold करता है।
- sequence → वह collection जिसके items पर loop चलता है।
- statement(s) → loop के अंदर execute होने वाले statements।



2. How it Works ?

- for loop sequence के पहले item से शुरू होता है।
- variable उस item को लेता है और block execute होता है।
- फिर sequence का अगला item लिया जाता है और process दोहराया जाता है।
- जब sequence में कोई item नहीं बचता, loop terminate हो जाता है।

Important :

for loop में हमें initialization, condition check और update अलग से लिखने की जरूरत नहीं होती, यह सब Python automatic handle करता है।

3. Example Programs

(A) 1 से 5 तक numbers print करना

```
for i in range(1, 6):  
    print(i)
```

Output :

1
2
3
4
5

(B) List के सभी items print करना

```
fruits = ['apple', 'mango', 'banana']  
for fruit in fruits:  
    print(fruit)
```

Output :

apple
mango
banana

(C) String के characters print करना

```
for ch in "Hello":  
    print(ch)
```

Output :

H
e
l
l
o

(D) 1 से 10 तक even numbers का sum

```
s = 0  
for i in range(2, 11, 2): # step = 2  
    s = s + i  
print("Sum =", s)
```

Output :

Sum = 30

4. range() Function

- range() for loop के साथ अक्सर use होता है।
- range(start, stop, step)
- start → default 0
- stop → इस number तक, लेकिन यह include नहीं होता।
- step → increment (default 1)

Examples :

- range(5) → 0, 1, 2, 3, 4
- range(1, 6) → 1, 2, 3, 4, 5
- range(2, 10, 2) → 2, 4, 6, 8

5. Key Points

- for loop known number of times iterate करता है।
- यह sequence के हर item को एक बार process करता है।
- for loop को list, tuple, string, range, set, dict आदि पर use किया जा सकता है।
- loop के अंदर break और continue का use किया जा सकता है।

Note : • for loop लिखना आसान और code को clean बनाता है, इसलिए इसे अधिकतर situations में prefer किया जाता है।

Unit - 4 : Collections

1. Introduction of Collection

- Collection एक framework है जो multiple objects (data) को single unit में store और manage करने के लिए use होता है।
- Collection का उपयोग data को organize, access, modify और process करने के लिए किया जाता है।
- यह Java में java.util package का part है और Java Collections Framework (JCF) के अंदर आता है।
- Collection data structures provide करता है जो dynamic और flexible होते हैं, यानी runtime पर size change हो सकती है।
- यह arrays से ज्यादा powerful होता है क्योंकि array का size fix होता है, जबकि collection में elements को आसानी से add, remove या modify किया जा सकता है।
- Collection में हम primitive data types (int, char, etc.) directly store नहीं कर सकते, यहाँ केवल objects store होते हैं। इसलिए primitive types के लिए wrapper classes (Integer, Character, आदि) का use किया जाता है।

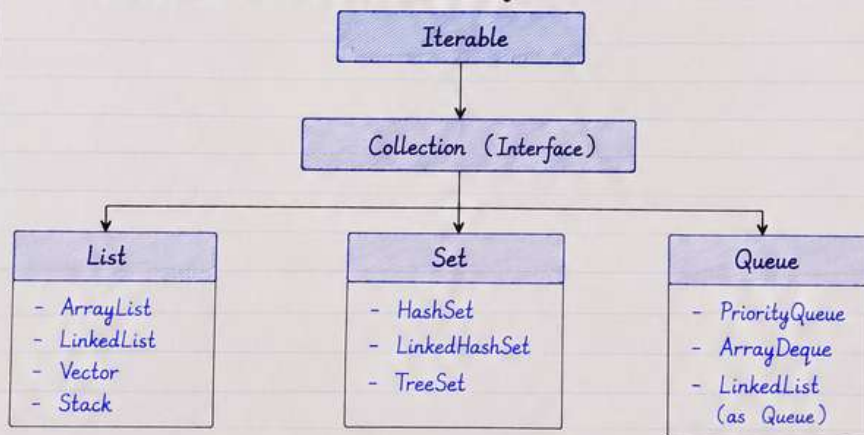
2. Why do we need Collection ?

- Multiple objects को एक साथ store करने के लिए।
- Data को efficient तरीके से manage करने के लिए।
- Dynamic size की आवश्यकता होती है (runtime पर size बदल सकता है)।
- Useful built-in methods provide करता है जैसे add(), remove(), search(), sort() आदि।
- Code को reusability और maintainability देता है।
- Real-world applications में data को handle करने के लिए collection बहुत useful है।

Key Point

- Collection = Group of Objects
- Part of java.util package
- Provides dynamic, flexible and efficient way to store and manage data.

3. Collection Framework Hierarchy



Note :

- Iterable is the root interface.
- Collection extends Iterable.
- List, Set और Queue Collection के main sub-interfaces हैं।
- इन interfaces के कई implementations (classes) होते हैं, जिन्हें हम requirement के अनुसार use करते हैं।

4. Important Interfaces with Short Description

Interface	Description
Collection	यह root interface है जो सभी collections के लिए common methods define करता है।
List	Ordered collection, duplicate elements allow करता है।
Set	Unique elements store करता है (duplicate allow नहीं)।
Queue	FIFO (First In First Out) order में elements store करता है।
Map (not child of Collection)	Key-Value pairs store करता है (यह Collection interface को extend नहीं करता)।

5. Common Methods (Collection Interface)

Method	Description
add(E e)	Collection में element add करता है।
remove(Object o)	Specified element को remove करता है।
size()	Total number of elements return करता है।
isEmpty()	Check करता है कि collection empty है या नहीं।
contains(Object o)	Check करता है कि element present है या नहीं।
clear()	सभी elements को remove करता है।
iterator()	Elements को iterate करने के लिए iterator return करता है।

6. Advantages of Collection

- Dynamic size
- Built-in useful methods
- Easy to use and implement
- Increases code reusability
- Better performance and efficient data handling
- Widely used in real-world applications

7. Summary

- Collection Java में objects को store और manage करने का एक powerful framework है, जो developer को data को efficiently handle करने में मदद करता है।

Topic : What is List ? 7 Major Feature of List

1. What is List ?

- List एक ordered collection (क्रमबद्ध संग्रह) होता है, जिसमें multiple elements को एक single unit में store किया जाता है।
- List आमतौर पर dynamic होती है, यानी इसका size runtime पर बदल सकता है।
- List की सबसे बड़ी खासियत यह है कि यह elements को insertion order (जिस क्रम में add किया गया है) उसी क्रम में maintain करती है।
- Java में List एक interface है जो java.util package के अंतर्गत आता है और यह Collection Framework का part है।
- List duplicate elements को allow करती है, और same element multiple times store किया जा सकता है।
- List index के through access provide करती है, जिससे हम किसी भी element को उसके position (index) द्वारा आसानी से प्राप्त कर सकते हैं।

List Representation (Example)

Index →	0	1	2	3	4
Elements →	10	20	30	40	50

List stores elements in an ordered way and each element can be accessed using its index.

Key Point

- List = Ordered collection
- Part of Collection Framework
- Maintains insertion order
- Allows duplicate elements
- Index based access
- Dynamic in nature

2. 7 Major Features of List

(1) Ordered Collection

- List elements को एक निश्चित क्रम (insertion order) में store करती है।
- हम जिस order में elements add करते हैं, उसी order में यह maintain रहती है।

Example : [10, 20, 30, 40]
(Same order will be maintained)

(2) Allows Duplicates

- List में duplicate elements allow होते हैं।
- एक ही value को multiple times add किया जा सकता है।

Example : [10, 20, 10, 30, 20]

(3) Index Based Access

- List में हर element का एक index (position) होता है, जो 0 से start होता है।
- हम index के द्वारा किसी भी element को आसानी से access कर सकते हैं।

Example : List.get(2) → तीसरा element प्राप्त होगा।

(4) Dynamic Size

- List का size fixed नहीं होता।
- हम elements को add या remove करके इसका size बढ़ा या घटा सकते हैं।
- यह runtime पर automatically manage होती है।

(5) Supports Multiple Data Types

- List में विभिन्न प्रकार के data types के elements store किए जा सकते हैं (Integer, String, Object, etc.)
- यह generic के साथ type safe भी बन सकती है।

Example : List<String> names = new ArrayList<>();

(6) Provides Useful Methods

- List interface कई built-in methods provide करती है, जैसे add(), remove(), get(), set(), size(), contains(), indexOf(), आदि।
- ये methods data को manage करने में बहुत मदद करते हैं।

(7) Part of Collection Framework

- List, Java Collection Framework का एक important interface है (java.util package में)।
- इसके कुछ common implementations हैं:

ArrayList, LinkedList, Vector, Stack
(ये सभी List interface को implement करते हैं।)

Advantages of List

- Data को ordered manner में store करती है।
- Duplicate elements allow करती है।
- Index based access provide करती है।
- Dynamic और flexible होती है।
- Built-in methods के कारण operations आसान हो जाते हैं।

Conclusion

List एक powerful collection है जो ordered, dynamic और flexible होती है। यह duplicate elements को allow करती है और index के द्वारा access provide करती है। Java programming में List का उपयोग बहुत व्यापक रूप से किया जाता है।

Topic : Modifying Lists

- List ek mutable data structure hai, yani iske elements ko modify (change) kiya jã sakta hai।
- Modifying ka matlab existing elements ko update karna, naye elements add karna, ya multiple elements ko ek saath change karna hota hai।

1. Multiple elements change karna

- List me ek se zyada elements ko ek saath change karne ke लिए indexing aur slicing ka use kar sakte hai।
- Isme hum specific range ke elements ko nayi values se replace kar sakte hai।

Example 1 : Indexing se multiple change

```
# original list
marks = [10, 20, 30, 40, 50]
# multiple elements change karna
marks[1] = 25
marks[3] = 45
print(marks)
# Output: [10, 25, 30, 45, 50]
```

Example 2 : Slicing se multiple change

```
nums = [1, 2, 3, 4, 5, 6]
# index 1 se 3 tak ke elements change
nums[1:4] = [20, 30, 40]
print(nums)
# Output: [1, 20, 30, 40, 5, 6]
```

Key Points

- Index se single element change hota hai।
- Slicing se multiple elements change kiye जा सकते हैं।
- Naya list ka size same ho zaruri hai।

2. append()

- append() method list ke end me ek single element ko add करता है।
- Yeh existing list me naya element jodta है और original list ko modify करता है।

Syntax

```
list.append(element)
```

Example

```
fruits = ['apple', 'banana', 'mango']
fruits.append('orange')
print(fruits)
# Output: ['apple', 'banana', 'mango', 'orange']
```

Key Points

- append() sirf ek element add करता है।
- Yeh element hamesha list ke end me add hota है।
- Original list modify hoti है।

3. insert()

- insert() method list me kisi specific index (position) par element ko add करता है।
- Isse hum element ko list ke beech me bhi insert kar sakte हैं।

Syntax

```
list.insert(index, element)
```

Example

```
numbers = [10, 20, 30, 40]
numbers.insert(2, 25) # index 2 par 25 insert
print(numbers)
# Output: [10, 20, 25, 30, 40]
```

Key Points

- insert() me index dena jaruri hota है।
- Element us index par add hota है, aur baaki elements apni jagah se shift ho jate हैं।

4. extend()

- extend() method ek list ke multiple elements ko dusri iterable (list, tuple, set etc.) se add करता है।
- Yeh append() se alag hota है, kyunki append() ek single element add करता है, jabki extend() puri iterable ke sabhi elements ko add करता है।

Syntax

```
list.extend(iterable)
```

Example

```
list1 = [1, 2, 3]
list2 = [4, 5, 6]
list1.extend(list2)
print(list1)
# Output: [1, 2, 3, 4, 5, 6]
```

Key Points

- extend() multiple elements add करता है।
- Argument ek iterable (list, tuple, set, etc.) hota है।
- Original list modify hoti है।
- Agar hum list ko append() se add karen, to poori list ek element ke रूप में जुड़ जाएगी।

append() vs extend()

```
l = [1, 2, 3]
l.append([4, 5]) # [1, 2, 3, [4, 5]]
l = [1, 2, 3]
l.extend([4, 5]) # [1, 2, 3, 4, 5]
```

Note :

- List mutable hoti है, isliye in methods se latest changes original list par hi apply होते हैं।
- In methods ka use karke hum data ko easily manage aur modify kar sakte हैं।

Topic : Removing elements (remove(), pop(), del, clear())

- List se unwanted elements ko remove karne ke liye Python me kai methods aur statements available hote hain.
- Hum element ko value se, index se, ya puri list ko ek saath bhi remove kar sakte hain.
- Yeh operations list ko modify (change) karte hain, isliye yeh original list par hi apply hote hain.

1. remove()

- remove() method list me diye gaye value (element) ke first occurrence ko remove karta hai.
- Agar value list me multiple times hai, to sirf pehla element remove hota hai.
- Agar value list me nahi milti, to ValueError aata hai.

Syntax :

```
list.remove(value)
```

Example :

```
fruits = ['apple', 'banana', 'apple', 'mango']
fruits.remove('apple')
print(fruits)
# Output: ['banana', 'apple', 'mango']
```

2. pop()

- pop() method default roop se list ke last element ko remove karta hai aur us element ko return bhi karta hai.
- Agar index diya jaye, to us index par maujood element ko remove karke return karta hai.

Syntax :

```
list.pop()      # last element remove
list.pop(index) # specific index element remove
```

Example :

```
nums = [10, 20, 30, 40]
x = nums.pop()      # last element remove
print(x)            # Output: 40
print(nums)          # Output: [10, 20, 30]
y = nums.pop(1)      # index 1 par element remove
print(y)             # Output: 20
print(nums)          # Output: [10, 30]
```

3. del (delete statement)

- del ek Python statement hai, jo list ke kisi specific index ke element ko remove karne ke liye use hota hai.
- Isse single element, multiple elements (slicing) ya poori list bhi delete ki ja sakti hai.

Syntax :

```
del list[index]
del list[start : end]
del list      # puri list delete
```

Example :

```
nums = [10, 20, 30, 40, 50]
del nums[2]      # index 2 (30) remove
print(nums)      # [10, 20, 40, 50]
del nums[1:3]    # index 1 se 2 tak remove
print(nums)      # [10, 50]
# del nums      # puri list delete (phir use nahi kar sakte)
```

4. clear()

- clear() method list ke sabhi elements ko remove kar deta hai, lekin list khud delete nahi hoti.
- Operation ke baad list empty ho jati hai (size = 0).

Syntax :

```
list.clear()
```

Example :

```
colors = ['red', 'green', 'blue']
colors.clear()
print(colors)      # Output: []
print(len(colors)) # Output: 0
```

Note :

- remove() → value se element remove karta hai (first occurrence).
- pop() → index se element remove karta hai aur return karta hai.
- del → index, slice ya puri list delete kar sakte hain.
- clear() → sirf list ko empty karta hai (list rehti hai).

5. Difference between remove(), pop(), del and clear()

Method / Statement	Kis basis par remove hota hai?	Return value	Multiple elements remove kar sakte hain?	List rehti hai?	Error (agar element nahi mila)
remove()	Value (first occurrence)	No	No (sirf pehla)	Yes	ValueError
pop()	Index (default last)	Yes (removed element)	No (ek baar me ek)	Yes	IndexError (invalid index)
del	Index / Slice / Puri list	No	Yes (slicing se)	Yes (unless full list delete ki)	IndexError (invalid index)
clear()	Sabhi elements	No	Yes (all at once)	Yes (empty list)	No error

Topic : List Operators

List operators ka use lists ke saath different operations perform karne ke liye hota hai. In operators ke dwara hum lists ko combine, repeat, compare aur check kar sakte hain.

1. + (Concatenation)

- '+' operators do lists ko jod (concatenate) karne ke liye use hota hai.
- Isse ek nayi list banti hai jisme dono lists ke elements same order me aate hain.

Example :

```
list1 = [1, 2, 3]
list2 = [4, 5, 6]
list3 = list1 + list2 # concatenate
print(list3)
# Output : [1, 2, 3, 4, 5, 6]
```

2. * (Repetition)

- '*' operator list ke elements ko specified number of times repeat karne ke liye use hota hai.
- Isse ek nayi list banti hai jisme original list ke saare elements multiple times hote hain.

Example :

```
list1 = [10, 20, 30]
list2 = list1 * 3 # repeat 3 times
print(list2)
# Output : [10, 20, 30, 10, 20, 30, 10, 20, 30]
```

3. in (Membership Operator)

- 'in' operator check karta hai ki koi element list me present hai ya nahi.
- Agar element list me milta hai to True return hota hai warna False.

Example :

```
fruits = ['apple', 'banana', 'mango']
print('banana' in fruits) # True
print('orange' in fruits) # False
```

4. not in (Membership Operator)

- 'not in' operator check karta hai ki koi element list me present nahi hai.
- Agar element list me nahi hota to True return hota hai warna False.

Example :

```
fruits = ['apple', 'banana', 'mango']
print('orange' not in fruits) # True
print('banana' not in fruits) # False
```

5. ==, != (Equality Operators)

- '==' operator do lists ki equality check karta hai.
- Yeh True return karta hai agar dono lists ke elements same order me aur same values ke saath hote hain.
- '!=' operator True return karta hai agar dono lists same nahi hote.

Example :

```
list1 = [1, 2, 3]
list2 = [1, 2, 3]
list3 = [3, 2, 1]
print(list1 == list2) # True
print(list1 != list3) # True
```

6. <, > (Comparison Operators)

- '<' aur '>' operators lists ko lexicographical (dictionary) order me compare karte hain.
- Comparison left se right hota hai, jahan pehla difference milta hai uske basis par result decide hota hai.
- Yeh operators tab use hote hain jab lists ke elements comparable (jaise numbers ya strings) hote hain.

Example :

```
list1 = [1, 2, 3]
list2 = [1, 2, 4]
print(list1 < list2) # True (3 < 4)
print(list2 > list1) # True
list3 = [1, 2, 3]
print(list1 < list3) # False (both equal)
```

Summary Table

Operator	Name	Purpose	Example	Result
+	Concatenation	Do lists ko jodna	[1,2] + [3,4]	[1,2,3,4]
*	Repetition	List ko repeat karna	[1,2] * 3	[1,2,1,2,1,2]
in	Membership	Element present hai?	3 in [1,2,3]	True
not in	Membership	Element present nahi hai?	5 not in [1,2,3]	True
==	Equality	Dono lists same?	[1,2] == [1,2]	True
!=	Inequality	Dono lists different?	[1,2] != [2,1]	True
<	Less than	Lexicographical compare	[1,2] < [1,3]	True
>	Greater than	Lexicographical compare	[2,1] > [1,9]	True

Key Points :

- '+' aur '*' se hamesha nayi list banti hai, original list change nahi hoti.
- 'in' aur 'not in' membership check ke liye use hote hain.
- '==' aur '!=' order ke saath compare karte hain.
- '<' aur '>' lexicographical order me comparison karte hain.
- In operators ka use real-world applications me data ko search, combine aur compare karne ke liye hota hai.

Topic : Important List Functions

- Python me list ke sath kaam karne ke liye kai built-in functions available hote hain.
- In functions ka use list se related common operations ko easily perform karne ke liye hota hai.
- Yeh functions time bachate hain aur code ko simple aur readable banate hain.

1. len()

- len() function list me total number of elements (length) batata hai.
- Yeh hume list ka size (kitne elements hain) janane ke liye use hota hai.

Example :

```
nums = [10, 20, 30, 40, 50]
print(len(nums))    # Output : 5
fruits = ['apple', 'banana', 'mango']
print(len(fruits))  # Output : 3
```

2. max()

- max() function list me se sabse badi value (maximum element) return karta hai.
- Yeh mainly numeric data ke liye use hota hai, lekin strings me bhi lexicographical order ke basis par kaam karta hai.

Example :

```
nums = [10, 50, 30, 20]
print(max(nums))    # Output : 50
names = ['ram', 'shyam', 'mohan']
print(max(names))   # Output : 'shyam'
```

3. min()

- min() function list me se sabse chhoti value (minimum element) return karta hai.
- Yeh bhi numeric data ke liye use hota hai aur strings me lexicographical order follow karta hai.

Example :

```
nums = [10, 5, 30, 20]
print(min(nums))    # Output : 5
names = ['ram', 'shyam', 'mohan']
print(min(names))   # Output : 'mohan'
```

4. sum()

- sum() function list ke sabhi numeric elements ka total (sum) return karta hai.
- Yeh sirf numbers (int, float) ke liye work karta hai.
- Agar list me non-numeric value hogi to TypeError aayega.

Example :

```
nums = [10, 20, 30, 40]
print(sum(nums))    # Output : 100
marks = [78, 82, 90, 85]
print(sum(marks))   # Output : 335
```

5. sorted()

- sorted() function list ke elements ko sorted order (ascending by default) me return karta hai.
- Yeh original list ko modify nahi karta, balki ek nayi sorted list return karta hai.

Example :

```
nums = [40, 10, 30, 20]
print(sorted(nums)) # Output : [10, 20, 30, 40]
names = ['mango', 'apple', 'banana']
print(sorted(names)) # Output : ['apple', 'banana', 'mango']
print(nums)          # Original list unchanged
                    # Output : [40, 10, 30, 20]
```

6. reversed()

- reversed() function list ke elements ko reverse order me return karta hai.
- Yeh ek iterator return karta hai, isliye result ko dekhne ke liye hume use list() me convert karna padta hai.

Example :

```
nums = [10, 20, 30, 40]
rev = reversed(nums)
print(list(rev))    # Output : [40, 30, 20, 10]
names = ['a', 'b', 'c']
print(list(reversed(names))) # Output : ['c', 'b', 'a']
```

7. list()

- list() function ka use kisi other iterable (jaise string, tuple, set, range, etc.) ko list me convert karne ke liye hota hai.
- Yeh nayi list banata hai.

Example :

```
s = 'hello'
print(list(s))      # Output : ['h', 'e', 'l', 'l', 'o']
t = (1, 2, 3)
print(list(t))      # Output : [1, 2, 3]
r = range(1, 6)
print(list(r))      # Output : [1, 2, 3, 4, 5]
```

Key Points :

- len() → list ki length batata hai.
- max() → sabse badi value return karta hai.
- min() → sabse chhoti value return karta hai.
- sum() → sabhi elements ka total deta hai.
- sorted() → sorted nayi list return karta hai (original list change nahi hoti).
- reversed() → elements ko reverse order me deta hai (iterator return karta hai).
- list() → kisi iterable ko list me convert karta hai.
- Yeh functions list par kaam karna simple, efficient aur easy bana dete hain.

Topic : Tuples

- Tuple Python में एक built-in data type है, जो multiple elements को
- एक single unit में store करने के लिए use होता है।
- Tuple ordered collection होती है और immutable (जिसे change नहीं किया जा सकता) होती है।
- Tuple में different data types के elements store किए जा सकते हैं।
- Tuple को round brackets () में लिखा जाता है और elements comma (,) से separate होती है।

1. Creating a Tuple

- Tuple को directly values देकर बनाया जाता है।
- Single element tuple के लिए value के बाद comma (,) लगाना जरूरी होता है, नहीं तो वह tuple नहीं माना जाएगा।

Example :

```
t1 = (10, 20, 30)           # tuple of integers
t2 = ("apple", "banana", "mango") # tuple of strings
t3 = (10, "Hello", 3.5, True) # mixed data types
t4 = (50,)                  # single element tuple
t5 = ()                     # empty tuple

print(t1)                   # (10, 20, 30)
print(type(t1))             # <class 'tuple'>
```

4. Tuple Operations

- Tuple पर कई operations perform किए जा सकते हैं जैसे concatenation (+), repetition (*), membership (in), length (len()), आदि।

Example :

```
t1 = (1, 2, 3)
t2 = (4, 5)
print(t1 + t2) # (1, 2, 3, 4, 5) (concatenation)
print(t1 * 2)  # (1, 2, 3, 1, 2, 3) (repetition)
print(2 in t1) # True (membership)
print(len(t1)) # 3 (length)
```

6. Tuple Methods

- Tuple में list की तरह change करने वाले methods नहीं होते, क्योंकि यह immutable है।
- लेकिन इसमें कुछ useful methods available होते हैं।

Method	Description	Example
count(x)	कितनी बार value आती है, यह return करता है।	t.count(10)
index(x)	पहली occurrence का index return करता है।	t.index(20)

9. When to Use Tuple ?

- जब data change नहीं होना चाहिए तब Tuple use करें।
- Functions से multiple values return करने के लिए।
- Fixed data collection के लिए (जैसे coordinates, days of week, etc.)
- Data की integrity बनाए रखने के लिए।

2. Accessing Tuple Elements

- Tuple के elements को index के द्वारा access किया जाता है।
- Indexing 0 से start होती है और negative indexing भी use की जा सकती है।

Example :

```
t = (10, 20, 30, 40, 50)
print(t[0])    # 10
print(t[2])    # 30
print(t[-1])   # 50 (last element)
print(t[-2])   # 40
```

3. Tuple Slicing

- Tuple में slicing का use करके elements की range प्राप्त की जा सकती है।

Example :

```
t = (10, 20, 30, 40, 50, 60)
print(t[1:4])  # (20, 30, 40)
print(t[:3])   # (10, 20, 30)
print(t[3:])   # (40, 50, 60)
print(t[::2])  # (10, 30, 50)
```

5. Immutability of Tuple

- Tuple immutable होती है, यानी एक बार बन जाने के बाद इसके elements को change, add या remove नहीं किया जा सकता।
- अगर हम किसी element को modify करने की कोशिश करते हैं, तो TypeError आता है।

Example :

```
t = (10, 20, 30)
t[0] = 100 # TypeError: 'tuple' object does not support
            # item assignment
```

Note :

- Tuple immutable होने के कारण यह data को secure रखने के लिए useful होती है।
- इसे functions में multiple values return करने के लिए भी use किया जाता है।

7. Advantages of Tuple

- Immutable होने के कारण data safe रहता है।
- List की तुलना में memory efficient होती है।
- Functions में multiple values return करने के लिए useful है।
- Heterogeneous (different data types) data store कर सकती है।

8. Disadvantages of Tuple

- Elements को change, add या remove नहीं कर सकते।
- Dynamic operations के लिए suitable नहीं है (list बेहतर होती है)।
- कम methods available हैं।

Topic : Sets

- Set Python में एक built-in data type है, जो unique (distinct) elements का unordered collection होता है।
- Set में duplicate values नहीं होती हैं।
- Set mutable होता है, इसलिए इसमें elements को add या remove किया जा सकता है।
- Set का use membership testing, duplicate हटाने और mathematical set operations (union, intersection, etc.) के लिए किया जाता है।

Key Points :

- Unordered collection
- Unique elements only
- Mutable
- Indexed नहीं होता।
- {} (curly braces) में लिखा जाता है।
- Mixed data types allowed

1. Creating a Set

- Set को curly braces {} में elements रखकर बनाया जाता है।
- या फिर set() function का use किया जाता है।

Example :

```
s1 = {10, 20, 30, 40}
s2 = {'apple', 'banana', 'mango'}
s3 = set([1, 2, 3, 3, 2, 1]) # duplicates remove
print(s3) # Output : {1, 2, 3}
```

2. Accessing Elements

- Set indexed नहीं होता, इसलिए हम index के द्वारा directly element access नहीं कर सकते।
- हम केवल membership operator (in, not in) का use करके check कर सकते हैं कि कोई element set में है या नहीं।

Example :

```
s = {10, 20, 30}
print(20 in s) # True
print(40 in s) # False
```

3. Adding Elements (add, update)

- add() method का use एक single element को add करने के लिए होता है।
- update() method का use multiple elements (iterable) को add करने के लिए होता है।

Example :

```
s = {10, 20, 30}
s.add(40) # {10, 20, 30, 40}
s.update([50, 60]) # {10, 20, 30, 40, 50, 60}
print(s)
```

4. Removing Elements (remove, discard, pop, clear)

- remove() - specified element remove करता है (अगर element नहीं है तो Error आता है)।
- discard() - specified element remove करता है (अगर element नहीं है तो Error नहीं आता)।
- pop() - set से कोई भी एक random element remove करता है।
- clear() - set के सभी elements को remove कर देता है (खाली set)।

Example :

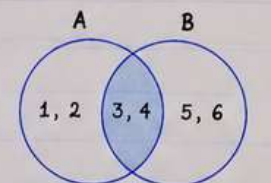
```
s = {10, 20, 30, 40}
s.remove(20) # {10, 30, 40}
s.discard(50) # No error
print(s.pop()) # removes and returns a random element
s.clear() # set() -> empty set
```

5. Set Operations

Operation	Symbol / Method	Description	Example
Union	or union()	सभी elements	A B
Intersection	& or intersection()	Common elements	A & B
Difference	- or difference()	A में है, B में नहीं	A - B
Symmetric Diff.	^ or symmetric_difference()	A या B में है, लेकिन दोनों में नहीं।	A ^ B

Example :

```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}
print(A | B) # {1, 2, 3, 4, 5, 6}
print(A & B) # {3, 4}
print(A - B) # {1, 2}
print(A ^ B) # {1, 2, 5, 6}
```



A | B = {1, 2, 3, 4, 5, 6}
A & B = {3, 4}

6. Useful Set Functions

Function	Description	Example
len(s)	Total elements	len({1, 2, 3})
min(s)	Minimum element	min({3, 1, 2})
max(s)	Maximum element	max({3, 1, 2})
sum(s)	Sum of elements	sum({1, 2, 3})
sorted(s)	Sorted list (returns list)	sorted({3, 1, 2})
set(iterable)	Convert iterable to set	set([1, 2, 2, 3])

7. Properties of Set

- Unordered - elements ka order fix नहीं होता।
- Unique Elements - duplicate values नहीं होती।
- Mutable - elements को add या remove किया जा सकता है।
- Heterogeneous - different data types रख सकते हैं।
- Indexing नहीं - index के द्वारा element access नहीं कर सकते।
- Fast Membership Test - in और not in बहुत fast काम करते हैं।

8. When to Use Set ?

- जब हमें unique values store करनी हों।
- Duplicate values हटाने के लिए।
- Membership test (element है या नहीं) के लिए।
- Mathematical set operations के लिए (union, intersection, etc.)।
- Data को efficiently manage करने के लिए।
- List से duplicate remove करने के लिए -> set(List)।

Note :

- Set एक powerful data structure है, जो simple syntax के साथ data को unique और organized रखने में मदद करता है।

Topic : Dictionaries

- Dictionary Python में एक built-in data type है, जो data को key-value pairs के form में store करता है।
- Dictionary unordered (Python 3.7+ में insertion order maintain होती है), mutable और indexed (key के द्वारा) होती है।
- Dictionary का use real-world data को represent करने के लिए किया जाता है, जैसे student details, employee records, etc.
- Dictionary में keys unique होती हैं, लेकिन values duplicate हो सकती हैं।

1. Creating a Dictionary

- Dictionary को curly braces {} में key : value pairs लिखकर बनाया जाता है।
- Key किसी भी immutable type (string, number, tuple, etc.) की हो सकती है।

Example :

```
d1 = {'name': 'Rahul', 'age': 20, 'city': 'Delhi'}
d2 = { 1: 'one', 2: 'two', 3: 'three' }
d3 = dict(name = 'Sneha', age = 22)
print(d1) # {'name': 'Rahul', 'age': 20, 'city': 'Delhi'}
```

2. Accessing Elements

- Dictionary के elements को key के द्वारा access किया जाता है।
- अगर key exist नहीं करती तो KeyError आता है।

Example :

```
d = {'name': 'Rahul', 'age': 20, 'city': 'Delhi'}
print(d['name']) # Rahul
print(d['age']) # 20
print(d.get('city')) # Delhi
print(d.get('course', 'Not Found')) # Not Found
```

3. Adding and Updating Elements

- नया key-value pair add करने के लिए assignment operator (=) का use करते हैं।
- अगर key पहले से exist करती है तो उसकी value update हो जाती है।

Example :

```
d = {'name': 'Rahul', 'age': 20}
d['city'] = 'Delhi' # new key add
d['age'] = 21 # existing key update
print(d) # {'name': 'Rahul', 'age': 21, 'city': 'Delhi'}
```

4. Removing Elements

- किसी key-value pair को remove करने के लिए del, pop() या popitem() methods का use किया जाता है।

Example :

```
d = {'name': 'Rahul', 'age': 20, 'city': 'Delhi'}
del d['age'] # remove by key
d.pop('city') # remove and return value
d.popitem() # remove last inserted item
print(d) # {'name': 'Rahul'}
```

5. Useful Dictionary Methods

Method	Description	Example
keys()	सभी keys return करता है।	d.keys()
values()	सभी values return करता है।	d.values()
items()	सभी key-value pairs return करता है।	d.items()
get()	key की value return करता है (safe)।	d.get('name')
update()	दूसरे dictionary से update करता है।	d.update(d2)
pop()	दी गई key को remove करता है।	d.pop('age')
popitem()	आखिरी inserted item remove करता है।	d.popitem()
clear()	सभी items remove करता है।	d.clear()
copy()	Dictionary की एक copy बनाता है।	d.copy()

6. Dictionary Operations

- Dictionary पर membership operators (in, not in), length (len()), और comparison (==, !=) का use किया जा सकता है।

Example :

```
d = {'a': 10, 'b': 20, 'c': 30}
print('a' in d) # True
print('x' not in d) # True
print(len(d)) # 3
d2 = {'a': 10, 'b': 20, 'c': 30}
print(d == d2) # True
```

7. Nested Dictionary

- Dictionary के अंदर एक और dictionary भी हो सकती है।
- इसका use complex data (जैसे multiple students की details) के लिए किया जाता है।

Example :

```
students = {
    's1': {'name': 'Rahul', 'age': 20},
    's2': {'name': 'Sneha', 'age': 22}
}
print(students['s1']['name']) # Rahul
```

8. Advantages of Dictionary

- Data को key-value pairs में store करने के लिए efficient है।
- Fast access (key के द्वारा) मिलता है।
- Real-world data को represent करने के लिए useful है।
- Flexible और mutable होता है।

Key Points :

- Dictionary unordered, mutable और indexed (by key) होती है।
- Keys unique होती हैं, values duplicate हो सकती हैं।
- Key के द्वारा data को access, add, update और remove किया जाता है।
- Dictionary real-world applications में बहुत useful है।

Topic : Sorting Dictionaries

- Dictionary Python में key-value pairs का unordered collection होता है, इसलिए यह default रूप से sorted नहीं होता।
- कई बार हमें dictionary के keys या values को sorted order में देखना होता है।
- Sorting के लिए हम built-in functions जैसे sorted() और कुछ methods का use करते हैं।
- हम keys के अनुसार, values के अनुसार (या दोनों) dictionary को sort कर सकते हैं।

1. Sorting by Keys (Ascending Order)

- Dictionary को keys के अनुसार ascending order (A → Z) में sort करने के लिए sorted() का use किया जाता है।
- यह एक list of tuples return करता है, इसलिए चाहें तो इसे dict() में convert कर सकते हैं।

Example :

```
d = {'banana': 3, 'apple': 5, 'mango': 1}
sorted_d = dict(sorted(d.items()))
print(sorted_d)
# Output : {'apple': 5, 'banana': 3, 'mango': 1}
```

2. Sorting by Keys (Descending Order)

- Keys को descending order (Z → A) में sort करने के लिए sorted() में reverse = True देते हैं।

Example :

```
d = {'banana': 3, 'apple': 5, 'mango': 1}
sorted_d = dict(sorted(d.items(), reverse=True))
print(sorted_d)
# Output : {'mango': 1, 'banana': 3, 'apple': 5}
```

3. Sorting by Values (Ascending Order)

- Values के अनुसार sort करने के लिए sorted() में key parameter का use करते हैं (item[1] → value को consider करता है)।
- इससे dictionary value के increasing order में sort हो जाती है।

Example :

```
d = {'banana': 3, 'apple': 5, 'mango': 1}
sorted_d = dict(sorted(d.items(), key=lambda item: item[1]))
print(sorted_d)
# Output : {'mango': 1, 'banana': 3, 'apple': 5}
```

4. Sorting by Values (Descending Order)

- Values को descending order में sort करने के लिए reverse = True का use करते हैं।

Example :

```
d = {'banana': 3, 'apple': 5, 'mango': 1}
sorted_d = dict(sorted(d.items(),
    key=lambda item: item[1], reverse=True))
print(sorted_d)
# Output : {'apple': 5, 'banana': 3, 'mango': 1}
```

5. Using sorted() Without dict()

- अगर केवल sorted order में items को display करना है, तो dict() में convert करना जरूरी नहीं है।
- sorted() एक list of tuples return करता है।

Example :

```
d = {'banana': 3, 'apple': 5, 'mango': 1}
print(sorted(d.items()))
# Output : [('apple', 5), ('banana', 3), ('mango', 1)]
```

6. Sorting with Custom Logic

- हम अपनी जरूरत के अनुसार custom sorting भी कर सकते हैं जैसे keys की length के अनुसार, या values के किसी condition के अनुसार।

Example : (Keys की length के अनुसार sort)

```
d = {'python': 50, 'c': 70, 'java': 60}
sorted_d = dict(sorted(d.items(), key=lambda item: len(item[0])))
print(sorted_d)
# Output : {'c': 70, 'java': 60, 'python': 50}
```

Key Points :

- Dictionary default रूप से unordered होता है।
- sorted() function dictionary के items को sort करने के लिए use किया जाता है।
- Keys या values - दोनों के basis पर sorting की जा सकती है।
- sorted() हमेशा list of tuples return करता है।
- अगर sorted result को फिर से dictionary में चाहिए तो dict() का use करें।
- reverse = True देने से descending order में sort होता है।
- Custom logic के लिए lambda function का use किया जा सकता है।

Summary Example :

```
d = {'banana': 3, 'apple': 5, 'mango': 1}
# by keys (ascending)
print(dict(sorted(d.items())))
# by keys (descending)
print(dict(sorted(d.items(), reverse=True)))
# by values (ascending)
print(dict(sorted(d.items(), key=lambda x: x[1])))
# by values (descending)
print(dict(sorted(d.items(), key=lambda x: x[1], reverse=True)))
```

Topic : Copying Collections

- Python में collections (List, Tuple, Set, Dictionary) को copy करने के लिए different methods available हैं।
- Copying का use तब किया जाता है जब हमें original collection को modify किए बिना उसका एक duplicate बनाना होता है।
- अगर हम सिर्फ assignment operator (=) का use करते हैं, तो नया variable same object को refer करता है।
- इसलिए किसी एक में change करने पर दोनों में change आ जाता है।
- इस problem से बचने के लिए copy(), list(), dict(), set(), slice [:] आदि methods का use करते हैं।

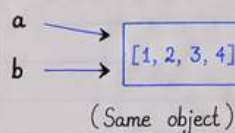
1. Assignment vs Copying (Important)

- = operator केवल reference copy करता है, नया object नहीं बनाता।
- दोनों variables एक ही memory location को refer करते हैं।

Example : (Assignment)

```
a = [1, 2, 3]
b = a
b.append(4)
print(a)      # [1, 2, 3, 4]
print(b)      # [1, 2, 3, 4]
```

Memory Diagram :



Note : a और b दोनों एक ही object को refer कर रहे हैं, इसलिए किसी एक में change करने पर दोनों में change हो जाता है।

2. Copying a List

- List को copy करने के लिए copy(), list() या slicing [:] का use किया जा सकता है।
- यह methods एक नया independent list object बनाते हैं।

Example :

```
a = [10, 20, 30]
b = a.copy()      # using copy()
c = list(a)       # using list()
d = a[:]          # using slicing
b.append(40)
print(a)          # [10, 20, 30] (original unchanged)
print(b)          # [10, 20, 30, 40] (new list modified)
```

Note : a, b, c, d सभी अलग-अलग list objects हैं।

3. Copying a Tuple

- Tuple immutable होता है, इसलिए आमतौर पर copy करने की जरूरत नहीं होती।
- फिर भी हम tuple() constructor या slicing से इसका copy बना सकते हैं।

Example :

```
t1 = (1, 2, 3)
t2 = tuple(t1)
t3 = t1[:]
print(t2)      # (1, 2, 3)
print(t3)      # (1, 2, 3)
```

Note : Tuple immutable होता है, इसलिए copy करने पर भी data same रहता है, लेकिन नया object बना जाता है।

4. Copying a Set

- Set को copy करने के लिए copy() या set() constructor का use किया जाता है।
- इससे एक नया independent set object बनता है।

Example :

```
s1 = {1, 2, 3}
s2 = s1.copy()
s3 = set(s1)
s2.add(4)
print(s1)        # {1, 2, 3}
print(s2)        # {1, 2, 3, 4}
```

Note : s1 और s2 अलग-अलग set objects हैं।

5. Copying a Dictionary

- Dictionary को copy करने के लिए copy(), dict() या dict comprehension का use किया जा सकता है।
- इससे नया dictionary बनता है, जो original से independent होता है।

Example :

```
d1 = {'name': 'Rahul', 'age': 20}
d2 = d1.copy()
d3 = dict(d1)
d4 = {k: v for k, v in d1.items()}
d2['age'] = 22
print(d1)      # {'name': 'Rahul', 'age': 20}
print(d2)      # {'name': 'Rahul', 'age': 22}
```

Key Points :

- केवल reference copy करता है, नया object नहीं बनाता।
- copy(), list(), dict(), set(), [:] → new independent object बनाते हैं।
- Tuple immutable होता है, इसलिए copy की जरूरत कम होती है।
- Shallow copy केवल top-level copy करता है, Deep copy पूरा copy करता है।
- Copying का use तब करें जब original data को सुरक्षित रखना हो।

6. Shallow vs Deep Copy (Brief)

- Shallow Copy केवल top-level elements को copy करता है।
- अगर collection के अंदर भी कोई mutable object (जैसे list, dict) है, तो वह shared रहता है।
- Deep Copy पूरा object और उसके अंदर के भी सभी objects की copy बनाता है, ताकि कोई भी change original को affect न करे।

Example : (Nested List)

```
import copy
a = [[1, 2], [3, 4]]
b = a.copy()      # shallow copy
c = copy.deepcopy(a) # deep copy
b[0].append(10)
print(a)          # [[1, 2, 10], [3, 4]] (changed)
print(c)          # [[1, 2], [3, 4]] (unchanged)
```

Conclusion :

- Collections को copy करना important है, क्योंकि इससे हम data को modify करते समय original collection को सुरक्षित रख सकते हैं।
- Different methods अलग-अलग situations में useful होते हैं, इसलिए सही method का चयन करना चाहिए।

Unit - 5 : Function in Python

1. Introduction of Function

- Function एक block of code होता है, जो किसी specific task को perform करने के लिए बनाया जाता है।
- यह code reusability को बढ़ाता है, जिससे हमें बार-बार वही code लिखने की जरूरत नहीं होती है।
- Function की मदद से बड़े program को छोटे-छोटे parts में divide किया जा सकता है।
- Function को define करने के बाद उसे जब जरूरत होती है तब call (invoke) किया जाता है।
- Python में कई built-in functions पहले से available होते हैं, जैसे print(), len(), type() आदि।
- हम अपनी आवश्यकता के अनुसार user-defined functions भी बना सकते हैं।

2. Advantages of Using Function

- ① Code Reusability - एक बार लिखे गए code को बार-बार उपयोग किया जा सकता है।
- ② Modularity - Program को छोटे-छोटे हिस्सों में बाँटा जा सकता है।
- ③ Readability - Code को पढ़ना और समझना आसान हो जाता है।
- ④ Maintainability - किसी गलती को सुधारना आसान होता है।
- ⑤ Helps in Team Work - बड़े projects में अलग-अलग functions पर अलग-अलग लोग काम कर सकते हैं।

3. Defining Your Own Functions

- Python में अपनी खुद की function बनाने के लिए def keyword का उपयोग किया जाता है।
- Function define करने के बाद उसके नाम के साथ parentheses () लगते हैं।
- Function के अंदर जो code लिखा जाता है, उसे indented block में लिखा जाता है।
- Function कुछ value return भी कर सकता है या बिना return के केवल task perform कर सकता है।

Syntax :

```
def function_name(parameters):  
    # function body  
    statements  
    return value # (optional)
```

Parts of a Function

1. def keyword
2. Function name
3. Parameters (optional)
4. Function body (indented code)
5. return statement (optional)

4. Example : Simple Function (Without Parameter)

```
def greet():  
    print("Hello! Welcome to Python")  
    print("Have a Nice Day!")  
  
# Function call  
greet()
```

Output :

```
Hello! Welcome to Python  
Have a Nice Day!
```

5. Example : Function with Parameter

```
def add(a, b):  
    sum = a + b  
    print("Sum is :", sum)  
  
# Function call  
add(5, 3)
```

Output :

```
Sum is : 8
```

6. Example : Function with Return Value

```
def square(n):  
    return n * n  
  
# Function call and store result  
result = square(4)  
print("Square is :", result)
```

Output :

```
Square is : 16
```

Key Points :

- ★ Function का नाम हमेशा meaningful होना चाहिए।
- ★ Python में function name case-sensitive होता है।
- ★ return statement function से value वापस भेजता है।
- ★ अगर return नहीं लिखा है, तो function None return करता है।

Parameters

- Parameter वह value होती है, जिसे Function definition के समय उसके अंदर receive किया जाता है।
- Parameter की मदद से हम Function को बाहर से data भेज सकते हैं।
- Parameter के बिना Function हर बार same result देगा, लेकिन Parameter के साथ हम अलग-अलग data पर Function को काम करवा सकते हैं।
- Python में Parameters के कई प्रकार होते हैं, जैसे Positional, Default, Keyword, Variable-length आदि।

1. Parameters क्या होते हैं ?

- Parameter एक variable की तरह होता है, जो function के अंदर value को receive करता है।
- जब हम function को call करते हैं, तब arguments के रूप में value भेजी जाती है, जो parameters में store हो जाती है।
- Parameters की मदद से function reusable और flexible बनता है।

Syntax :

```
def function_name(parameters):  
    # function body  
    return value
```

2. Types of Parameters in Python

Python में मुख्य रूप से निम्नलिखित प्रकार के Parameters होते हैं -

- ① Positional Parameters
- ② Default Parameters
- ③ Keyword Parameters
- ④ Variable-length Parameters (*args, **kwargs)

Note :

- Parameter function definition में होता है।
- Argument function call के समय भेजी जाने वाली value होती है।
- दोनों की संख्या और order सही होना चाहिए (कुछ cases को छोड़कर)।

3. Positional Parameters

- इसमें values position के आधार पर parameters को मिलती हैं।
- function call करते समय values उसी order में भेजी जाती हैं।

Example :

```
def add(a, b):  
    sum = a + b  
    print("Sum is :", sum)  
add(5, 3)      # 5 → a, 3 → b
```

Output :

Sum is : 8

4. Default Parameters

- Default parameter वह होता है, जिसे function definition में ही default value दे दी जाती है।
- अगर function call करते समय value नहीं भेजी जाती, तो default value use होती है।

Example :

```
def greet(name = "Student"):  
    print("Hello", name)  
greet("Brajkant")      # value passed  
greet()                 # default value used
```

Output :

Hello Brajkant
Hello Student

5. Keyword Parameters

- इसमें values parameter name के साथ भेजी जाती हैं।
- इससे order की चिंता नहीं करनी पड़ती।

Example :

```
def student(name, age):  
    print("Name :", name)  
    print("Age :", age)  
student(age = 23, name = "Brajkant")
```

Output :

Name : Brajkant
Age : 23

6. Variable-length Parameters

- जब हमें function में अनिश्चित संख्या में values लेनी होती हैं, तब हम *args और **kwargs का उपयोग करते हैं।
- *args → Non-keyworded, variable-length arguments (tuple के रूप में मिलते हैं)
- **kwargs → Keyworded, variable-length arguments (dictionary के रूप में मिलते हैं)

Example :

```
def show(*args, **kwargs):  
    print("Values (args):", args)  
    print("Values (kwargs):", kwargs)  
show(1, 2, 3, name="Brajkant", age=23)
```

Output :

Values (args): (1, 2, 3)
Values (kwargs): {'name': 'Brajkant',
 'age': 23 }

Key Points :

1. Parameter function definition में लिखे जाते हैं, argument function call में दिए जाते हैं।
2. Parameters की मदद से function को reusable बनाया जा सकता है।
3. Default, keyword और variable-length parameters function को ज्यादा powerful बनाते हैं।
4. सही प्रकार का parameter चुनना program की आवश्यकता पर निर्भर करता है।

Code with Parameters, Code without Parameters in Function

1. Function without Parameters (बिना Parameters के)

- जब किसी function को define करते समय उसके अंदर कोई parameter नहीं दिया जाता, तो उसे function without parameters कहते हैं।
- यह function कुछ fixed काम करता है और उसे call करने पर वही काम execute होता है।
- इसमें बाहर से कोई value pass नहीं की जाती।
- ऐसे function का उपयोग तब किया जाता है जब function को किसी बाहरी data की आवश्यकता नहीं होती।

Syntax :

```
def function_name():  
    # function body  
    statements  
  
# function call  
function_name()
```

Example :

```
def greet():  
    print("Hello! Welcome to Python")  
    print("Have a Nice Day!")  
  
# function call  
greet()
```

Output :

Hello! Welcome to Python
Have a Nice Day!

Explanation :

- यहाँ greet() नाम का एक function बनाया गया है जिसमें कोई parameter नहीं है।
- जब हम greet() को call करते हैं, तो function के अंदर लिखे दोनों statements execute होते हैं।
- यह एक simple function है जो केवल message print करता है।

2. Function with Parameters (Parameters के साथ)

- जब function को define करते समय उसके अंदर parameter दिए जाते हैं, तो उसे function with parameters कहते हैं।
- इसमें function को call करते समय arguments के रूप में values pass की जाती हैं।
- यह function अलग-अलग values के अनुसार अलग-अलग result दे सकता है।
- यह अधिक flexible होता है और reusable code लिखने में मदद करता है।

Syntax :

```
def function_name(parameters):  
    # function body  
    statements  
  
# function call  
function_name(arguments)
```

Example :

```
def add(a,b):  
    sum = a + b  
    print("Sum is :", sum)  
  
# function call  
add(5, 3)
```

Output :

Sum is : 8

Explanation :

- यहाँ add() नाम का function बनाया गया है जिसमें दो parameters (a,b) हैं।
- जब हम add(5,3) call करते हैं, तो 5 और 3 इन parameters में pass हो जाते हैं।
- Function इन दोनों values को जोड़कर result print करता है।
- Parameter की वजह से हम एक ही function को अलग-अलग values के साथ use कर सकते हैं।

Note :

- Parameters function definition के समय लिखे जाते हैं।
- Arguments function call के समय pass की जाती हैं।
- Without parameter function fixed काम करता है।
- With parameter function dynamic होता है और अलग-अलग input ले सकता है।
- दोनों प्रकार के function Python में महत्वपूर्ण हैं और program को modular तथा reusable बनाने में मदद करते हैं।

Key Points :

- ★ Function without parameters में बाहर से कोई value नहीं दी जाती।
- ★ Function with parameters में values pass की जाती हैं।
- ★ Parameters code को reusable और flexible बनाते हैं।
- ★ दोनों का उपयोग अलग-अलग परिस्थितियों में किया जाता है।

Function Documentation

1. What is Function Documentation?

- Function documentation का मतलब है किसी function के बारे में जानकारी (description) देना, जिससे पता चले कि वह function क्या करता है, उसके parameters क्या हैं और वह क्या return करता है।
- Python में function documentation को docstring कहा जाता है।
- Docstring function की पहली line में लिखी जाती है और यह triple quotes (""" """ या ''' ''') के बीच होती है।
- यह code को समझने में मदद करती है और program को पढ़ने और maintain करने में आसान बनाती है।

2. Syntax of Function Documentation (Docstring)

```
def function_name(parameters):  
    """  
    Documentation string (Docstring)  
    """  
  
    # function body  
    return value
```

Key Points :

- ★ Docstring triple quotes में लिखी जाती है।
- ★ यह function के अंदर सबसे पहले लिखी जाती है।
- ★ यह optional होती है लेकिन अच्छी programming practice के लिए जरूरी है।
- ★ Docstring को help() या __doc__ के द्वारा देखा जा सकता है।

3. Example : Simple Function with Documentation

Code :

```
def add(a, b):  
    """  
    This function takes two numbers as parameters  
    and returns their sum.  
    """  
  
    sum = a + b  
    return sum
```

Output :

```
print(add.__doc__)
```

This function takes two numbers as parameters and returns their sum.

4. Accessing Function Documentation

- हम function की documentation को कई तरीकों से access कर सकते हैं -
- ① `function_name.__doc__` → यह docstring को string के रूप में return करता है।
- ② `help(function_name)` → यह docstring के साथ और भी जानकारी दिखाता है।
- ③ IDE (जैसे VS Code, PyCharm) में function पर hover करने से भी docstring दिखाई जाती है।

5. Example : Using __doc__ and help()

Code :

```
def greet(name):  
    """  
    This function greets the user with a message.  
    Parameter:  
        name (str): Name of the user  
    Returns:  
        str: Greeting message  
    """  
    return "Hello " + name
```

Using __doc__ :

```
print(greet.__doc__)
```

Output :

This function greets the user with a message.
Parameter:
 name (str): Name of the user
Returns:
 str: Greeting message

Using help() :

```
help(greet)
```

Output (short form) :

Help on function greet in module __main__:
greet(name)
 This function greets the user with a message.

6. Advantages of Function Documentation

- ① Code को समझने में मदद मिलती है।
- ② Team में काम करने पर दूसरों के लिए code पढ़ना आसान हो जाता है।
- ③ Program को maintain और debug करना आसान होता है।
- ④ यह professional और clean coding की पहचान है।

Conclusion :

Function documentation (docstring) Python में एक बहुत ही उपयोगी feature है। यह code की readability को बढ़ाती है, दूसरों के लिए समझने में आसान बनाती है और long-term maintenance में मदद करती है।

Keyword and Optional Parameters

1. Keyword Parameters

- Keyword parameter वह parameter होता है, जिसमें value को parameter name के साथ specify किया जाता है।
- इसमें arguments को position के अनुसार देने की restriction नहीं होती है।
- जब function में बहुत सारे parameters होते हैं, तब keyword parameter का उपयोग करने से code की readability बढ़ जाती है।
- Python में function call करते समय हम parameters को किसी भी order में उनके नाम (keyword) के साथ pass कर सकते हैं।
- यह खासकर useful होता है, जब हमें केवल कुछ specific parameters की value change करनी हो।

Syntax :

```
def function_name(param1, param2, ...):  
    # function body  
    return value  
  
# function call with keyword arguments  
function_name(param1 = value1, param2 = value2)
```

Example :

```
def student(name, age, course):  
    print("Name :", name)  
    print("Age :", age)  
    print("Course :", course)  
  
# function call (keyword arguments)  
student(age = 23, name = "Brajkant", course = "PGDCA")
```

Output :

```
Name : Brajkant  
Age : 23  
Course : PGDCA
```

Key Points :

- Keyword parameter में arguments को किसी भी order में pass किया जा सकता है।
- यह code को ज्यादा readable बनाता है।
- सभी parameters को keyword के रूप में नहीं देना जरूरी नहीं है, कुछ normal भी हो सकते हैं।
- यह तब useful होता है जब function में कई parameters हों।

2. Optional Parameters

- Optional parameter वह होता है, जिसका default value पहले से defined होता है।
- यदि function call करते समय उसकी value pass नहीं की जाती है, तो वह default value use होती है।
- Optional parameters की मदद से हम function को flexible बना सकते हैं।
- यह हमेशा keyword parameter के रूप में दिए जाते हैं (name = default_value)।
- Optional parameter का उपयोग तब होता है, जब किसी parameter की value हमेशा change नहीं होती है।

Syntax :

```
def function_name(param1, param2 = default_value):  
    # function body  
    return value  
  
# function call  
function_name(value1)           # only required argument  
function_name(value1, value2)  # with optional argument
```

Example :

```
def greet(name, msg = "Hello"):  
    print(msg, name)  
  
# function call (without optional argument)  
greet("Brajkant")  
  
# function call (with optional argument)  
greet("Brajkant", "Welcome")
```

Output :

```
Hello Brajkant  
Welcome Brajkant
```

Key Points :

- Optional parameter का default value पहले से set होता है।
- यदि value pass नहीं की जाती है, तो default value use होती है।
- यह function को flexible और user-friendly बनाता है।
- Optional parameters हमेशा keyword के रूप में लिखे जाते हैं।
- इससे function call करते समय कम arguments देने पर भी function काम करता है।

Passing Collections to a Function

1. Introduction

- Collection का अर्थ होता है ऐसे data structure जिनमें एक से अधिक values store होती हैं, जैसे List, Tuple, Set, Dictionary आदि।
- Python में हम इन collections को function के अंदर argument के रूप में pass कर सकते हैं।
- जब हम collection को function में pass करते हैं, तो function उसके सभी elements पर काम कर सकता है, जैसे read करना, modify करना, add या remove करना आदि।
- List और Dictionary mutable (बदले जा सकने वाले) होते हैं, इसलिए function के अंदर उसमें किया गया परिवर्तन original collection पर भी प्रभाव डाल सकता है।

2. Passing a List to a Function

- List को function में pass करने पर हम उसके सभी elements को access कर सकते हैं।
- Function चाहे तो List को modify भी कर सकता है।

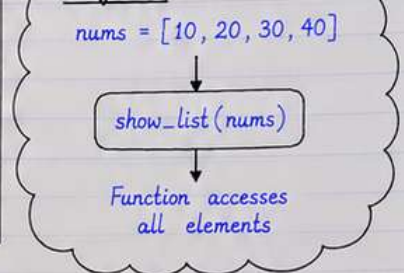
Example :

```
def show_list(items):  
    print("List elements are :")  
    for item in items:  
        print(item)  
  
nums = [10, 20, 30, 40]  
show_list(nums)
```

Output :

```
List elements are :  
10  
20  
30  
40
```

Diagram :



3. Modifying a List inside a Function

- List mutable होती है, इसलिए function के अंदर यदि हम List में changes करते हैं, तो वह original List पर भी लागू होता है।
- यह इसलिए होता है क्योंकि List को reference के द्वारा pass किया जाता है।

Example :

```
def add_item(items, value):  
    items.append(value)  
  
nums = [1, 2, 3]  
add_item(nums, 4)  
print(nums)
```

Output :

```
[1, 2, 3, 4]
```

Note :

- append() List के अंत में element जोड़ता है।
- Function के अंदर किया गया change original List को भी modify करता है।

4. Passing a Tuple to a Function

- Tuple immutable होती है, इसलिए function के अंदर उसके elements को modify नहीं कर सकते।
- हम tuple को function में pass करके उसके values को read कर सकते हैं।

Example :

```
def show_tuple(data):  
    for item in data:  
        print(item)  
  
t = (5, 10, 15)  
show_tuple(t)
```

Output :

```
5  
10  
15
```

Note :

- Tuple के elements को change करने पर error आएगा।
- Tuple केवल read-only access के लिए useful होता है।

5. Passing a Dictionary to a Function

- Dictionary को function में pass करने पर हम उसके keys और values को access कर सकते हैं।
- Function के अंदर dictionary को modify भी किया जा सकता है।

Example :

```
def show_dict(d):  
    for key, value in d.items():  
        print(key, ":", value)  
  
student = {"name": "Brajkant", "age": 23}  
show_dict(student)
```

Output :

```
name : Brajkant  
age : 23
```

Key Points :

- Collection को function में argument के रूप में pass किया जाता है।
- List और Dictionary mutable होते हैं।
- Tuple immutable होता है।
- Function के अंदर परिवर्तन original collection पर भी लागू हो सकता है।
- Collections का उपयोग करने से code अधिक modular और reusable बनता है।

Variable Number of Arguments

1. Introduction

- Python में हम किसी function को ऐसे define कर सकते हैं जो variable number of arguments ले सके, यानी function call करते समय arguments की संख्या fixed नहीं होती है।
- इसके लिए हम *(asterisk) operator का उपयोग करते हैं।
- यह feature तब उपयोगी होता है जब हमें पहले से यह पता न हो कि user कितने arguments pass करेगा।
- Variable number of arguments को हम दो प्रकार से use कर सकते हैं -
 - ① *args (Non-keyword variable arguments)
 - ② **kwargs (Keyword variable arguments)

2. Using *args (Non-keyword Arguments)

- *args का उपयोग तब करते हैं जब हम किसी function में variable number of positional arguments pass करना चाहते हैं।
- यह सभी arguments को एक tuple के रूप में store करता है।
- args केवल एक नाम है, आप इसकी जगह कोई भी valid name लिख सकते हैं।

Example :

```
def add(*args):  
    sum = 0  
    for num in args:  
        sum = sum + num  
    return sum  
  
print(add(5, 10, 15))  
print(add(1, 2, 3, 4, 5))
```

Output :

30
15

Explanation :

- यहाँ function add() किसी भी संख्या में arguments ले सकता है।
- सभी values args नाम के tuple में store होती हैं।
- for loop द्वारा सभी values को जोड़ा जाता है और result return किया जाता है।

3. Using **kwargs (Keyword Arguments)

- **kwargs का उपयोग तब करते हैं जब हम variable number of keyword arguments pass करना चाहते हैं।
- यह सभी arguments को एक dictionary के रूप में store करता है।
- kwargs केवल एक नाम है, आप इसकी जगह कोई भी valid name लिख सकते हैं।

Example :

```
def student_info(**kwargs):  
    for key, value in kwargs.items():  
        print(key, ":", value)  
  
student_info(name = "Brajkant",  
             age = 23, course = "PGDCA")
```

Output :

name : Brajkant
age : 23
course : PGDCA

Explanation :

- यहाँ function student_info() में variable number of keyword arguments pass किए गए हैं।
- सभी key-value pairs kwargs dictionary में store होती हैं।
- for loop द्वारा dictionary के items को access करके print किया जाता है।

4. Using Both *args and **kwargs Together

- हम एक ही function में *args और **kwargs दोनों का उपयोग कर सकते हैं।
- *args हमेशा **kwargs से पहले लिखा जाता है।
- इससे function अधिक flexible और powerful बन जाता है।

Example :

```
def demo(*args, **kwargs):  
    print("Positional arguments :", args)  
    print("Keyword arguments :", kwargs)  
  
demo(1, 2, 3, name = "Brajkant", course = "PGDCA")
```

Output :

Positional arguments : (1, 2, 3)
Keyword arguments :
{'name': 'Brajkant',
'course': 'PGDCA'}

Key Points :

- *args → tuple के रूप में store करता है।
- **kwargs → dictionary के रूप में store करता है।
- इनका उपयोग function को flexible बनाने के लिए किया जाता है।
- *args हमेशा **kwargs से पहले लिखा जाता है।
- यह feature real-time programming में बहुत उपयोगी है।

Conclusion :

Variable number of arguments का उपयोग करके हम ऐसे functions बना सकते हैं जो किसी भी संख्या में arguments को handle कर सकें, जिससे code अधिक flexible, reusable और user-friendly बनता है।

Scope • Functions - "First Class Citizens"

1. Scope

- Scope का मतलब है किसी variable का वह area जहाँ वह accessible (use) हो सकता है।
- Python में हर variable एक specific scope में defined होता है।
- Scope यह decide करता है कि हम उस variable को program के किस भाग में access कर सकते हैं।
- यदि variable अपने scope के बाहर access किया जाता है तो error आता है।
- Python में मुख्यतः तीन प्रकार के scope होते हैं - Local Scope, Global Scope और Built-in Scope।

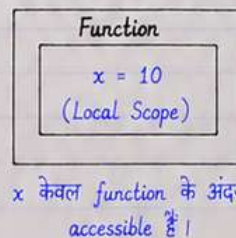
1.1 Local Scope

- जो variable किसी function के अंदर define किया जाता है, वह local scope में होता है।
- यह variable केवल उसी function के अंदर accessible होता है।
- Function के बाहर इसे access नहीं किया जा सकता।

Example :

```
def show():  
    x = 10 # local variable  
    print(x)  
  
show() # Output: 10  
print(x) # Error: x is not defined
```

Diagram : Local Scope



1.2 Global Scope

- जो variable function के बाहर define किया जाता है, वह global scope में होता है।
- यह variable पूरे program में कहीं भी access किया जा सकता है।

Example :

```
x = 5 # global variable  
def show():  
    print(x) # access global  
  
show() # Output: 5  
print(x) # Output: 5
```

1.3 Built-in Scope

- Python में कुछ pre-defined names (जैसे print(), len(), range() आदि) built-in scope में होते हैं।
- ये सभी modules और functions में automatically available होते हैं।
- इन्हें हम override नहीं करना चाहिए क्योंकि ये Python के core part होते हैं।

2. Functions - "First Class Citizens"

- Python में functions को "First Class Citizens" माना जाता है, जिसका मतलब है कि functions को भी variable की तरह treat किया जा सकता है।
- Functions को assign करना, pass करना, return करना और data structure में store करना possible है।
- यह feature Python को और अधिक flexible और powerful बनाता है।

2.1 Assigning Function to a Variable

- हम किसी function को एक variable में assign कर सकते हैं।

Example :

```
def greet():  
    print("Hello!")  
  
msg = greet # function assign  
msg() # Output: Hello!
```

2.2 Passing Function as an Argument

- हम किसी function को दूसरे function में argument के रूप में pass कर सकते हैं।

Example :

```
def greet():  
    print("Welcome!")  
  
def call_func(func):  
    func()  
  
call_func(greet) # Output: Welcome!
```

2.3 Returning a Function

- एक function दूसरे function से return भी किया जा सकता है।

Example :

```
def outer():  
    def inner():  
        print("I am inner function")  
    return inner  
  
f = outer()  
f() # Output: I am inner function
```

2.4 Storing Functions in a Data Structure

- Functions को list, tuple, dictionary आदि में store किया जा सकता है।

Example :

```
def add(a, b): return a + b  
def sub(a, b): return a - b  
  
operations = [add, sub]  
print(operations[0](5, 3)) # Output: 8  
print(operations[1](5, 3)) # Output: 2
```

Key Points :

- Variable का scope यह तय करता है कि वह कहाँ accessible है।
- Local variable केवल function में अंदर available होता है।
- Global variable पूरे program में accessible होता है।
- Built-in scope Python के pre-defined names के लिए होता है।
- Python में functions "First Class Citizens" हैं।
- Functions को assign, pass, return और data structure में store किया जा सकता है।
- यह feature higher-order functions और functional programming के लिए बहुत उपयोगी है।

Passing Functions to a Function

1. Introduction

- Python में functions को भी objects की तरह treat किया जाता है, इसलिए हम एक function को दूसरे function के argument के रूप में pass कर सकते हैं।
- यह feature Python की "First Class Functions" property के कारण संभव होता है।
- जब हम किसी function को argument के रूप में pass करते हैं, तब हम उस function को call नहीं करते, बल्कि उसका reference पास करते हैं।
- जिस function को argument के रूप में pass किया जाता है, उसे higher-order function में कभी (parameter) के रूप में receive किया जाता है और आवश्यकता के अनुसार उसे call किया जाता है।
- यह technique code को flexible, reusable और modular बनाने में मदद करती है।

2. Syntax

```
def higher_order_function(func, other_args):  
    # func is a function passed as argument  
    func(other_args)  
  
def some_function():  
    # function body  
    pass  
  
higher_order_function(some_function, value)
```

Key Points :

- Function का नाम बिना parentheses के pass किया जाता है।
- यह function का reference पास करता है, उसका result नहीं।
- Higher-order function received function को अपने code के अंदर call करता है।
- इससे code में reusability और flexibility बढ़ती है।

3. Simple Example : Passing a Function

Code :

```
def greet():  
    print("Hello from greet()!")  
  
def call_function(func):  
    print("Calling the function...")  
    func()  
  
call_function(greet)
```

Output :

```
Calling the function...  
Hello from greet()!
```

Explanation :

- greet function को call_function में argument के रूप में pass किया गया है।
- func parameter में greet function का reference प्राप्त होता है।
- func() के द्वारा greet() function execute होता है।

4. Example with Arguments

Code :

```
def add(a, b):  
    print("Sum :", a + b)  
  
def operate(func, x, y):  
    print("Performing operation...")  
    func(x, y)  
  
operate(add, 5, 3)
```

Output :

```
Performing operation...  
Sum : 8
```

Explanation :

- यहाँ add function को operate function में argument के रूप में pass किया गया है।
- operate function, func(x, y) के द्वारा add(5, 3) को call करता है।
- इस तरह हम किसी भी function को dynamic रूप से use कर सकते हैं।

5. Real-world Use Case : Using Function as a Callback

Code :

```
def process_data(data, func):  
    result = func(data)  
    return result  
  
def square(n):  
    return n * n  
  
print(process_data(4, square))
```

Output :

```
16
```

Explanation :

- process_data function में square function को pass किया गया है।
- process_data, func(data) के द्वारा square(4) को call करता है।
- यह callback pattern का एक example है, जिसका उपयोग data processing, event handling आदि में किया जाता है।

Conclusion :

Python में function को function के argument के रूप में pass करना एक powerful feature है जो higher-order programming को संभव बनाता है और code को अधिक flexible और reusable बनाता है।

map

1. Introduction

- `map()` Python का एक built-in function है।
- यह किसी भी function को किसी iterable (जैसे List, Tuple, Set, आदि) के सभी elements पर apply करता है।
- `map()` हर element को function के अंदर भेजता है और उसका result वापस देता है।
- यह एक map object (iterator) return करता है, जिसे हम `list()`, `tuple()` आदि की help से प्राप्त कर सकते हैं।
- `map()` का उपयोग तब किया जाता है जब हमें iterable के सभी elements पर एक ही operation करना है।
- यह code को छोटा, सरल और readable बनाता है।

Syntax :

`map(function, iterable)`

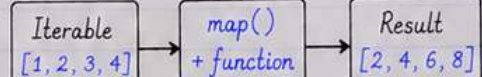
function → वह function जो
हर element पर apply होगा

iterable → List, Tuple, Set,
String आदि

2. How map() Works ?

- `map()` iterable के हर element को एक-एक करके function को देता है।
- function उस element पर operation perform करता है।
- सभी results को मिलाकर map object return होता है।
- इसे `list()` जैसी function की help से list में convert किया जा सकता है।

Working Flow :



3. Example 1 : Using map() with a User-defined Function

Code :

```
def square(n):  
    return n * n  
  
nums = [1, 2, 3, 4]  
result = map(square, nums)  
print(list(result))
```

Output :

[1, 4, 9, 16]

Explanation :

- `square()` function को `map()` के साथ pass किया गया है।
- `nums` के सभी elements पर `square()` apply होता है।
- `result` एक map object होता है, इसे list में convert करके output प्राप्त किया गया है।

4. Example 2 : Using map() with a Lambda Function

Code :

```
nums = [1, 2, 3, 4]  
result = map(lambda x: x + 10, nums)  
print(list(result))
```

Output :

[11, 12, 13, 14]

Explanation :

- यहाँ lambda function का उपयोग किया गया है।
- यह हर element में 10 add करता है।
- `map()` के साथ lambda का उपयोग code को और भी compact बनाता है।

5. Example 3 : Using map() with Multiple Iterables

Code :

```
def add(a, b):  
    return a + b  
  
list1 = [1, 2, 3]  
list2 = [10, 20, 30]  
result = map(add, list1, list2)  
print(list(result))
```

Output :

[11, 22, 33]

Explanation :

- `map()` एक से अधिक iterable को भी ले सकता है।
- यह `list1` और `list2` के corresponding elements को add करता है।
- `add()` function दोनों values को receive करके उनका sum return करता है।

6. Important Points

- `map()` हमेशा एक map object (iterator) return करता है।
- इसे `list()`, `tuple()` या `set()` की help से convert किया जा सकता है।
- यह for loop की तुलना में अधिक concise और readable होता है।
- `map()` का उपयोग किसी भी iterable (List, Tuple, Set, String, आदि) के साथ किया जा सकता है।
- यह functional programming का एक महत्वपूर्ण tool है।

Key Takeaways :

- `map()` → function + iterable
- हर element पर operation
- map object return करता है।
- `list()` से result प्राप्त करें
- code को सरल और छोटा बनाता है।

filter

1. Introduction

- `filter()` Python का एक built-in function है।
- यह किसी iterable के elements को एक condition (function) के आधार पर filter करता है।
- `filter()` हर element को function में भेजता है और केवल उन elements को return करता है जिनके लिए function True return करता है।
- यह एक filter object (iterator) return करता है, जिसे हम `list()`, `tuple()`, `set()` आदि में convert कर सकते हैं।
- `filter()` का उपयोग तब किया जाता है जब हमें iterable में से केवल कुछ specific elements को select करना होता है।
- यह code को सरल, concise और readable बनाता है।

Syntax :

`filter(function, iterable)`

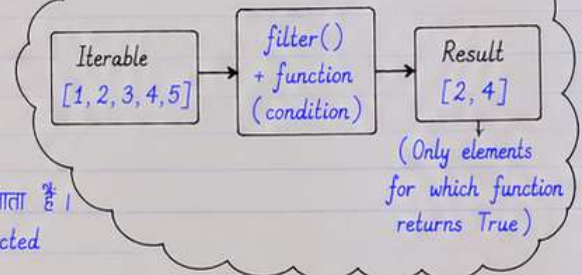
function → वह function जो हर element पर condition check करके True या False return करता है।

iterable → List, Tuple, Set, String आदि

2. How filter() Works ?

- `filter()` iterable के हर element को एक-एक करके function को भेजता है।
- function हर element पर condition check करता है।
- यदि function True return करता है, तो वह element result में शामिल होता है।
- यदि False return करता है, तो वह element skip हो जाता है।
- अंत में filter object return होता है, जिसमें केवल selected elements होते हैं।

Working Flow :



3. Example 1 : Filter Even Numbers

Code :

```
def is_even(n):  
    return n % 2 == 0  
  
nums = [1, 2, 3, 4, 5, 6]  
result = filter(is_even, nums)  
print(list(result))
```

Output :

[2, 4, 6]

Explanation :

- `is_even()` function हर number पर check करता है कि वह even है या नहीं।
- केवल वे elements जिनके लिए True return होता है, result में आते हैं।
- filter object को list में convert करके output प्राप्त किया गया है।

4. Example 2 : Filter Strings with Specific Length

Code :

```
def long_word(word):  
    return len(word) > 4  
  
words = ["cat", "python", "java", "code"]  
result = filter(long_word, words)  
print(list(result))
```

Output :

['python', 'java']

Explanation :

- `long_word()` function check करता है कि string की length 4 से अधिक है या नहीं।
- केवल वे strings जिनकी length > 4 है, result में शामिल होती हैं।
- Output में केवल "python" और "java" प्राप्त हुए हैं।

5. Example 3 : Using Lambda Function

Code :

```
nums = [10, 15, 20, 25, 30]  
result = filter(lambda x: x > 18, nums)  
print(list(result))
```

Output :

[20, 25, 30]

Explanation :

- यहाँ lambda function दिया गया है जो check करता है कि number 18 से बड़ा है या नहीं।
- जो elements condition को satisfy करते हैं, वही result में आते हैं।

6. Important Points

- `filter()` एक filter object (iterator) return करता है।
- इसे `list()`, `tuple()` या `set()` में convert किया जा सकता है।
- `filter()` का उपयोग iterable से specific elements को select करने के लिए होता है।
- यह code को छोटा, सरल और readable बनाता है।
- यह functional programming का एक महत्वपूर्ण tool है।

Key Takeaways :

- `filter()` → function + iterable
- केवल True वाले elements return होते हैं।
- Result एक filter object होता है।
- `list()` का उपयोग करके output प्राप्त करें।
- यह code को concise और efficient बनाता है।

Mapping Functions in a Dictionary

1. Introduction

- Python में हम dictionary के अंदर functions को store कर सकते हैं।
- यह technique हमें different actions (functions) को key (जैसे string) के साथ map करने की सुविधा देती है।
- इसका उपयोग तब किया जाता है जब हमें किसी key के आधार पर specific function को call करना हो।
- Dictionary में function का नाम नहीं, बल्कि function object (reference) store होता है।
- यह approach code को organized, flexible और scalable बनाता है।
- यह menu-driven program, event handling और dynamic behaviour के लिए बहुत useful होता है।

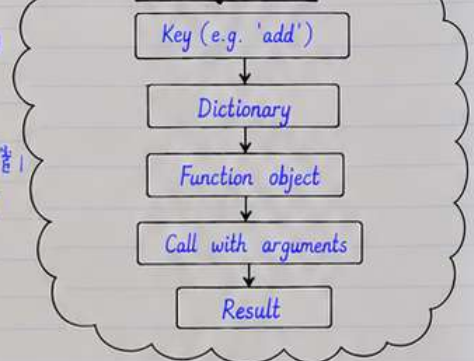
Syntax :

```
func_dict = {  
    'key1' : function1,  
    'key2' : function2,  
    ...  
}  
  
# call  
func_dict["key"](arguments)
```

2. How It Works ?

- हम functions को dictionary के key-value pair के रूप में store करते हैं।
- जब किसी key का उपयोग किया जाता है, तो उससे संबंधित function dictionary से retrieve किया जाता है।
- उस function को फिर उसके required arguments के साथ call किया जाता है।
- यदि key मौजूद नहीं है, तो KeyError आता है, इसलिए key को पहले check करना अच्छा practice होता है।

Working Flow :



3. Example 1 : Basic Function Mapping

Code :

```
def add(a, b):  
    return a + b  
  
def sub(a, b):  
    return a - b  
  
operations = {  
    'add' : add,  
    'sub' : sub  
}  
  
print(operations['add'](5, 3))  
print(operations['sub'](5, 3))
```

Output :

8
2

Explanation :

- 'add' key के साथ add() function map किया गया है।
- 'sub' key के साथ sub() function map किया गया है।
- operations['add'] हमें add function का reference देता है, जिसे arguments के साथ call किया जाता है।
- यह approach code को dynamic और extendable बनाता है।

4. Example 2 : Using Dictionary for a Menu

Code :

```
def greet(name):  
    return f"Hello {name}!"  
  
def square(n):  
    return n * n  
  
func_dict = {  
    'greet' : greet,  
    'square' : square  
}  
  
print(func_dict['greet']("Braj Kant"))  
print(func_dict['square'](6))
```

Output :

Hello Braj Kant!
36

Explanation :

- यहाँ 'greet' और 'square' functions dictionary में map किए गए हैं।
- Key के आधार पर सही function retrieve करके call किया गया है।
- यह menu-driven programs में बहुत useful होता है।
- इससे नए functions को आसानी से add किया जा सकता है।

5. Important Points

- Dictionary में function का reference (function object) store होता है।
- Key का type आमतौर पर string होता है, लेकिन कोई भी immutable type हो सकता है।
- Key को call करने से पहले check करना जरूरी है ताकि KeyError से बचा जा सके।
- यह technique large programs में code management को आसान बनाती है।
- Lambda functions को भी dictionary में map किया जा सकता है।
- यह method loose coupling और flexible design के लिए अच्छा माना जाता है।

Key Takeaways :

- Functions को dictionary में map किया जा सकता है।
- Key के आधार पर function call किया जाता है।
- यह approach dynamic, modular और scalable code के लिए useful है।
- Real-world applications : menu system, event handling, command mapping आदि।

Lambda

1. Introduction

- Lambda Python में एक anonymous function (बिना नाम वाला function) होता है।
- यह एक single expression में define किया जाता है।
- यह छोटे और simple functions के लिए उपयोगी होता है।
- Lambda function को lambda keyword द्वारा बनाया जाता है।
- इसका उपयोग प्रायः higher-order functions (जैसे map(), filter(), sorted()) आदि के साथ किया जाता है।
- यह code को concise और readable बनाता है।
- Lambda function का कोई नाम नहीं होता, इसलिए इसे anonymous function भी कहा जाता है।

Syntax :

lambda arguments : expression

Example :

lambda x : x * 2

- इसमें x एक argument है।
- x * 2 एक expression है।
- यह expression का result automatically return करता है।

2. Lambda vs Normal Function

Basis	Normal Function	Lambda Function
Name	Function का name होता है	कोई name नहीं (anonymous)
Syntax	def keyword का उपयोग	lambda keyword का उपयोग
Body	एक से अधिक statements हो सकते हैं	केवल एक single expression
Return	return keyword का उपयोग	Automatically return करता है
Use Case	Complex logic के लिए	छोटे और simple functions के लिए
Readability	थोड़ा बड़ा code	Compact और shorter code

3. Examples

Example 1 : Simple Lambda

Code :

```
square = lambda x : x * x  
print(square(5))
```

Output :

25

Explanation :

- यह lambda function एक number का square return करता है।
- इसे एक variable (square) में store किया गया है।

Example 2 : Lambda with Multiple Arguments

Code :

```
add = lambda x, y : x + y  
print(add(10, 20))
```

Output :

30

Explanation :

- यह दो numbers को add करता है।
- Multiple arguments भी दिए जा सकते हैं।

Example 3 : Using Lambda with Higher-order Function (map)

Code :

```
nums = [1, 2, 3, 4, 5]  
result = list(map(lambda x : x * 2, nums))  
print(result)
```

Output :

[2, 4, 6, 8, 10]

Explanation :

- map() के साथ lambda function का उपयोग किया गया है।
- यह सभी elements को 2 से multiply करता है।

4. More Uses of Lambda

- filter() के साथ → condition के आधार पर elements filter करने के लिए।
- sorted() के साथ → custom sorting के लिए।
- key parameter में → dictionary या list को specific key के आधार पर sort करने के लिए।
- Function को argument के रूप में pass करने के लिए।
- Callback functions में उपयोग।

Example : Using filter()

Code :

```
nums = [1, 2, 3, 4, 5, 6]  
even = list(filter(lambda x : x % 2 == 0, nums))  
print(even)
```

Output : [2, 4, 6]

5. Important Points

- Lambda function में केवल एक expression हो सकता है।
- यह automatically result return करता है।
- Complex logic के लिए lambda function का उपयोग नहीं करना चाहिए।
- यह code को short और clean बनाता है।
- Lambda functions functional programming में बहुत उपयोगी होते हैं।

Key Takeaways :

- Lambda → anonymous function
- Used for small and simple tasks
- Single expression only
- Works well with map(), filter(), sorted()
- Makes code concise and readable

Conclusion :

Lambda function Python की एक powerful feature है जो कम code में अधिक काम करने में मदद करती है। यह विशेष रूप से higher-order functions के साथ बहुत उपयोगी होती है।

Inner Functions, Closures

1. Introduction

- Python में किसी function के अंदर defined function को inner function (nested function) कहते हैं।
- Inner function का उपयोग code को modular और organized बनाने के लिए किया जाता है।
- Inner function, outer function के local scope को access कर सकता है।
- जब inner function, outer function के variables को remember करता है और बाद में भी use करता है, तो उसे closure कहते हैं।
- Closure हमें data को private रखने में मदद करता है।
- यह technique functional programming में बहुत useful होती है।

Key Terms :

- Inner Function → Function defined inside another function
- Closure → Function with preserved enclosing variables
- Enclosing Scope → Outer function का scope
- Nonlocal → Outer scope के variable को modify करने के लिए
- Lexical Scope → Scope determined by code structure

2. Inner Function Syntax

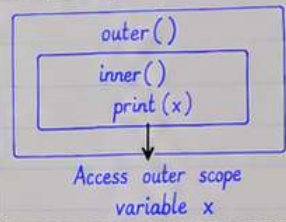
Code :

```
def outer():  
    x = 10  
    def inner():  
        print(x)  
    inner()
```

Explanation :

- inner() function, outer() के अंदर defined है।
- inner() outer function के variable x को access कर सकता है।
- यह structure encapsulation के लिए useful होता है।

Diagram :



3. Example 1 : Accessing Outer Variable

Code :

```
def outer():  
    msg = "Hello"  
    def inner():  
        print(msg)  
    inner()  
outer()
```

Output :

Hello

Explanation :

- inner() function, outer() के variable msg को access कर रहा है।
- यह closure का basic example है।
- inner() के पास अपने enclosing scope का access होता है।

4. Example 2 : Closure (Returning Inner Function)

Code :

```
def outer(a):  
    def inner(b):  
        return a + b  
    return inner  
  
add = outer(5)  
print(add(3))
```

Output :

8

Explanation :

- outer() एक inner function को return करता है।
- returned function (inner) अपने enclosing variable a को remember रखता है।
- यह एक closure है, क्योंकि inner() अपने definition के बाद भी variable a को access कर सकता है।

5. Example 3 : Using nonlocal Keyword

Code :

```
def counter():  
    count = 0  
    def increment():  
        nonlocal count  
        count += 1  
        return count  
    return increment  
  
c = counter()  
print(c())  
print(c())
```

Output :

1
2

Explanation :

- nonlocal keyword का उपयोग outer scope के variable को modify करने के लिए किया जाता है।
- यह closure के साथ मिलकर काम करता है।
- हर बार c() call करने पर count की value update होती है, क्योंकि variable preserved रहता है।

6. Important Points

- Inner function outer function के local variables को access कर सकता है।
- Closure, function के साथ उसके enclosing scope को store करता है।
- nonlocal keyword outer scope के variable को modify करने के लिए उपयोगी है।
- यह technique encapsulation, decorators और callbacks में बहुत useful होती है।
- Closures data hiding और state maintenance के लिए powerful tool हैं।

Key Takeaways :

- Inner function → code organization
- Closure → remembers enclosing variables
- nonlocal → modify outer variable
- Useful in decorators, callbacks, etc.
- Helps in writing clean and modular code

Unit - 6 : Modules

Topic : Modules

- Module Python में एक file होती है जिसमें related functions, classes, variables और code होते हैं।
- Module का use code को organize करने के लिए किया जाता है ताकि उसे reuse कर सकें।
- Python में बहुत से built-in modules available होते हैं और हम अपने own modules भी बना सकते हैं।
- Module use करने के लिए उसे import करना होता है।
- यह code को modular बनाता है, जिससे program को maintain करना आसान हो जाता है।

1. What is a Module ?

- Module एक Python file (.py) होती है जिसमें कुछ useful code लिखा होता है।
- इसमें functions, classes, variables या executable statements हो सकते हैं।
- Module का use code को divide करने और reuse करने के लिए होता है।

Example :

```
# mymodule.py
def greet():
    print("Hello from my module!")
x = 10
```

4. Ways to Import a Module

Method	Syntax	Description
1. Import module	import module	पूरा module import होता है।
2. Import with alias	import module as m	Module को short name देना।
3. Import specific function	from module import function	केवल एक function import करना।
4. Import all	from module import *	Module के सभी members import करना (generally not recommended)।

6. Creating a Module

- हम अपना खुद का module एक .py file बनाकर create कर सकते हैं।
- उस file में functions, classes, variables या कोई भी valid Python code लिख सकते हैं।

Example : (file: calculator.py)

```
def add(a, b):
    return a + b
def subtract(a, b):
    return a - b
pi = 3.14159
```

अब इस module को दूसरी file में import करके use कर सकते हैं।

8. Some Useful Built-in Modules

Module	Use
math	Mathematical functions (sqrt, pi, factorial, etc.)
random	Generate random values
datetime	Work with date and time
os	Operating system related functions
sys	System specific parameters and functions
pathlib	Work with file paths

2. Types of Modules

- Built-in Modules : जो Python के साथ पहले से available होते हैं (जैसे math, random, os, datetime आदि)।
- User-defined Modules : जो हम खुद बनाते हैं (custom modules)।
- Third-party Modules : जो external libraries से install किए जाते हैं (जैसे numpy, pandas आदि)।

3. Importing a Module

- किसी module के functions, classes या variables को use करने के लिए उसे import करना होता है।
- import statement के द्वारा module को program में include किया जाता है।

Example :

```
import math          # पूरा module import करना
import random        # built-in module
import mymodule       # user-defined module
```

5. Using Module Members

- Import करने के बाद हम module के functions, variables और classes को dot (.) operator के साथ access करते हैं।

Example :

```
import math
print(math.sqrt(25))    # 5.0
print(math.pi)         # 3.14159...

import mymodule
mymodule.greet()       # Hello from my module!
print(mymodule.x)      # 10
```

7. Advantages of Modules

- Code reusability बढ़ती है।
- Large programs को छोटे-छोटे modules में divide किया जा सकता है।
- Code को maintain करना आसान हो जाता है।
- Built-in और third-party modules से already available functionality का use कर सकते हैं।
- Collaboration में help मिलती है क्योंकि different developers different modules पर work कर सकते हैं।

Key Points :

- Module एक Python file (.py) होती है।
- Modules code को organize और reuse करने में मदद करते हैं।
- import, from ... import, और as keyword का use किया जाता है।
- Built-in, user-defined और third-party modules होते हैं।
- Module के members को dot (.) operator से access करते हैं।
- Modules program को modular, maintainable और scalable बनाते हैं।

Topic : Standard Modules - sys

- sys Python का एक built-in (standard) module है, जो Python interpreter के साथ interact करने के लिए कई useful functions, variables और objects provide करता है।
- इस module का use system specific parameters और runtime environment की information प्राप्त करने के लिए किया जाता है।
- sys module का use command line arguments, exit करना, path को modify करना, और interpreter से related कई tasks के लिए होता है।

1. Importing sys Module

- sys एक built-in module है, इसलिए इसे अलग से install करने की आवश्यकता नहीं होती।
- इसे use करने के लिए simply import करना होता है।

Example :

```
import sys
print(sys)           # module location दिखाता है
print(sys.__name__)  # output : 'sys'
```

2. Command Line Arguments (sys.argv)

- sys.argv list command line से passed arguments को store करती है।
- पहला element (index 0) हमेशा script का नाम होता है।

Example : (file: demo.py)

```
import sys
print(sys.argv)      # Output (यदि run करें):
                     # ['demo.py', 'arg1', 'arg2']
```

3. Exiting from a Program (sys.exit)

- sys.exit() का use program को terminate करने के लिए किया जाता है।
- इसमें हम exit status भी दे सकते हैं (0 → success, non-zero → error)।

Example :

```
import sys
print("Program is running...")
sys.exit()           # program terminate
print("This will not execute") # यह नहीं चलेगा
```

4. Python Version Information

- sys module से हम current Python version की जानकारी प्राप्त कर सकते हैं।
- यह debugging और compatibility check के लिए useful है।

Example :

```
import sys
print(sys.version)    # full version info
print(sys.version_info) # version tuple
print(sys.platform)   # platform name
```

5. Module Search Path (sys.path)

- sys.path एक list होती है जिसमें उन directories के path होते हैं जहाँ Python modules को search करता है।
- हम इसमें अपने custom module के path को add भी कर सकते हैं।

Example :

```
import sys
print(sys.path)        # search path list
sys.path.append("D:/MyModules") # add new path
```

6. Standard Input, Output, Error

- sys module के द्वारा हम standard input (stdin), output (stdout) और error (stderr) को access कर सकते हैं।
- इनका use advanced I/O handling के लिए किया जाता है।

Example :

```
import sys
print("Output using sys.stdout", file=sys.stdout)
print("Error message", file=sys.stderr)
user_input = sys.stdin.readline() # input पढ़ने के लिए
print("You entered:", user_input)
```

7. Useful Attributes and Functions

Attribute / Function	Description
sys.argv	Command line arguments की list
sys.exit()	Program को terminate करता है
sys.version	Python version की जानकारी
sys.version_info	Version information as tuple
sys.platform	Platform (OS) का नाम
sys.path	Module search path की list
sys.stdin	Standard input stream
sys.stdout	Standard output stream
sys.stderr	Standard error stream
sys.modules	Loaded modules की dictionary

Key Points :

- sys एक built-in (standard) module है।
- इसका use Python interpreter और environment से related information प्राप्त करने के लिए किया जाता है।
- sys.argv command line arguments को store करती है।
- sys.exit() program को terminate करने के लिए होता है।
- sys.path में module search path होते हैं, जिन्हें modify किया जा सकता है।
- यह module debugging, automation और system specific tasks के लिए बहुत useful है।

Topic : Standard Modules - math

- math Python का एक built-in (standard) module है।
- यह module mathematical functions और constants provide करता है।
- math module में trigonometric, logarithmic, power, factorial, rounding आदि कई useful functions होते हैं।
- इसे use करने के लिए पहले import करना होता है। `import math`
- यह mainly numbers (int, float) के साथ work करता है।

Key Points :

- math एक built-in module है।
- Mathematical calculations के लिए use होता है।
- Functions को use करने के लिए import math करते हैं।
- यह केवल numeric data के साथ काम करता है।
- यह module fast और reliable है।

1. Importing math Module

- math module को use करने के लिए import करना आवश्यक होता है।
- इसके बाद हम इसके functions को dot (.) operator द्वारा access करते हैं।

Example :

```
import math
print(math.pi)      # 3.141592653589793
print(math.e)       # 2.718281828459045
```

2. Important Mathematical Constants

Constant	Use	Value
math.pi	Pi (π) का मान	3.141592653589793
math.e	Euler's number	2.718281828459045
math.tau	2 π का मान	6.283185307179586
math.inf	Positive infinity	inf
-math.inf	Negative infinity	-inf
math.nan	Not a Number	nan

3. Commonly Used Functions

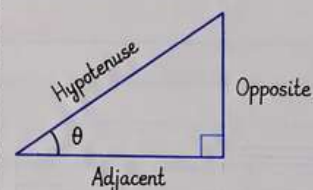
Function	Description	Example	Output
sqrt(x)	Square root	math.sqrt(25)	5.0
pow(x,y)	x raised to y	math.pow(2, 3)	8.0
ceil(x)	Next integer (up)	math.ceil(4.2)	5
floor(x)	Previous integer	math.floor(4.8)	4
factorial(x)	Factorial of x	math.factorial(5)	120
log(x)	Natural logarithm	math.log(10)	2.302...
log10(x)	Base 10 logarithm	math.log10(100)	2.0
sin(x)	Sine (radians)	math.sin(math.pi/2)	1.0
cos(x)	Cosine (radians)	math.cos(0)	1.0
tan(x)	Tangent (radians)	math.tan(math.pi/4)	1.0
radians(x)	Degree to radian	math.radians(180)	3.141...
degrees(x)	Radian to degree	math.degrees(math.pi)	180.0
fabs(x)	Absolute value	math.fabs(-7.5)	7.5
fmod(x,y)	Remainder	math.fmod(7, 3)	1.0

4. Trigonometric Functions (in Radians)

- Trigonometric functions angle को radian में लेते हैं।
- अगर angle degree में हो, तो पहले उसे radians() द्वारा convert करते हैं।

Example :

```
import math
angle = 30
rad = math.radians(angle) # degree to radian
print(math.sin(rad))      # 0.5
print(math.cos(rad))      # 0.8660...
print(math.tan(rad))      # 0.5773...
```



$$\sin(\theta) = \frac{\text{Opposite}}{\text{Hypotenuse}}$$

$$\cos(\theta) = \frac{\text{Adjacent}}{\text{Hypotenuse}}$$

$$\tan(\theta) = \frac{\text{Opposite}}{\text{Adjacent}}$$

5. Rounding Functions

- Rounding के लिए math module में ceil(), floor() और trunc() functions available होते हैं।
- ये functions number को integer में convert करने के लिए useful होते हैं।

Function	Description	Example	Output
ceil(x)	Next integer (up)	math.ceil(3.1)	4
floor(x)	Previous integer	math.floor(3.9)	3
trunc(x)	Remove decimal part	math.trunc(3.9)	3

Summary :

- math एक standard module है जो mathematical operations के लिए use होता है।
- import math के बाद इसके functions को dot (.) operator से access करते हैं।
- यह module scientific और general calculations दोनों के लिए बहुत useful है।
- सही function का चयन करके हम अलग-अलग mathematical problems को आसानी से solve कर सकते हैं।

6. Other Useful Functions

- math module में और भी कई useful functions होते हैं।
- ये functions different mathematical operations के लिए इस्तेमाल किए जाते हैं।

Function	Description	Example	Output
isfinite(x)	Check finite number	math.isfinite(10)	True
isinf(x)	Check infinity	math.isinf(math.inf)	True
isnan(x)	Check NaN	math.isnan(math.nan)	True
copysign(x,y)	Sign copy	math.copysign(3, -1)	-3.0
hypot(x,y)	$\sqrt{x^2 + y^2}$	math.hypot(3, 4)	5.0
gcd(x,y)	Greatest common divisor	math.gcd(16, 24)	8
lcm(x,y)	Least common multiple	math.lcm(4, 6)	12

Topic : Standard Modules - time

- time Python का एक built-in (standard) module है जो time से related functions provide करता है।
- इमें module का use current time प्राप्त करने, time को format करने, program को delay देने, performance measure करने और time conversion आदि के लिए किया जाता है।
- यह module system clock के साथ interact करता है और time से related कई useful functions देता है।
- time module में time data generally seconds (epoch) के रूप में represent होता है।

1. Importing time Module

- time module already Python में available होता है, इसलिए हमें इसे install करने की आवश्यकता नहीं होती है।
- इसे use करने के लिए simply import करते हैं।

Example :

```
import time
print("Time module imported successfully!")
```

2. Current Time Functions

- current time प्राप्त करने के लिए time() और ctime() functions का use किया जाता है।
- time() वर्तमान time को seconds (epoch) में return करता है।
- ctime() उस time को readable format में convert करता है।

Example :

```
import time
t = time.time()
print(t) # 1738664920.123 (epoch time)
print(time.ctime(t)) # Tue Feb 4 21:48:40 2025
```

3. Working with local time

- localtime() function epoch time को local time structure (struct_time) में convert करता है।
- यह year, month, day, hour, minute, second आदि की information देता है।

Example :

```
import time
t = time.time()
local = time.localtime(t)
print(local) # time.struct_time(tm_year=2025, ...)
print(local.tm_year) # 2025
print(local.tm_mon) # 2
print(local.tm_mday) # 4
```

struct_time (local time structure)

tm_year	Year (जैसे 2025)
tm_mon	Month (1 - 12)
tm_mday	Day (1 - 31)
tm_hour	Hour (0 - 23)
tm_min	Minute (0 - 59)
tm_sec	Second (0 - 59)
tm_wday	Weekday (0 = Monday)
tm_yday	Day of year (1 - 366)
tm_isdst	Daylight saving flag

4. Formatting Time

- strftime() function से हम time को अपनी आवश्यकता के अनुसार format में convert कर सकते हैं।
- इसके लिए format codes का use किया जाता है।

Example :

```
import time
now = time.localtime()
print(time.strftime("%Y-%m-%d", now)) # 2025-02-04
print(time.strftime("%d/%m/%Y %H:%M:%S", now))
# 04/02/2025 21:48:40
```

Common Format Codes

Code	Meaning
%Y	4 digit year (2025)
%m	Month (01 - 12)
%d	Day (01 - 31)
%H	Hour (00 - 23)
%M	Minute (00 - 59)
%S	Second (00 - 59)
%A	Weekday name (Tuesday)
%B	Month name (February)

5. Pausing Execution (Sleep)

- sleep() function program को कुछ समय के लिए pause करता है।
- इसमें time seconds में दिया जाता है।

Example :

```
import time
print("Start")
time.sleep(2) # 2 seconds pause
print("End (after 2 seconds)")
```

6. Measuring Execution Time

- किसी code या program के execution time को measure करने के लिए time() function का use किया जाता है।

Example :

```
import time
start = time.time()
for i in range(1000000):
    pass
end = time.time()
print("Execution time:", end - start, "seconds")
```

7. Other Useful Functions

Function	Description
time.time()	Current time in seconds (epoch)
time.ctime()	Convert time to readable string
time.localtime()	Convert to local time structure
time.strftime()	Format time as string
time.sleep()	Pause execution for given seconds
time.gmtime()	Convert to UTC time structure
time.perf_counter()	High resolution timer (for performance measurement)

Key Points :

- time module एक built-in module है।
- time() current time seconds में देता है।
- ctime() readable format में time देता है।
- strftime() से custom format बनाया जा सकता है।
- sleep() program को delay करने के लिए use होता है।
- time module performance measurement के लिए भी useful है।
- यह module कई real-life applications (logging, scheduling, etc.) में use किया जाता है।

Topic : The dir() Function

- `dir()` Python का एक built-in function है, जो किसी object के सभी available attributes (जैसे variables, functions, methods, classes आदि) की list return करता है।
- यह function introspection के लिए उपयोगी होता है, क्योंकि इसके द्वारा हम किसी module, class, object या current namespace में मौजूद names को देख सकते हैं।
- `dir()` हमेशा sorted (alphabetical order) में एक list return करता है।

1. Syntax

```
dir([object])
```

- `object (optional)`: वो object जिसका attributes देखना है।
- अगर object नहीं दिया जाता, तो `dir()` current local scope के names की list return करता है।

2. Return Value

- `dir()` हमेशा एक list return करता है जिसमें strings के रूप में सभी valid names होते हैं।
- इन names में variables, functions, classes, methods और other attributes शामिल होते हैं।

3. Common Use Cases

- किसी module में उपलब्ध functions और variables देखने के लिए।
- किसी object के methods और attributes जानने के लिए।
- Exploring Python library/module के contents के लिए।
- Interactive Python shell में debugging और learning के लिए।

4. Examples

a) Current Local Scope

```
>>> a = 10
>>> b = 'hello'
>>> dir()
```

Output (partial):

```
['__annotations__',
'__builtins__',
'__doc__',
'a', 'b', ...]
```

Note: `dir()` बिना argument के current scope के सभी names दिखाता है।

b) On a Module

```
>>> import math
>>> dir(math)
```

Output (partial):

```
['__doc__', '__file__',
'acos', 'asin', 'atan',
'ceil', 'cos', 'sin', ...]
```

Note: इससे `math` module के सभी functions और attributes की list प्राप्त होती है।

c) On a Built-in Type

```
>>> dir(list)
```

Output (partial):

```
['__add__', '__class__',
'__contains__', 'append',
'clear', 'copy', 'count', ...]
```

Note: `dir(list)` `list` class के सभी methods दिखाता है।

d) On a User-defined Object

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age
    def show(self):
        print(self.name, self.age)
s = Student("Rahul", 20)
print(dir(s))
```

Output (partial):

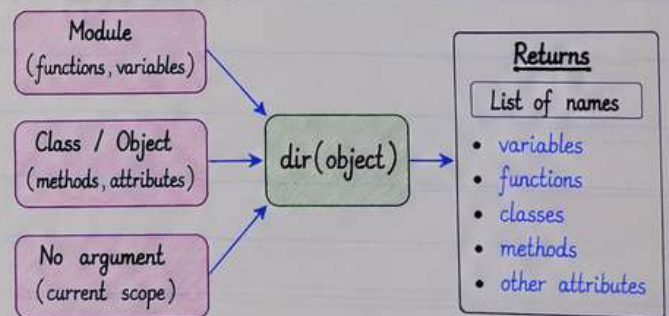
```
['__class__', '__delattr__', '__dict__', '__dir__',
'__doc__', '__eq__', 'age', 'name', 'show', ...]
```

Note: इसमें object के attributes (name, age) और methods (show) भी दिखाई दे रहे हैं।

5. Important Points

- `dir()` केवल valid attributes की list दिखाता है, उनके values नहीं।
- यह list हमेशा alphabetical order में होती है।
- `dir()` अलग-अलग प्रकार के objects (module, class, instance, function आदि) पर काम करता है।
- यह private और special methods (जैसे `__init__`, `__str__`, `__dict__` आदि) भी दिखाता है।
- यह debugging, exploration और learning के लिए बहुत useful है।

6. Visualization



7. Example Output in Python Shell

```
>>> import time
>>> dir(time)
['CLOCK_BOOTTIME', 'CLOCK_MONOTONIC', 'ctime',
'daylight', 'gmtime', 'localtime', 'mktime', 'sleep',
'strftime', 'time', 'timezone', 'tzname', ...]
```

Key Takeaways:

- `dir()` Python का built-in function है।
- यह किसी object के सभी valid attributes की list return करता है।
- यह values नहीं, केवल names दिखाता है।
- यह introspection, debugging और exploration के लिए बहुत useful है।

Unit - 7 : Exceptions

Topic : Errors, Runtime Errors

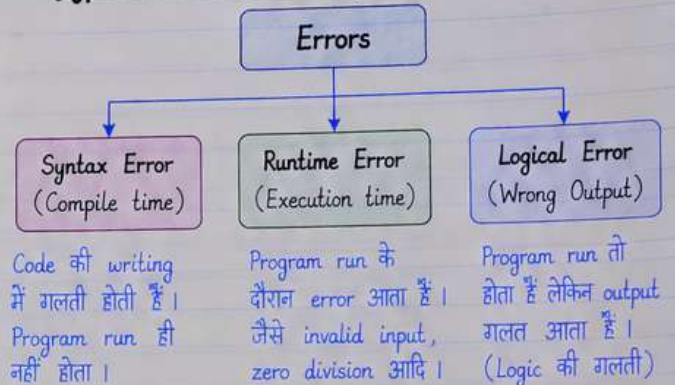
- Python program को run करते समय कभी-कभी errors (तुटियाँ) आती हैं, जिनके कारण program सही से execute नहीं हो पाता।
- Errors के different types होते हैं जैसे Syntax Error, Runtime Error और Logical Error।
- Runtime Errors वे errors होते हैं जो program run होने के दौरान (execution time पर) occur होते हैं।
- इन errors को handle करने के लिए Python में Exception Handling का use किया जाता है।

1. What is an Error ?

- Error एक ऐसी स्थिति होती है जिसमें program अपक्षित रूप से काम नहीं कर पाता।
- यह code में किसी गलती (mistake) या गलत input के कारण होता है।
- Python में error आने पर interpreter एक error message दिखाता है जिसे traceback कहते हैं।

Note : सभी errors programs को रोक देते हैं, जब तक उनको handle न किया जाए।

2. Types of Errors in Python



3. Syntax Error

- जब code Python के syntax rules के अनुसार नहीं लिखा होता, तो Syntax Error आता है।
- यह compile time पर detect हो जाता है, इसलिए program run नहीं होता।
- Interpreter तुरंत error message दिखाता है।

Example :

```
print("Hello"),
# SyntaxError: unterminated string literal
```

4. Runtime Error

- Runtime Error program के execution के दौरान आता है।
- यह तब होता है जब program syntactically correct होता है, लेकिन run-time पर कोई invalid operation किया जाता है।
- जैसे किसी संख्या को 0 से divide करना, invalid input देना, file न मिलना आदि।

Example :

```
a = 10
b = 0
print(a / b) # ZeroDivisionError: division by zero
```

5. Logical Error

- Logical Error में code का syntax सही होता है और program run भी हो जाता है, लेकिन output अपक्षित (expected) नहीं होता है।
- यह programmer की logic में गलती के कारण होता है।

Example :

```
a = 5
b = 3
print(a + b) # यदि हमें multiplication चाहिए था
# तो यह logic error है (output गलत)
```

6. Common Runtime Errors

Error Type	Description
ZeroDivisionError	किसी संख्या को 0 से divide करने पर
TypeError	गलत data type के operation पर
ValueError	Invalid value देने पर (जैसे int("abc"))
IndexError	List के index range से बाहर access करने पर
KeyError	Dictionary में non-existent key access करने पर
FileNotFoundError	File न मिलने पर
NameError	Undefined variable के use करने पर
AttributeError	Invalid attribute access करने पर

7. Runtime Error Example (Different Types)

ValueError

```
n = int("abc")
# ValueError: invalid literal
# for int() with base 10
```

TypeError

```
a = 5
print(a + "hello")
# TypeError: unsupported
# operand type(s)
```

IndexError

```
lst = [1, 2, 3]
print(lst[5])
# IndexError: list index
# out of range
```

FileNotFoundError

```
f = open("data.txt")
# FileNotFoundError:
# [Errno 2] No such file
```

8. Key Points

- Errors program का normal flow रोक देते हैं।
- Runtime Errors execution time पर occur होते हैं।
- हर error के लिए Python एक specific exception class देता है।
- Error message (traceback) से error का कारण पता चलता है।
- इन errors को handle करने के लिए try-except block का use किया जाता है (यह अगले topic में पढ़ेंगे)।

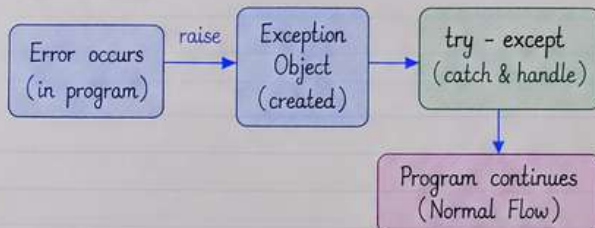
Topic : The Exception Model , Exception Hierarchy

- Python में Exception एक event होता है जो program के execution के दौरान occur होता है, जब कोई error आती है और normal flow of program break हो जाता है।
- Exception Model यह define करता है कि exception को कैसे handle किया जाता है।
- Exception Hierarchy, Python में built-in exception classes की एक hierarchical structure है।
- यह structure हमें different types की exceptions को understand और handle करने में मदद करती है।

1. The Exception Model

- Exception Model बताता है कि जब program में कोई error होती है, तब Python उस error को एक Exception object के रूप में create करता है और उसे raise करता है।
- यह exception object में error से related information (जैसे error type, message, stack trace) होती है।
- Program में try-except blocks के द्वारा इन exceptions को catch और handle किया जाता है ताकि program abnormally terminate न हो।

Exception Flow



2. How Exception Handling Works?

- ① Error occur होती है (जैसे invalid input, file not found आदि)।
- ② Python उस error के लिए appropriate exception class का object create करता है।
- ③ यह exception object को raise करता है।
- ④ Program try block में उस exception को catch करता है। (अगर matching except block है)।
- ⑤ Exception handle होने के बाद program normal flow में continue करता है।

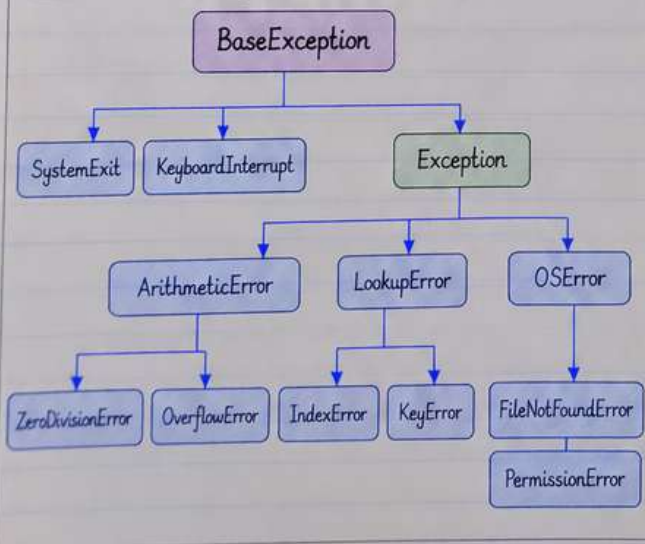
Example :

```
try:                                     # try block
    a = 10
    b = 0
    print(a / b)                         # error (division by zero)
except ZeroDivisionError as e:           # specific exception handle
    print("Error:", e)                   # error message print
print("Program continues...")            # यह line execute होगी
```

3. Exception Hierarchy

- Python में सभी built-in exceptions एक class hierarchy में organized होती हैं।
- Base class "BaseException" होती है, जो सभी exceptions की parent class है।
- "Exception" class, BaseException की subclass है और अधिकतर built-in exceptions इसी के under आती हैं।
- इस hierarchy की मदद से हम general या specific exceptions को easily handle कर सकते हैं।

Exception Hierarchy (Simplified View)



4. Some Common Exceptions

Exception Class	Meaning / When it occurs
ZeroDivisionError	जब किसी संख्या को 0 से divide किया जाता है।
ValueError	जब function को सही type का value नहीं मिलता।
TypeError	जब wrong type का operand दिया जाता है।
IndexError	जब list/tuple के valid range से बाहर index access किया जाता है।
KeyError	जब dictionary में non-existent key access की जाती है।
FileNotFoundError	जब specified file नहीं मिलती है।
PermissionError	जब file या resource के लिए permission नहीं होती।
AttributeError	जब object का कोई attribute मौजूद नहीं होता।

5. Key Points

- Exception program के runtime पर occur होती है।
- सभी exceptions "BaseException" class से derive होती हैं।
- try-except का use करके exceptions को handle किया जाता है।
- Multiple except blocks का use करके different exceptions को handle किया जा सकता है।
- अगर exception handle नहीं की जाती, तो program terminate हो जाता है।
- Exception hierarchy हमें सही exception class choose करने में मदद करती है।

Conclusion :

Exception Model और Exception Hierarchy, Python में robust और error-free program लिखने के लिए बहुत important है। इनकी मदद से हम runtime errors को gracefully handle कर सकते हैं और program के crash होने से बचा सकते हैं।

Topic : Handling Multiple Exceptions

- किसी program में एक से अधिक (multiple) exceptions आ सकती हैं, इसलिए हमें उन्हें properly handle करना होता है।
- Python में हम multiple except blocks का use करके different types की exceptions की handle कर सकते हैं।
- यह program की crash होने से बचाता है और हर error के लिए appropriate message देता है।
- Multiple exceptions की handle करने के लिए try-except structure का use किया जाता है।

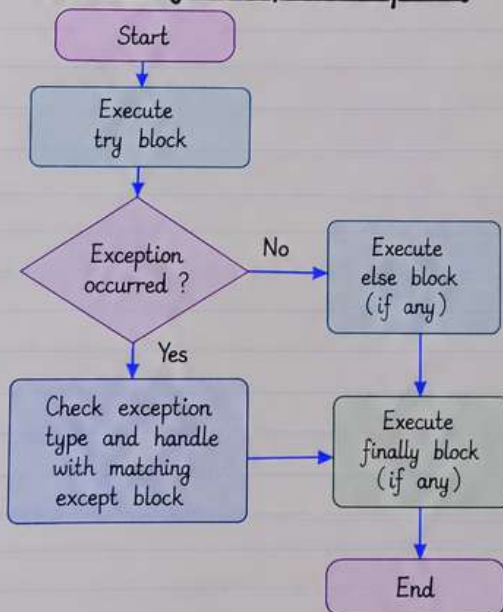
1. Syntax

```
try:
    # code that may raise an exception
    statement(s)
except ExceptionType1:
    # handle exception 1
    statement(s)
except ExceptionType2:
    # handle exception 2
    statement(s)
    :
except ExceptionTypeN:
    # handle exception N
    statement(s)
else:
    # (optional) run if no exception occurs
    statement(s)
finally:
    # (optional) always runs
    statement(s)
```

2. Example : Handling Multiple Exceptions

```
try:
    a = int(input("Enter first number: "))
    b = int(input("Enter second number: "))
    result = a / b
    print("Result = ", result)
except ValueError:
    print("Invalid input ! Please enter an integer.")
except ZeroDivisionError:
    print("Cannot divide by zero.")
except Exception as e:
    print("An unexpected error occurred :", e)
else:
    print("Program executed successfully.")
finally:
    print("This block will always run.")
```

3. Flow of Handling Multiple Exceptions



4. Common Exceptions

Exception Class	When it Occurs
ValueError	Invalid value is provided (e.g. converting non-integer to int)
ZeroDivisionError	Division by zero is performed
TypeError	Operation is performed on incompatible types
FileNotFoundError	Specified file is not found
IndexError	List index is out of range
KeyError	Dictionary key is not found
AttributeError	Invalid attribute access
IOError / OSError	Input/Output related error

6. Key Points

- Multiple except blocks का use करके different exceptions को handle किया जाता है।
- Exception handling se program robust और stable बनता है।
- Specific exception classes को पहले लिखना चाहिए, और general Exception को बाद में।
- else block तब execute होता है जब कोई exception नहीं आती।
- finally block हमेशा execute होता है चाहे exception आए या न आए।
- Proper error messages देने से debugging और user experience बेहतर होता है।

5. Example : User Input with Multiple Exceptions

```
try:
    num = int(input("Enter a number: "))
    print("You entered:", num)
except ValueError:
    print("That is not a valid integer.")
except KeyboardInterrupt:
    print("Input cancelled by user.")
except Exception as e:
    print("Some error occurred :", e)
```

Conclusion :

Handling multiple exceptions is an important concept in Python जो program को unexpected errors से बचाता है और उसे gracefully handle करने में मदद करता है।

Topic : Raise , assert

- Python में raise और assert दोनों का use exceptions के साथ होता है।
- raise का use किसी specific exception को manually trigger (raise) करने के लिए किया जाता है।
- assert का use किसी condition (expression) की जाँच करने के लिए किया जाता है, जो False होने पर AssertionError raise करता है।
- ये दोनों program को error handling और debugging में मदद करते हैं।

1. raise Statement

- raise statement का use किसी exception को explicitly raise (trigger) करने के लिए किया जाता है।
- हम built-in exception या user-defined exception को raise कर सकते हैं।
- raise के साथ exception class और optional error message दिया जाता है।
- यह तब useful होता है जब हमें किसी invalid condition को detect करके खुद से exception generate करना हो।

Syntax :

```
raise ExceptionClass("error message")  
(message optional है)
```

2. Examples of raise

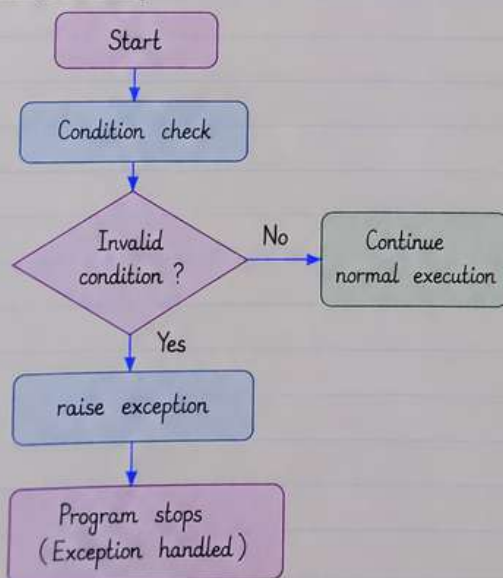
Example 1 : using built-in exception

```
age = -5  
if age < 0:  
    raise ValueError("Age cannot be negative")  
print("Age is :", age) # Age cannot be negative
```

Example 2 : using user-defined exception

```
class InvalidMarksError(Exception):  
    pass  
marks = 150  
if marks > 100:  
    raise InvalidMarksError("Marks cannot be greater than 100")  
# Marks cannot be greater than 100
```

3. Flow of raise



4. assert Statement

- assert statement का use किसी condition (expression) को test करने के लिए किया जाता है।
- अगर condition True होती है, तो program normal रूप से continue करता है।
- अगर condition False होती है, तो AssertionError raise होता है।
- यह mainly debugging और development के समय useful होता है, ताकि logical errors को जल्दी पकड़ा जा सके।
- assert का use program की correctness को verify करने के लिए किया जाता है।

Syntax :

```
assert condition, "error message"  
(message optional है)
```

5. Examples of assert

Example 1 : simple assert

```
age = 18  
assert age >= 0, "Age cannot be negative"  
print("Age is valid :", age) # Age is valid : 18
```

Example 2 : assert with condition

```
x = 10  
y = 0  
assert y != 0, "y should not be zero"  
result = x / y  
print("Result :", result)  
# AssertionError: y should not be zero
```

6. Difference between raise and assert

Point	raise	assert
Purpose	Manually trigger an exception	Test a condition (mainly for debugging)
Exception Type	Any exception (built-in or user-defined)	Always raises AssertionError
Condition	Used when an error condition is detected	Used to check if a condition is True
Use Case	For proper error handling in program	For internal checks during development
Message	Optional	Optional
Best Practice	Used in actual program logic	Used for debugging (not for user input validation)

7. Key Points

- raise का use custom error handling के लिए करें।
- assert का use logical conditions को verify करने के लिए करें।
- assert statements को production code में कम use करना चाहिए।
- raise से हम किसी भी exception class को raise कर सकते हैं।
- assert सिर्फ AssertionError raise करता है।
- दोनों का सही use program को robust और error-free बनाता है।

Topic : in one Example full unit coding.

(Exceptions : Errors, Runtime Errors, Exception Handling, raise, assert)

- इस example में हम अब तक पढ़े गए पूरे unit (Exceptions) के सभी important concepts को एक program में use करेंगे ताकि complete flow समझ आ सके।
- इस program में हम different types की errors को handle करेंगे, raise और assert का use करेंगे, और multiple exceptions को manage करेंगे।
- यह एक realistic example है जो user से input लेकर कुछ operations perform करता है।

1. Problem Statement

User से दो numbers लेना है और उन पर division, list access और file reading जैसी operations perform करनी हैं। Program में सभी possible errors को handle करना है, validate करने के लिए assert use करता है और custom exception raise करती है।

2. Complete Python Program

```
# Full Unit Example : Exception Handling
# with Errors, Runtime Errors, raise and assert

class NegativeNumberError(Exception):
    """Custom exception for negative numbers"""
    pass

def process_numbers():
    try:
        # Taking input from user
        a = int(input("Enter first number: "))
        b = int(input("Enter second number: "))
        # Using assert to validate input
        assert a >= 0 and b >= 0, "Numbers must be non-negative"
        # Custom exception using raise
        if a < 0 or b < 0:
            raise NegativeNumberError("Negative number entered!")
        # Division operation (may raise ZeroDivisionError)
        result = a / b
        print("Division Result:", result)
        # List access (may raise IndexError)
        nums = [10, 20, 30]
        index = int(input("Enter index to access list (0-2): "))
        print("List value:", nums[index])
        # File reading (may raise FileNotFoundError)
        filename = input("Enter filename to read:")
        with open(filename, "r") as f:
            data = f.read()
            print("\nFile Content:")
            print(data)
    except ZeroDivisionError:
        print("Error: Cannot divide by zero.")
    except IndexError:
        print("Error: Invalid index! Please enter 0, 1 or 2:")
    except FileNotFoundError:
        print("Error: File not found! Please check filename.")
    except NegativeNumberError as ne:
        print("Custom Error:", ne)
    except ValueError:
        print("Error: Please enter a valid integer.")
    except AssertionError as ae:
        print("Assertion Error:", ae)
    except Exception as e:
        print("An unexpected error occurred:", e)
    else:
        print("\nAll operations completed successfully.")
    finally:
        print("\nProgram finished. (finally block always runs)")

# Calling the function
process_numbers()
```

3. Sample Run (Output)

```
Enter first number: 10
Enter second number: 2
Division Result : 5.0
Enter index to access list (0-2): 1
List value : 20
Enter filename to read: demo.txt

File Content:
Hello Python!
This is a sample file.

All operations completed successfully.
Program finished. (finally block always runs)
```

Example : When error occurs

```
Enter first number: 10
Enter second number: 0
Error: Cannot divide by zero.
Program finished. (finally block always runs)
```

Example : Negative number (raise)

```
Enter first number: -5
Enter second number: 3
Assertion Error: Numbers must be non-negative
Program finished. (finally block always runs)
```

4. Explanation

- **try block** : Risky code (जैसे input, division, file handling) को execute करता है।
- **assert** : Input की validity check करने के लिए use किया (अगर condition False हूँ तो AssertionError आता है)।
- **raise** : Custom exception (NegativeNumberError) को manually trigger करने के लिए use किया।
- **Multiple except blocks** : Different errors (ZeroDivisionError, IndexError, FileNotFoundError, ValueError, AssertionError) को अलग-अलग handle किया।
- **else block** : जब कोई error नहीं आता, तब execute होता है।
- **finally block** : यह हमेशा execute होता है, चाहे error आए या न आए।

5. Key Takeaways

- Exceptions program को crash होने से बचाते हैं।
- assert और raise debugging में बहुत useful हैं।
- Multiple exceptions को handle करने से program robust बनता है।
- try-except-else-finally का proper use program को reliable और user-friendly बनाता है।

Unit - 8 : Input and Output

Topic : Introduction, Data Streams

1. Introduction

- Input and Output (I/O) computer system का एक important part है जो program को user, files और other devices के साथ communicate करने की सुविधा देता है।
- Program को data प्राप्त करने के लिए Input की आवश्यकता होती है और result को बाहर भेजने के लिए Output की।
- Python में Input and Output operations को handle करने के लिए built-in functions और I/O classes का use किया जाता है।
- I/O operations का use file processing, user interaction, device communication और data transfer के लिए होता है।
- Python में सभी I/O operations "streams" के रूप में handle किए जाते हैं, जिससे data को एक sequence (flow) में पढ़ा या लिखा जा सकता है।

Note : Input → program के लिए data प्राप्त करना
Output → program द्वारा result बाहर भेजना

2. Need of Input and Output

- User से data लेने के लिए (जैसे keyboard input)
- Program का result दिखाने के लिए (जैसे screen output)
- File में data store करने और पढ़ने के लिए
- Other devices (जैसे printer, network, sensors) के साथ communication के लिए
- Data को permanently storage में रखने के लिए

3. Input - Output Process

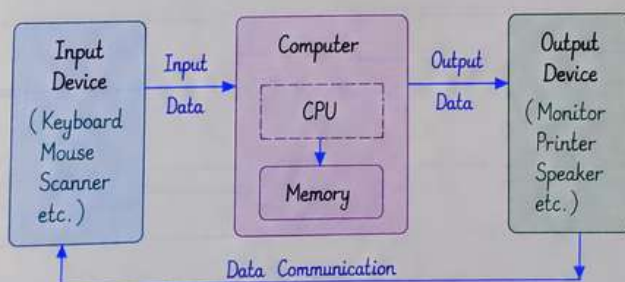


Fig : Input - Processing - Output

4. Standard Input, Output and Error Streams

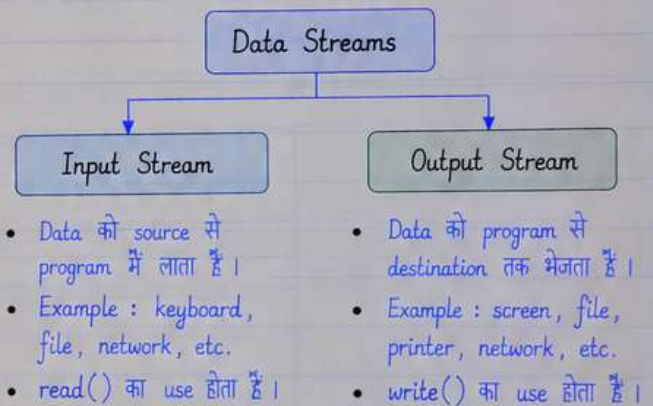
- Python में कुछ pre-defined standard streams होती हैं :
 - ① `sys.stdin` → standard input stream (keyboard input)
 - ② `sys.stdout` → standard output stream (screen output)
 - ③ `sys.stderr` → standard error stream (error messages)
- ये streams program को input लेने, output देने और error messages दिखाने के लिए automatically available होती हैं।

5. What are Data Streams?

- Data Stream एक continuous flow of data होता है, जिसके द्वारा data sequential form में एक source से दूसरे destination तक भेजा या प्राप्त किया जाता है।
- Stream को एक "flow of bytes" के रूप में देखा जाता है।
- Python में input और output दोनों stream के रूप में handle किए जाते हैं।
- Stream से data एक-एक करके (sequentially) read या write किया जाता है।
- Data Stream file, device, network connection या memory से हो सकता है।

Data Stream = Sequence of data (bytes/characters) flowing from source to destination.

6. Types of Data Streams



7. Stream Operations

Operation	Description	Common Methods
Open	Stream को open करना	<code>open()</code>
Read	Stream से data पढ़ना	<code>read()</code> , <code>readline()</code> , <code>readlines()</code>
Write	Stream में data लिखना	<code>write()</code> , <code>writelines()</code>
Close	Stream को close करना	<code>close()</code>
Flush	Buffer का data तुरंत write करना	<code>flush()</code>

8. Key Points

- Input और Output operations Python में streams के रूप में work करते हैं।
- Data Stream एक continuous flow of data होता है।
- Standard streams (`stdin`, `stdout`, `stderr`) हमेशा available रहती हैं।
- Streams के माध्यम से हम files, devices और network के साथ easily communicate कर सकते हैं।
- Proper stream handling से program अधिक efficient और error-free बनता है।

Topic : Creating Your Own Data Streams , Access Modes

1. Creating Your Own Data Streams

- Python में हम अपने program के लिए user-defined data streams create कर सकते हैं, ताकि हम files, devices या other sources से data read और write कर सकें।
- जब हम कोई file open करते हैं, तो Python एक file object (create) करता है, जो एक data stream की तरह काम करता है।
- इस data stream के माध्यम से हम data को sequentially read या write कर सकते हैं।
- यह हमें program में external data (files) को handle करने की सुविधा देता है।

Creating a Data Stream (File)



2. Steps to Create and Use a Data Stream

- ① Open the file using open() function.
- ② Perform read() or write() operations.
- ③ Process the data as required.
- ④ Close the file using close() function (or use with statement).
- ⑤ Handle exceptions (if any).

3. Example : Creating and Using a Data Stream

Creating your own data stream (file)

try:

Open a file in write mode

f = open("sample.txt", "w")

f.write("Hello! This is my own data stream.\n")

f.write("Python makes it easy to work with files.\n")

f.close()

print("Data written successfully.")

except Exception as e:

print("Error :", e)

try:

Open the same file in read mode

f = open("sample.txt", "r")

print("\nFile Content :")

data = f.read()

print(data)

f.close()

except Exception as e:

print("Error :", e)

Output :

Data written successfully.

File Content :

Hello! This is my own data stream.

Python makes it easy to work with files.

4. Access Modes

- Access mode यह decide करता है कि file को किस purpose के लिए open किया जा रहा है, जैसे read करने के लिए, write करने के लिए या दोनों के लिए।
- Python में open() function के साथ access mode specify किया जाता है।
- हर mode का अपना specific काम होता है।
- अगर file exist नहीं करती है और हम write mode में open करते हैं, तो Python नई file create कर देता है।

5. Common Access Modes

Mode	Full Form	Description
r	Read	File को read करने के लिए open करता है। (File exist होना चाहिए)
w	Write	File को write करने के लिए open करता है। (अगर file exist नहीं है तो नई file create हो जाती है)
a	Append	File के end में data जोड़ने के लिए open करता है। (File exist नहीं है तो नई file create होती है)
r+	Read and Write	File को read और write दोनों के लिए open करता है। (File exist होना चाहिए)
w+	Write and Read	File को write और read दोनों के लिए open करता है। (नई file create हो जाती है)
a+	Append and Read	File में data append और read दोनों के लिए open करता है। (File exist नहीं है तो नई file create होती है)

6. Example : Different Access Modes

Write Mode (w)

f = open("demo.txt", "w")

f.write("This is write mode.\n")

f.close()

Append Mode (a)

f = open("demo.txt", "a")

f.write("This is append mode.\n")

f.close()

Read Mode (r)

f = open("demo.txt", "r")

print(f.read())

f.close()

Read and Write Mode (r+)

f = open("demo.txt", "r+")

f.write("Using r+ mode.\n")

f.seek(0) # move to start

print(f.read())

f.close()

7. Key Points

- Data stream एक continuous flow of data होता है।
- File handling के लिए open() function का use किया जाता है।
- Access mode यह तय करता है कि file को किस operation के लिए open करना है।
- Always close the file after use (या with statement का use करें)।
- Invalid mode या गलत operation करने पर error आ सकता है।
- Proper access mode का use करने से data loss से बचा जा सकता है।

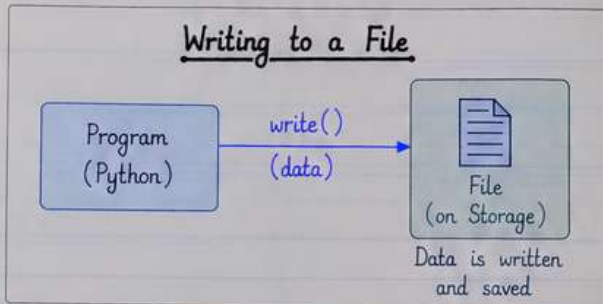
Conclusion :

Python में अपने data streams बनाना और सही access mode का use करना file handling को आसान, flexible और efficient बनाता है।

Topic : Writing Data to a File • Reading Data From a File

1. Writing Data to a File (Introduction)

- File में data write करने के लिए हम `open()` function का use करते हैं।
- File को write mode ("w"), append mode ("a") आदि में open किया जा सकता है।
- `write()` function द्वारा string या data file में लिखा जाता है।
- अगर file exist नहीं करती है, तो write mode में open करने पर नई file create हो जाती है।
- File में data लिखने के बाद file को `close()` करना जरूरी होता है ताकि data properly save हो जाए।



2. Syntax

```
f = open("filename.txt", "w") # open in write mode
f.write("data...")           # write data
f.close()                    # close the file
```

3. Example : Writing Data to a File

```
# Example : write some text to a file
f = open("sample.txt", "w")
f.write("Hello!\n")
f.write("This is written to a file.\n")
f.write("Python is easy to learn.")
f.close()
print("Data written successfully.")
```

4. Modes for Writing

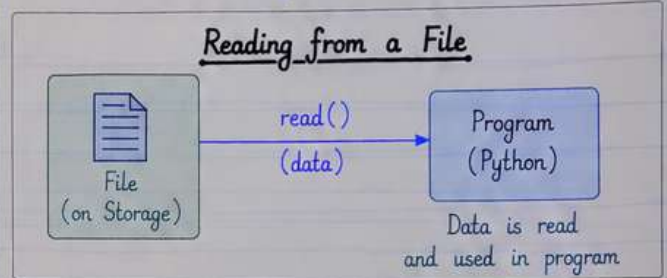
Mode	Description
"w"	Write mode (old data overwrite)
"a"	Append mode (data end में add होता है)
"x"	Create new file (अगर file exist नहीं करती है)
"w+"	Read and write (overwrite)
"a+"	Read and write (append)

Note :

- `write()` के लिए data string के रूप में होता चाहिए।
- अगर हमें numbers लिखने हैं, तो उन्हें string में convert करना होगा (जैसे `str(10)`)।
- File close करना हमेशा अच्छी practice है।

5. Reading Data From a File (Introduction)

- File से data read करने के लिए हम `open()` function का use read mode ("r") में करते हैं।
- `read()` function file का पूरा content एक string के रूप में return करता है।
- `readline()` function file की एक line पढ़ता है।
- `readlines()` function file की सभी lines को list के रूप में return करता है।
- अगर file exist नहीं करती है, तो `FileNotFoundError` मिलता है, इसलिए error handling करना अच्छी practice है।



6. Syntax

```
f = open("filename.txt", "r") # open in read mode
data = f.read()              # read all data
f.close()                     # close the file
```

7. Example : Reading Data from a File

```
# Example : read the content of a file
f = open("sample.txt", "r")
data = f.read()
print("File Content :")
print(data)
f.close()
```

8. Other Methods for Reading

Method	Description
<code>read()</code>	पूरा file content पढ़ता है (string के रूप में)
<code>readline()</code>	एक line पढ़ता है
<code>readlines()</code>	सभी lines को list में return करता है
<code>for line in f</code>	file की line by line पढ़ने के लिए use होता है

9. Example : Read Line by Line

```
f = open("sample.txt", "r")
print("Reading line by line:")
for line in f:
    print(line.strip()) # strip() नए line हटाता है
f.close()
```

Key Points :

- File handling से data को permanent storage में save और retrieve किया जा सकता है।
- Proper mode का use करें (write/read/append)।
- File operations के दौरान error handling जरूर करें।
- काम पूरा होने के बाद file को `close()` करना न भूलें।

Topic : Additional File Methods, Using Pipes as Data Streams

1. Additional File Methods (Introduction)

- Python file objects provide several useful methods jo file handling ko aur flexible बनाते हैं।
- ये methods read, write, file position, status check और file management के लिए use होते हैं।
- इन methods का use करके हम file के साथ better control और efficient data processing कर सकते हैं।

2. Common Additional File Methods

Method	Description
f.read(size)	size bytes का data read करता है। (यदि size नहीं दिया तो पूरा file read करेगा)
f.write(data)	data को file में लिखता है।
f.readline()	file की एक line read करता है।
f.readlines()	file की सभी lines को list के रूप में return करता है।
f.seek(offset)	file pointer को specified position पर ले जाता है।
f.tell()	current position (file pointer) को return करता है।
f.flush()	buffer का data तुरंत file में लिखता है।
f.close()	file को close करता है।
f.readable()	check करता है कि file readable है या नहीं।
f.writable()	check करता है कि file writable है या नहीं।
f.seekable()	check करता है कि file seek (reposition) किया जा सकता है या नहीं।

3. Example : Using Additional File Methods

```
# Example to demonstrate additional file methods
f = open("example.txt", "r+")
print("Is file readable?", f.readable())
print("Is file writable?", f.writable())
print("Current position :", f.tell())
line = f.readline()
print("First line :", line)
f.seek(0) # move to beginning
print("After seek, position :", f.tell())
f.close()
```

4. Key Points (File Methods)

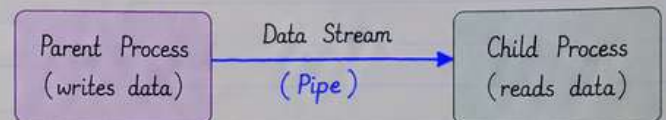
- seek() का use file pointer ko move karne के लिए होता है।
- tell() current position प्राप्त करने के लिए use होता है।
- readline() और readlines() sequential reading के लिए useful हैं।
- flush() का use buffer का data तुरंत save करने के लिए होता है।
- File को close() करना हमेशा good practice है।

5. Using Pipes as Data Streams (Introduction)

- Pipe एक communication mechanism है जिसका use दो processes के बीच data transfer करने के लिए होता है।
- Python में pipes को data streams के रूप में use किया जा सकता है।
- Pipes के माध्यम से एक program दूसरे program को data भेजता है और दूसरा program उस data को read कर सकता है।
- यह technique inter-process communication (IPC) के लिए बहुत useful होती है।

6. How Pipes Work ?

- एक process (parent) pipe में data लिखता है।
- दूसरा process (child) उसी pipe से data पढ़ता है।
- Pipe एक unidirectional data flow provide करता है (आमतौर पर parent → child)।
- Python में os module या subprocess module का use करके pipes create किए जा सकते हैं।



7. Example : Using Pipe with os Module

```
import os
# Create a pipe
read_fd, write_fd = os.pipe()
pid = os.fork()
if pid > 0:
    # Parent process
    message = "Hello from parent!"
    os.write(write_fd, message.encode())
    os.close(write_fd)
else:
    # Child process
    os.close(write_fd)
    data = os.read(read_fd, 1024)
    print("Child received :", data.decode())
    os.close(read_fd)
```

8. Key Points (Pipes)

- Pipe ek simple और efficient way है दो processes के बीच data transfer करने का।
- os.pipe() दो file descriptors (read और write) return करता है।
- os.fork() द्वारा child process create किया जाता है।
- Parent process pipe में data लिखता है और child process उस data को read करता है।
- Pipes का use large data transfer और real-time communication में भी किया जाता है।

Topic : Handling IO Exceptions

1. Introduction

- File handling करते समय कई बार Input/Output (I/O) related errors आ सकते हैं, जैसे file न मिलना, permission का न होना, disk full होना आदि।
- इन परिस्थितियों में program terminate हो सकता है, इसलिए I/O operations में exception handling करना बहुत जरूरी है।
- Python में I/O errors सामान्यतः OSError class के under आते हैं, जिसके कई specific subclasses होती हैं (जैसे FileNotFoundError)
- Try-except block का use करके इन exceptions को handle किया जाता है ताकि program crash न हो और user को meaningful error message मिल सके।

2. Common IO Exceptions

Exception	When it occurs	Example Situation
FileNotFoundError	जब specified file exist नहीं करती	open("data.txt")
PermissionError	जब file को access करने की permission नहीं होती	read/write protected file
IsADirectoryError	जब file की जगह directory को open करने की कोशिश होती है	open("folder/", "r")
NotADirectoryError	जब directory की जगह file को directory की तरह use किया जाता है	open("file.txt/abc")
OSError	General I/O related error	Disk error, device not available etc.

3. Basic Example : Handling IO Exception

Example: reading a file with exception handling
try:

```
f = open("data.txt", "r")
content = f.read()
print("File Content: ")
print(content)
f.close()
```

```
except FileNotFoundError as e:
    print("Error: File not found.", e)
except PermissionError as e:
    print("Error: Permission denied.", e)
except OSError as e:
    print("General I/O error:", e)
```

Output (if file not found):

Error: File not found. [Errno 2] No such file or directory: 'data.txt'

4. Using Multiple except Blocks

- Different I/O exceptions को अलग-अलग except blocks में handle करना best practice है।
- इससे हमें specific error का पता चलता है और उचित action लेना आसान होता है।
- अंत में general exception (OSError) को भी handle करना चाहिए ताकि कोई unexpected error न छूटे।

5. Example : Writing to a File

Example: writing data to a file with exception handling
try:

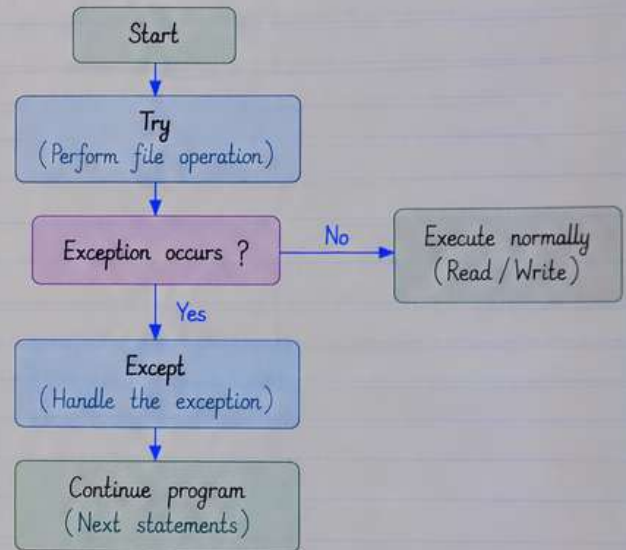
```
f = open("output.txt", "w")
f.write("Hello Python!\n")
f.write("File handling with exceptions.\n")
f.close()
print("Data written successfully.")
```

```
except PermissionError:
    print("Error: You don't have permission to write this file.")
except OSError as e:
    print("I/O Error:", e)
```

Output (on success):

Data written successfully.

6. Flow of Exception Handling in File I/O



7. Best Practices

- File operations हमेशा try-except block के अंदर करें।
- Specific exceptions (जैसे FileNotFoundError, PermissionError) को separately handle करें।
- User को clear और meaningful error message दें।
- File को use करने के बाद हमेशा close() करें (या with statement का use करें)।
- Unexpected I/O errors के लिए general OSError exception को handle करना न भूलें।
- Proper exception handling से program reliable और user-friendly बनता है।

8. Key Takeaways

- I/O operations में errors आना सामान्य है।
- Python इन errors को exceptions के रूप में handle करता है।
- Try-except block program को crash होने से बचाता है।
- सही exception handling से program robust और maintainable बनता है।

Topic : 3 examples of Input and Output

- Input and Output (I/O) program को user के साथ interact करने की सुविधा देता है।
- Input के माध्यम से हम data (जैसे number, string आदि) user से प्राप्त करते हैं।
- Output के माध्यम से हम result को screen पर display करते हैं या file में write करते हैं।
- Python में input() function और print() function I/O के लिए सबसे common functions हैं।

1. Example 1 : Taking Input from User and Displaying Output

Problem :

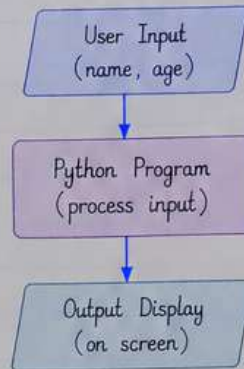
User से ससका नाम और उम्र (age) input लेकर उसे output में display करना।

Code :

```
# Taking input from user
name = input("Enter your name : ")
age = input("Enter your age : ")

# Displaying output
print("\nHello", name, "!")
print("You are", age, "years old.")
```

Flow / Working :



Sample Input :

Enter your name : Rahul
Enter your age : 20

Sample Output :

Hello Rahul !
You are 20 years old.

Explanation :

- input() function से user का data string के रूप में प्राप्त होता है।
- print() function उस data को output के रूप में display करता है।

2. Example 2 : Reading Data from a File and Displaying Output

Problem :

एक text file से data पढ़कर उसे screen पर display करता।

Code :

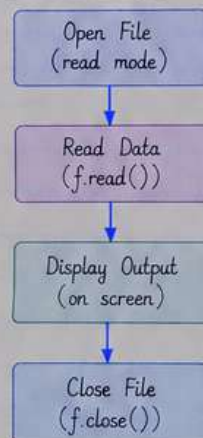
```
# Open the file in read mode
f = open("sample.txt", "r")

# Read the content
data = f.read()

# Display the content
print("File Content : ")
print(data)

# Close the file
f.close()
```

Flow / Working :



Sample File (sample.txt) :

Python is easy to learn.
Input and Output makes
programs interactive.
Keep Practicing !

Sample Output :

File Content :
Python is easy to learn.
Input and Output makes
programs interactive.
Keep Practicing !

Explanation :

- open() function file को read mode में open करता है।
- read() function से पूरा data एक string के रूप में प्राप्त होता है।
- File को use करने के बाद हमेशा close() करना चाहिए।

3. Example 3 : Writing Data to a File (with User Input)

Problem :

User से एक message input लेकर उसे file में write करना और success message display करना।

Code :

```
# Taking input from user
msg = input("Enter a message to save : ")

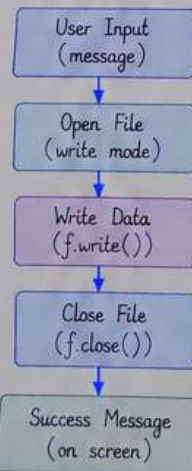
# Open the file in write mode
f = open("message.txt", "w")

# Write the message
f.write(msg)

# Close the file
f.close()

print("Message saved successfully !")
```

Flow / Working :



Sample Input :

Enter a message to save : I love Python!

File Content (message.txt) :

I love Python!

Sample Output :

Message saved successfully !

Explanation :

- input() से user का message प्राप्त किया जाता है।
- write() function उस message को file में लिखता है।
- File को close() करना जरूरी है ताकि data properly save हो सके।
- यह method log file, notes या user data store करने के लिए useful होता है।

Key Point : Input और Output की मदद से program user, files और devices के साथ communicate कर सकता है।

Unit - 9 : Class in Python

1. Introduction

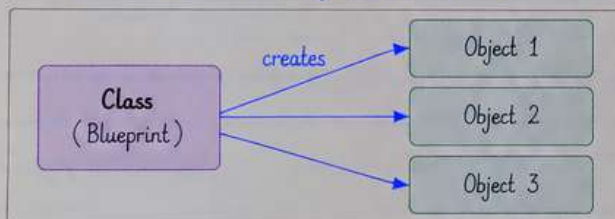
- Class in Python एक blueprint (खाका) होता है, जिसके द्वारा हम objects (वस्तु) create करते हैं।
- यह real world entities को model करने के लिए use होता है।
- Class में data (variables) और functions (methods) दोनों होते हैं, जो मिलकर object का behavior define करते हैं।
- Python एक object-oriented programming language है, इसलिए almost सभी चीजें class और object के रूप में represent होती हैं।

2. Why Use Class ?

- Code को organize करने के लिए।
- Real world problems को model करने के लिए।
- Code reusability बढ़ाने के लिए।
- Data को सुरक्षित रखने (data hiding) के लिए।
- Large programs को manage करने के लिए।

3. Class and Object

- **Class :** यह एक template होता है जिसमें attributes और methods define होते हैं।
- **Object :** Class का instance (वास्तविक रूप) होता है।
- हम एक class से कई objects बना सकते हैं।



4. Syntax of Class

```
class ClassName:           ← class definition
    def __init__(self, parameters): ← constructor (special method)
        # initialize attributes
        self.attribute = value      ← instance variable

    def method_name(self): ← method
        # method definition
        pass
```

5. Example : Creating a Class and Object

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def display(self):
        print(f"Name : {self.name}")
        print(f"Age : {self.age}")

# creating object
s1 = Student("Rahul", 20)
s1.display()
```

6. Output

Output :

Name : Rahul
Age : 20

7. __init__ () Method (Constructor)

- `__init__()` एक special method (constructor) होता है जो object create होते समय automatically call होता है।
- इसका use object के initial values set करने के लिए किया जाता है।
- यह method का नाम हमेशा `__init__` होता है।

8. self Keyword

- `self` एक reference variable है जो current object को represent करता है।
- इसका use class के attributes और methods को access करने के लिए होता है।
- हर method में `self` को first parameter के रूप में लिखना होता है।

9. Types of Variables in Class

Type	Description	Example
Instance Variable	यह हर object के लिए अलग होता है।	<code>self.name</code>
Class Variable	यह सभी objects के लिए common होता है।	<code>school = "ABC"</code>
Local Variable	यह method के अंदर define होता है और केवल उसी method में use होता है।	<code>x = 10</code>

10. Example : Class and Instance Variable

```
class Student:
    school = "ABC College" # class variable
    def __init__(self, name):
        self.name = name # instance variable
    def show(self):
        print(f"Name : {self.name}")
        print(f"School : {Student.school}")

s1 = Student("Vishal")
s2 = Student("Aman")
s1.show()
s2.show()
```

Output :

Name : Vishal
School : ABC College

Name : Aman
School : ABC College

11. Key Points

- Class एक blueprint है और object उसका instance होता है।
- `__init__()` constructor object के initial values set करता है।
- `self` current object को refer करता है।
- Class variable सभी objects के लिए common होता है।
- Instance variable हर object के लिए अलग होता है।
- Class का use करने से code organized, reusable और maintainable बनता है।

Conclusion : Class in Python हमें real world problems को model करने का एक powerful और flexible तरीका प्रदान करता है।

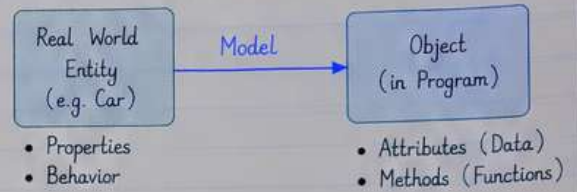
Topic : Principles of Object Orientation

1. Introduction

- Object Oriented Programming (OOP) एक programming paradigm है जो real world की problems को solve करने के लिए use होता है।
- OOP में program को objects के रूप में organize किया जाता है, जो data (attributes) और behavior (methods) को combine करते हैं।
- इसके main principles कुछ basic concepts हैं जिनके द्वारा OOP काम करता है।

"Object Oriented Programming makes code more modular, reusable and easy to maintain."

Real World to OOP



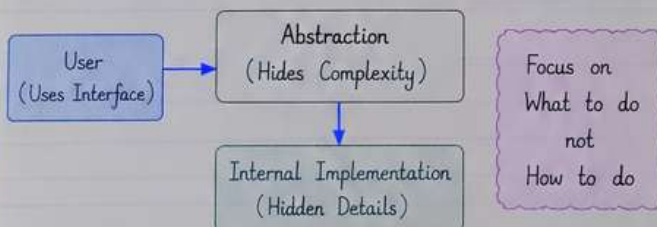
2. Main Principles of OOP

- OOP के चार main principles होते हैं :

1. Encapsulation
 2. Abstraction
 3. Inheritance
 4. Polymorphism
- ये सभी principles मिलकर code को modular, secure, flexible और reusable बनाते हैं।

4. Abstraction

- Abstraction का मतलब है केवल जरूरी (important) details को दिखाना और unnecessary details को hide करना।
- यह user को simple interface देता है।
- Abstraction के लिए हम abstract class और interface का use करते हैं (Python में abstract class के लिए abc module होता है)।



3. Encapsulation

- Encapsulation का मतलब है data (variables) और methods को एक single unit (class) में bind करना।
- यह data को protect करता है और direct access को restrict करता है।
- इसके लिए हम access modifiers (public, private, protected) का use करते हैं।

```
class Student:
    def __init__(self, name, age):
        self.__name = name # private variable
        self.__age = age # private variable
    def get_name(self): # getter method
        return self.__name
```

Data Hiding
Security
Better Control

5. Inheritance

- Inheritance का मतलब है एक class (child class) का दूसरी class (parent class) के properties और methods को प्राप्त करना।
- यह code reusability को बढ़ाता है।
- Child class अपने parent class के behavior को extend भी कर सकती है (नई functionality add करके)।

```
class Animal:
    def sound(self):
        print("Animal makes a sound")
class Dog(Animal): # Inheritance
    def sound(self): # Method Overriding
        print("Dog barks")
```

Code Reuse
Extend
Modify

6. Polymorphism

- Polymorphism का मतलब है "one interface, many forms"।
- एक ही method अलग-अलग objects के लिए अलग behavior कर सकता है।
- यह method overriding और method overloading (कुछ languages में) के द्वारा achieve होता है।
- Python में method overloading directly नहीं होता, but हम default arguments का use करके similar behavior achieve कर सकते हैं।

```
class Cat:
    def sound(self):
        print("Cat meows")
class Cow:
    def sound(self):
        print("Cow moos")
# Same method name, different behavior
def make_sound(animal):
    animal.sound()
make_sound(Cat())
make_sound(Cow())
```

Same Interface
Different Behavior
Flexibility

7. Conclusion

Principles of Object Orientation (Encapsulation, Abstraction, Inheritance, Polymorphism) मिलकर OOP को एक powerful programming approach बनाते हैं, जो real world applications को develop करने के लिए बहुत useful होता है।

Topic : Creating Classes, Instance Methods

1. Creating Classes

- Class एक blueprint (स्काफ) होता है जिसके द्वारा हम objects create करते हैं।
- Class में हम attributes (data) और methods (functions) define करते हैं जो उसके objects के behavior को तय करते हैं।
- class का use real world entities को model करने के लिए होता है।
- Python में class define करने के लिए class keyword का use किया जाता है।

2. Syntax of Creating a Class

```
class ClassName:  
    def __init__(self, parameters):  
        # initialize attributes  
        self.attribute = value
```

3. Example : Creating a Class

```
class Student:  
    def __init__(self, name, age):  
        self.name = name  
        self.age = age  
  
    def display(self):  
        print(f"Name : {self.name}")  
        print(f"Age : {self.age}")
```

4. Creating Objects (Instances)

- Class से object (instance) create करने के लिए class name को call किया जाता है।
- Object बनाने के बाद हम उसके methods को access कर सकते हैं।

```
# creating objects  
s1 = Student("Rahul", 20)  
s2 = Student("Priya", 19)  
  
# calling method  
s1.display()  
s2.display()
```

Output :

```
Name : Rahul  
Age : 20  
  
Name : Priya  
Age : 19
```

Note :

- हर object के अपने अलग attributes होते हैं।
- एक ही class से कई objects बना सकते हैं।

5. Instance Methods

- Instance method एक ऐसा method होता है जो object के साथ काम करता है और उसके data (attributes) को access करता है।
- Instance method का पहला parameter हमेशा self होता है, जो current object को refer करता है।
- इस method का use object के behavior को define करने के लिए होता है।

6. Syntax of Instance Method

```
class ClassName:  
    def method_name(self, parameters):  
        # method body  
        # use self to access instance variables  
        pass
```

7. Example : Instance Methods

```
class Rectangle:  
    def __init__(self, length, width):  
        self.length = length  
        self.width = width  
  
    def area(self): # instance method  
        return self.length * self.width  
  
    def display(self):  
        print(f"Length : {self.length}")  
        print(f"Width : {self.width}")  
        print(f"Area : {self.area()}")
```

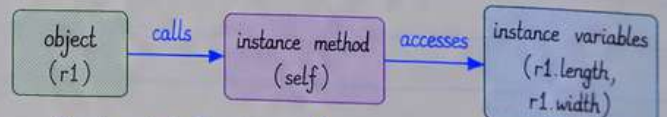
8. Creating Object and Calling Methods

```
r1 = Rectangle(5, 3)  
r1.display()  
  
r2 = Rectangle(10, 4)  
r2.display()
```

Output :

Length : 5	Length : 10
Width : 3	Width : 4
Area : 15	Area : 40

9. How Instance Method Works ?



- Object method को call करता है।
- self उस object को refer करता है।
- Method उस object के data को access करके operation perform करता है।

10. Key Points

- Class एक template है, objects उसके instances होते हैं।
- Instance method हमेशा self को पहला parameter लेता है।
- Instance method object के data को access और manipulate करता है।
- Methods के द्वारा हम object के behavior को define करते हैं।

Topic : File Organization

1. Introduction

- File Organization का मतलब है data को files में किस तरह से store, arrange और manage किया जाता है।
- यह एक method है जिसके द्वारा data को storage device (जैसे Hard Disk, SSD आदि) में logically organize किया जाता है।
- इसका main purpose data को efficiently store करना और later use के लिए easily access करना है।
- Proper file organization से storage space का better use होता है और data को search, read, update और maintain करना आसान हो जाता है।

2. File, Record and Field

- File** : Related data का collection जो एक नाम के साथ store होता है।
- Record** : File में store एक single data unit (जैसे एक student का पूरा data) होता है।
- Field** : Record का एक part होता है जो single information (जैसे name, age) को represent करता है।
- उदाहरण :

File : Student.txt

Roll No	Name	Age	Branch
101	Rahul	20	CSE
102	Priya	19	IT
103	Aman	21	ECE

← Field

← Record

← File (Collection of Records)

3. Need for File Organization

- Data को systematic तरीके से store करने के लिए।
- Data को जल्दी access करने के लिए।
- Storage space का efficient use करने के लिए।
- Data के update, insert और delete को आसान बनाने के लिए।
- Large amount of data को manage करने के लिए।
- Data redundancy और inconsistency को कम करने के लिए।

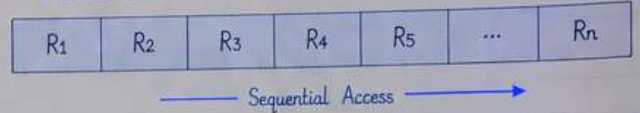
4. Types of File Organization

- Data को store करने के लिए अलग-अलग file organization methods use किए जाते हैं। मुख्य प्रकार निम्नलिखित हैं :

1. Sequential File Organization
2. Direct (Random) File Organization
3. Indexed File Organization
4. Hashed File Organization

5. Sequential File Organization

- इस method में records को एक fixed order में store किया जाता है (जैसे roll number के increasing order में)।
- Data को read करने के लिए file को beginning से sequentially access करना पड़ता है।
- यह method simple होता है और small data के लिए suitable है।



6. Direct (Random) File Organization

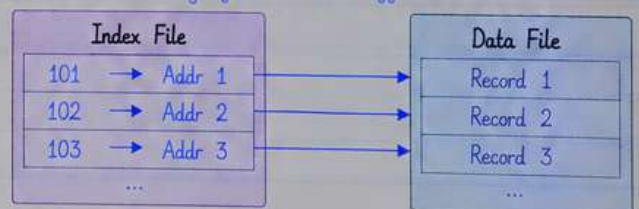
- इस method में हर record का एक unique address होता है।
- किसी भी record को directly access किया जा सकता है, बीच के records को read करने की जरूरत नहीं होती।
- यह method large data के लिए fast access प्रदान करता है।

R ₁	R ₂	R ₃	R ₄	R ₅	...	R _n
0	1	2	3	4	...	n

Record can be directly accessed by its address

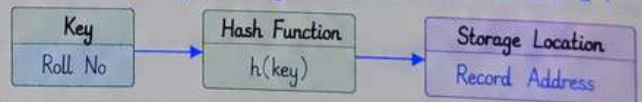
7. Indexed File Organization

- इस method में एक index file maintain की जाती है जो हर record की location (address) को store करती है।
- Index की मदद से required record को जल्दी find किया जा सकता है।
- यह method large files के लिए efficient होता है।



8. Hashed File Organization

- इस method में एक hash function का use करके record की storage location determine की जाती है।
- यह method बहुत fast access प्रदान करता है।
- यह mainly large and dynamic data के लिए use होता है।



9. Advantages of File Organization

- Data को efficiently store और manage करना।
- Fast data access और retrieval।
- Storage space का better utilization।
- Data consistency और integrity maintain करना।
- Large data को handle करने में मदद मिलना।

Conclusion :

File organization एक महत्वपूर्ण concept है जो data को systematic, efficient और easily accessible बनाने में मदद करता है।

Topic : Special Methods

1. Introduction

- Special Methods (या Dunder Methods) Python में ऐसे predefined methods होते हैं जो double underscore (__) से start और end होते हैं।
- इनका use, objects के behavior को define करने के लिए किया जाता है।
- ये methods Python द्वारा automatically call होते हैं जब हम built-in functions या operators का use करते हैं।
- Special methods की मदद से हम अपने class के objects को Python की built-in functionalities के साथ work करने के लिए customize कर सकते हैं।

2. Syntax

```
class ClassName:  
    def __method__(self, parameters):  
        # method body  
        pass
```

3. Common Special Methods

Method	Purpose / Description
__init__()	Constructor, object initialize करने के लिए
__str__()	Object की string representation (print)
__repr__()	Official string representation (debugging)
__len__()	len() function के लिए
__add__()	+ operator के लिए
__eq__()	== operator के लिए
__lt__()	< operator के लिए
__call__()	Object को function की तरह call करने के लिए
__del__()	Object delete होने पर cleanup के लिए

4. __init__() Method (Constructor)

- यह special method object create होते समय automatically call होता है।
- Iska use object ke initial values set करने के लिए किया जाता है।

Example :

```
class Student:  
    def __init__(self, name, age):  
        self.name = name  
        self.age = age  
  
s1 = Student("Rahul", 20)  
print(s1.name, s1.age)
```

Output :

Rahul 20

5. __str__() and __repr__()

- __str__(): user के लिए readable string return करता है (print() में use होता है)।
- __repr__(): object का official representation return करता है (mainly debugging के लिए)।

Example :

```
class Book:  
    def __init__(self, title, price):  
        self.title = title  
        self.price = price  
  
    def __str__(self):  
        return f"{self.title} (₹{self.price})"  
  
    def __repr__(self):  
        return f"Book('{self.title}', {self.price})"  
  
b = Book("Python", 450)  
print(b)  
print(repr(b))
```

Output :

Python (₹450)
Book('Python', 450)

6. __len__() Method

- यह method len() function के साथ काम करता है।
- इसका use object की length या size बताने के लिए किया जाता है।

Example :

```
class MyList:  
    def __init__(self, items):  
        self.items = items  
  
    def __len__(self):  
        return len(self.items)  
  
m = MyList([1, 2, 3, 4, 5])  
print(len(m))
```

Output :

5

7. Operator Overloading (e.g. __add__())

- Special methods की मदद से हम Python के operators (जैसे +, -, ==) के behavior को customize कर सकते हैं।

Example :

```
class Number:  
    def __init__(self, value):  
        self.value = value  
  
    def __add__(self, other):  
        return Number(self.value + other.value)  
  
    def __str__(self):  
        return str(self.value)  
  
n1 = Number(10)  
n2 = Number(5)  
n3 = n1 + n2  
print(n3)
```

Output :

15

8. __del__() Method (Destructor)

- यह method object delete होने से पहले automatically call होता है।
- Iska use cleanup activities (जैसे file close करना आदि) के लिए किया जा सकता है।

Key Point :

- Special methods हमें अपने objects को Python की built-in functionalities के साथ integrate करने में मदद करते हैं।
- इनका सही use code को अधिक readable, flexible और powerful करता है।

Topic : Class Variables

1. Introduction

- Class Variable एक ऐसा variable होता है जो class के level पर define किया जाता है, न कि किसी specific object (instance) के लिए।
- यह सभी objects के लिए common होता है।
- Class variable को class name के द्वारा access किया जाता है, लेकिन इसे object के द्वारा भी access किया जा सकता है।
- इसका उपयोग उन data के लिए किया जाता है जो सभी instances के लिए same (common) होने चाहिए।
- यह memory की बचत करता है क्योंकि इसकी केवल एक copy class में बनती है, जो सभी objects के बीच share होती है।

2. Syntax

```
class ClassName:
    class_variable = value # class variable
    def __init__(self, ...):
        self.instance_variable = value # instance variable

# Access
print(ClassName.class_variable)
print(obj.class_variable)
```

3. Example : Class Variable

```
class Student:
    school = "ABC College" # class variable
    def __init__(self, name, age):
        self.name = name # instance variable
        self.age = age # instance variable

    def display(self):
        print(f"Name : {self.name}")
        print(f"Age : {self.age}")
        print(f"School : {Student.school}")
```

4. Output

```
s1 = Student("Rahul", 20)
s2 = Student("Priya", 19)

s1.display()
s2.display()
print(Student.school)
print(s1.school)
print(s2.school)
```

Output :

Name : Rahul
Age : 20
School : ABC College

Name : Priya
Age : 19
School : ABC College

ABC College
ABC College
ABC College

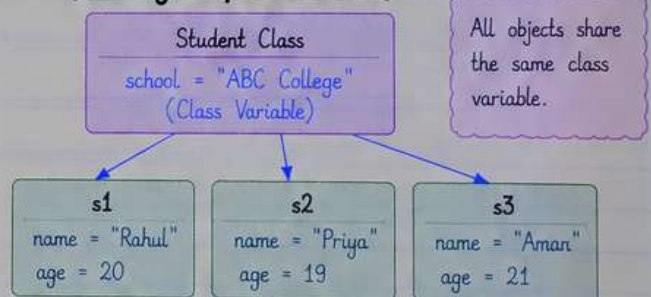
Class Variable vs Instance Variable

Feature	Class Variable	Instance Variable
Defined in	Inside class (outside methods)	Inside method (usually <code>__init__</code>)
Common to	All objects	Single object
Declaration	Directly in class	Using <code>self</code> in method
Memory	Single copy	Separate copy for each object
Access	<code>ClassName.var</code> or <code>obj.var</code>	<code>obj.var</code> (using <code>self</code>)
Use case	Common data (e.g. count)	Individual data (e.g. name, age)

5. How it Works ?

- जब class create होती है, तब class variable memory में एक बार बनता है।
- सभी objects उसी class variable को share करते हैं।
- अगर किसी object के द्वारा class variable को modify किया जाता है, तो वह change सभी objects पर effect करेगा (जब तक कि उसी नाम का instance variable न बन जाए)।
- यदि किसी instance में उसी नाम का instance variable बना दिया जाता है, तो वह class variable को उस object के लिए override कर देता है।

6. Memory Representation



7. Modifying Class Variable

```
Student.school = "XYZ University" # change using class name
print(s1.school) # XYZ University
print(s2.school) # XYZ University
```

8. Key Points

- Class variable सभी objects के लिए common होता है।
- इसे class के अंदर define किया जाता है (methods के बाहर)।
- इसे `ClassName` या object दोनों के द्वारा access किया जा सकता है।
- यह तब उपयोगी होता है जब हमें सभी instances के लिए same data maintain करना हो।
- अगर instance में उसी नाम का variable बना दिया जाता है, तो वह class variable को override कर देता है (सिर्फ उसी instance के लिए)।

Conclusion :

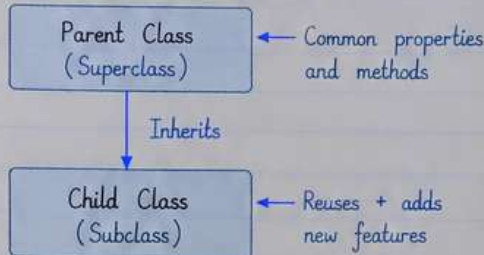
Class variable class के सभी objects के लिए common data provide करता है और memory की efficient use में मदद करता है।

Topic : Inheritance, Polymorphism

1. Inheritance (उत्तराधिकार)

- Inheritance OOP का एक important concept है जिसमें एक class (child class / subclass) दूसरी class (parent class / superclass) की properties और methods को inherit (प्राप्त) करती है।
- इसका use code reusability के लिए होता है।
- Child class, parent class के existing members का use कर सकती है और अपने नए members भी add कर सकती है।
- यह "is-a" relationship को represent करता है।

Inheritance Hierarchy



2. Syntax (Single Inheritance)

```
class Parent:
    def method1(self):
        pass

class Child(Parent): # Inherits from Parent
    def method2(self):
        pass
```

3. Example : Inheritance

```
class Animal:
    def sound(self):
        print("Animal makes a sound")

class Dog(Animal): # Dog inherits from Animal
    def bark(self):
        print("Dog barks")

d = Dog()
d.sound() # inherited method
d.bark() # child class method
```

Output :

Animal makes a sound
Dog barks

4. Types of Inheritance

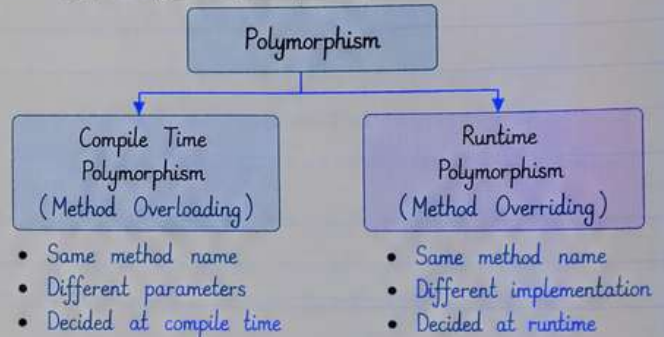
1. Single Inheritance - एक parent, एक child
2. Multiple Inheritance - एक से अधिक parent
3. Multilevel Inheritance - chain की तरह (A → B → C)
4. Hierarchical Inheritance - एक parent, कई child
5. Hybrid Inheritance - combination of multiple types

5. Polymorphism (बहुरूपता)

- Polymorphism का अर्थ है "one interface, many forms" यानी एक ही method अलग-अलग objects के लिए अलग behavior दिखा सकता है।
- यह same method name के साथ different behavior provide करता है।
- Polymorphism code को flexible और extensible बनाता है।
- यह runtime (method overriding) और compile time (method overloading) दोनों में हो सकता है।
- यह "same interface, different implementation" के concept पर काम करता है।

"Polymorphism allows the same function to behave differently in different situations."

6. Types of Polymorphism



7. Example : Method Overloading (Compile Time)

```
class Calculator:
    def add(self, a, b):
        return a + b

    def add(self, a, b, c):
        return a + b + c
```

Same method name "add()" but different number of parameters. (Python does not support true overloading, this is conceptual example)

8. Example : Method Overriding (Runtime)

```
class Animal:
    def sound(self):
        print("Animal makes a sound")

class Cat(Animal):
    def sound(self): # overriding
        print("Cat meows")

a = Animal()
c = Cat()
a.sound()
c.sound()
```

Same method name "sound()" but different implementation in child class.

Output :

Animal makes a sound
Cat meows

9. Key Points

- Inheritance code reusability के लिए होता है।
- Polymorphism same interface के साथ different behavior provide करता है।
- Inheritance "is-a" relationship है, Polymorphism "many forms" देता है।
- Polymorphism का use flexibility और scalability के लिए किया जाता है।
- Dono OOP के important concepts हैं और real world applications में बहुत useful हैं।

Topic : Example 1 of Class in Python

1. Objective

- Class in Python का basic example समझना ।
- Object बनाना और उसके attributes को access करना ।
- Instance method को define और call करना ।
- यह देखना कि class हमें real world entities को model करने में कैसे मदद करती हैं ।

2. Program : (Example 1)

```
class Student:                                # class definition
    def __init__(self, name, age, course):    # constructor
        self.name = name                    # instance variable
        self.age = age
        self.course = course

    def display(self):                        # instance method
        print(f"Name : {self.name}")
        print(f"Age : {self.age}")
        print(f"Course : {self.course}")

    def intro(self):                         # another method
        print(f"Hello, I am {self.name}.")
        print(f"I am {self.age} years old and I am studying {self.course}.")

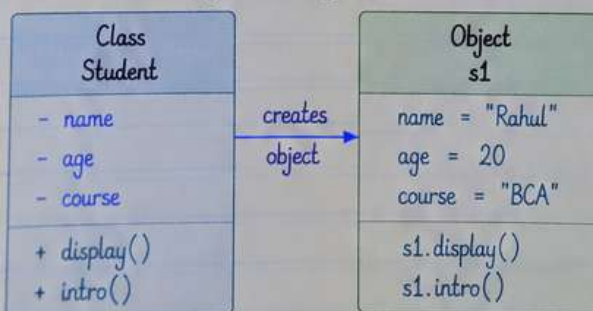
# Creating object of class
s1 = Student("Rahul", 20, "BCA")            # object creation

# Calling methods using object
s1.display()                                # method call
print("-----")
s1.intro()
```

3. Real World Analogy

- जैसे किसी College में Student के पास name, age, course जैसी information होती है ।
- उसी तरह हम Student class बनाकर इन properties को model कर सकते हैं ।
- हर student (object) की अपनी अलग values हो सकती हैं लेकिन सभी एक ही class के होते हैं ।

4. Class and Object Diagram



5. Output

Output :

Name : Rahul
Age : 20
Course : BCA

Hello, I am Rahul.

I am 20 years old and I am studying BCA.

6. Explanation

1. class Student : यहाँ Student नाम की class बनाई गई है ।
2. __init__() : यह constructor होता है जो object बनाने समय automatically call होता है और initial values set करता है ।
3. self : यह current object को refer करता है ।
4. self.name, self.age, self.course : ये instance variables हैं जो object के data को store करते हैं ।
5. display() and intro() : ये instance methods हैं जो object के behavior को define करते हैं ।
6. s1 = Student(...) : यहाँ Student class का object बनाया गया है ।
7. s1.display() और s1.intro() : object के methods को call करके output प्राप्त किया जाता है ।

Key Point :

- Class एक blueprint (स्काफ़) की तरह होती है ।
- Object class का instance होता है ।
- __init__() constructor object initialize करने के लिए उपयोग होता है ।
- Instance variables object के data को store करते हैं ।
- Instance methods object के behavior को define करते हैं ।

7. Advantages

- Code reusability (एक class से कई objects बनाए जा सकते हैं) ।
- Real world modeling में मदद करता है ।
- Data और functions को एक साथ organize करता है ।
- Program को modular और easy to maintain बनाता है ।

8. Conclusion

इस example से हमने देखा कि class in Python का उपयोग करके हम real world entities (जैसे Student) को आसानी से model कर सकते हैं । Class, object, constructor, instance variables और methods मिलकर Python को एक powerful और flexible programming language बनाते हैं ।

Topic : Program 1 of Class in Python

(Question and Solve)

1. Question :

Create a class named Student with the following attributes and methods :

- Attributes : name, age, course
- Method 1 : `display()` : to show the student details
- Method 2 : `change_course()` : to update the course of the student

Create an object of the class, call the methods and show the output.

2. Approach / Logic :

- सबसे पहले Student नाम की class बनाएँगे।
- `__init__()` constructor में name, age और course की initial values लेंगे और instance variables में store करेंगे।
- `display()` method द्वारा student का detail show करेंगे।
- `change_course()` method द्वारा course को update करेंगे।
- Class का object बनाएँगे और methods को call करेंगे।

3. Program (Solution) :

```
class Student:
    def __init__(self, name, age, course):
        self.name = name
        self.age = age
        self.course = course
```

```
    def display(self):
        print("Name :", self.name)
        print("Age :", self.age)
        print("Course :", self.course)
```

```
    def change_course(self, new_course):
        self.course = new_course
        print("Course updated successfully!")
```

Creating object

```
s1 = Student("Rahul", 20, "BCA")
```

Displaying initial details

```
print("---- Student Details (Before) ----")
s1.display()
```

Changing course

```
s1.change_course("MCA")
```

Displaying updated details

```
print("---- Student Details (After) ----")
s1.display()
```

4. Output :

--- Student Details (Before) ---

Name : Rahul

Age : 20

Course : BCA

Course updated successfully!

--- Student Details (After) ---

Name : Rahul

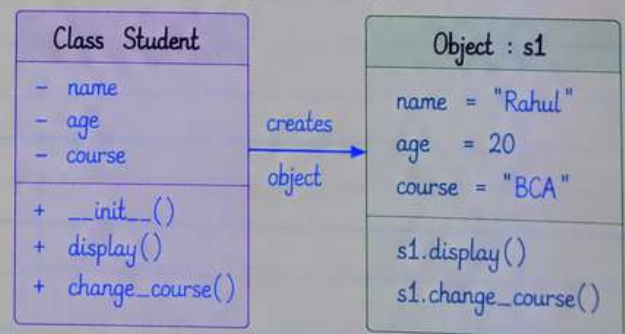
Age : 20

Course : MCA

5. Explanation :

1. `class Student` : यह एक class है जो student की information को represent करती है।
2. `__init__()` : यह constructor होता है जो object बनते समय automatically call होता है और name, age, course को initialize करता है।
3. `self` : यह current object को refer करता है।
4. `display()` : यह method student के details को print करता है।
5. `change_course()` : यह method course को update करता है।
6. `s1 = Student(...)` : इससे class का object बनता है।
7. `s1.display()` और `s1.change_course()` : object के methods को call करके output प्राप्त किया जाता है।

6. Class and Object Diagram :



7. Key Points :

- Class एक blueprint (खाका) की तरह होता है।
- Object class का instance होता है।
- `__init__()` constructor object initialize करने के लिए उपयोग होता है।
- Instance variables object के data को store करते हैं।
- Methods object के behavior को define करते हैं।
- Program को और useful बनाने के लिए इसमें और भी methods जोड़े जा सकते हैं।

Unit - 10 : Regular Expressions

1. Introduction

- Regular Expression (Regex) एक pattern (पैटर्न) होता है जो text (string) में specific pattern को search, match, validate या manipulate करने के लिए उपयोग किया जाता है।
- यह किसी भी text data में characters के sequence को define करने का एक powerful तरीका है।
- Regular Expression का उपयोग text processing, data validation, search, replace और information extraction जैसे tasks के लिए किया जाता है।
- Python में re module regular expression operations के लिए provide किया जाता है।
- Regex का pattern string के रूप में लिखा जाता है और उसे re module की functions के साथ use किया जाता है।

"Regular Expression एक छोटा सा pattern है लेकिन powerful tool है जो text data को handle करने के लिए बहुत useful है।"

2. Why use Regular Expressions?

- Complex text search को आसान बनाता है।
- Data validation (जैसे email, phone number, date) के लिए useful है।
- Text से specific information extract करने में मदद करता है।
- Search और replace operations को fast और flexible बनाता है।
- Programming और real world applications दोनों में widely used होता है।

3. re Module in Python

- Python में regular expression के लिए re (regular expression) module use किया जाता है।
- यह module pattern matching के लिए कई useful functions provide करता है।
- सबसे commonly used functions :

Function	Description
re.match()	String के start में match खोजता है
re.search()	पूरी string में कहीं भी match खोजता है
re.findall()	सभी matches की list return करता है
re.sub()	Match हुए text को replace करता है
re.split()	Pattern के आधार पर string को split करता है

Note :

re module को use करने से पहले import करना होता है।
import re

4. Regular Expression Pattern

- Regular expression pattern में normal characters के साथ कुछ special characters (metacharacters) भी होते हैं।
- ये special characters pattern को define करने के लिए use किए जाते हैं।

Example :

Pattern : ^a.*b\$

^ → string की शुरुआत को दर्शाता है

a → 'a' character से start

.* → उसके बाद कोई भी characters (0 या अधिक)

b → 'b' character

\$ → string के अंत को दर्शाता है

5. Simple Character Matches

- Simple character match में हम सीधे characters का use करके pattern बनाते हैं।
- यह basic matching के लिए use होता है जहाँ हम specific characters को search करना चाहते हैं।
- उदाहरण के लिए, किसी string में कोई particular letter, digit या word को ढूँढना।

6. Common Simple Character Examples

Pattern	Meaning	Example String	Match Result
a	character 'a'	apple	a (match)
abc	exact string 'abc'	abcdef	abc (match)
Python	exact word 'Python'	I love Python	Python (match)
7	digit '7'	a7b	7 (match)
xyz	exact string 'xyz'	xyz123	xyz (match)

7. Example Program

```
import re
text = "I love Python and Python is easy."
# search() example
match = re.search("Python", text)
if match:
    print("Found:", match.group())
else:
    print("Not Found")

# findall() example
matches = re.findall("Python", text)
print("All matches:", matches)
```

Output :

Found: Python
All matches: ['Python', 'Python']

8. Key Points

- Regular expressions text को search और manipulate करने के लिए एक powerful tool है।
- Simple character matches basic pattern matching के लिए use होते हैं।
- Python में re module regular expression operations के लिए provide किया जाता है।
- Regular expressions का practice करने से complex patterns को समझना आसान हो जाता है।

Topic : Special Characters , Character Classes

1. Special Characters (Metacharacters)

- Regular Expressions में कुछ characters ऐसे होते हैं जिनका special meaning होता है, इन्हें Special Characters या Metacharacters कहते हैं।
- यह characters normal text की तरह match नहीं होते, बल्कि pattern को define करने के लिए use होते हैं।
- इनका उपयोग search, validate और manipulate करने के लिए किया जाता है।

Special Character	Meaning	Example / Use
.	किसी भी single character	a.t → act, a1t
^	string की शुरुआत	^a → apple
\$	string का अंत	t\$ → cat, bat
*	0 या अधिक बार	ab* → a, ab, abb
+	1 या अधिक बार	ab+ → ab, abb
?	0 या 1 बार (optional)	colou?r → color, colour
	either / or (alternative)	cat dog → cat या dog
()	grouping	(ab)+ → ab, abab
[]	character set	[abc] → a, b या c
{ }	exact repetition	a{3} → aaa
\	escape character	\. → dot को match
\d	digit (0-9)	\d → 0 to 9
\w	word character	\w → a-z, A-Z, 0-9, _
\s	whitespace	\s → space, tab, newline

2. Character Classes (Custom Set)

- Square brackets [] का use करके हम अपना खुद का character set define कर सकते हैं।
- इसमें दिए गए किसी भी character में से एक character match होता है।
- Range का भी use कर सकते हैं, जैसे a-z, A-Z, 0-9.
- यदि ^ को brackets के अंदर लिखा जाए तो वह negation (उसके अलावा) को show करता है।

Pattern	Meaning	Example Match
[abc]	a, b या c में से कोई एक	a, b, c
[a-z]	छोटे अंग्रेजी अक्षर (a से z)	a, m, z
[A-Z]	बड़े अंग्रेजी अक्षर (A से Z)	A, M, Z
[0-9]	digits (0 से 9)	0, 5, 9
[a-zA-Z]	सभी letters	a, Z, m
[0-9a-z]	digit या lowercase letter	5, a, 9
[^abc]	a, b, c को छोड़कर कोई भी	d, x, 1
[^0-9]	digit के अलावा कोई भी	a, #, @

3. Character Classes (Predefined)

- Character Classes कुछ pre-defined sets होते हैं जो characters के group को represent करते हैं।
- इनका use frequently होता है क्योंकि ये commonly used patterns के लिए shortcut प्रदान करते हैं।

Character Class	Meaning	Equivalent
\d	Digit (0-9)	[0-9]
\D	Non-digit	[^0-9]
\w	Word character	[a-zA-Z0-9_]
\W	Non-word character	[^\w]
\s	Whitespace	[\t\n\r\f\v]
\S	Non-whitespace	[^\s]

Note :

Character classes की मदद से हम बड़े patterns को छोटे और readable form में लिख सकते हैं।

4. Examples of Character Classes

Pattern	Matches	Example Strings
\d+	one or more digits	123, 007
\w+	word (letters/digits/_)	name_1, test
\s+	one or more spaces	. (space) , \t
\D+	non-digit characters	abc, # \$
\W+	non-word characters	@#!, .
\S+	non-space characters	abc, 123
[a-z]+	lowercase letters	abc, hello
[A-Z]+	uppercase letters	ABC, PYTHON
[0-9]{3}	exact 3 digits	123, 456
[a-zA-Z0-9_]+	alphanumeric word	user_1, test22

5. Key Points

- Special characters pattern को powerful बनाते हैं।
- Character classes से हम group of characters को easily match कर सकते हैं।
- Predefined classes (\d, \w, \s आदि) commonly use होते हैं।
- Custom character sets [] से हम अपनी आवश्यकता के अनुसार pattern बना सकते हैं।
- Regular expressions का सही उपयोग text processing, validation और data extraction के लिए बहुत useful है।

Topic : Quantifiers, The Dot Character, Greedy Matches

1. Quantifiers (मात्रा सूचक)

- Quantifiers regular expressions में बताते हैं कि किसी character या group की कितनी बार occurrence होनी चाहिए।
- इनका use pattern में repetition specify करने के लिए होता है।
- ये character, group या character class पर apply होते हैं।
- Quantifiers की मदद से हम flexible और powerful patterns बना सकते हैं।

Quantifier	Meaning	Example	Matches
*	0 या अधिक बार	a*	'' , a, aa, aaa
+	1 या अधिक बार	a+	a, aa, aaa
?	0 या 1 बार (optional)	colou?r	color, colour
{n}	ठीक n बार	a{3}	aaa
{n,}	कम से कम n बार	a{2,}	aa, aaa, aaaa
{n,m}	n से m बार	a{2,4}	aa, aaa, aaaa

Example :

Pattern : a+	String : "aabb"	Match : "aaa"
Pattern : ab*	String : "abbb"	Match : "abbb"
Pattern : colou?r	String : "colour"	Match : "colour"
Pattern : \d{2,4}	String : "1234"	Match : "1234"

Key Points :

- * → 0 या अधिक बार
- + → 1 या अधिक बार
- ? → 0 या 1 बार
- {n}, {n,}, {n,m} → specific range के लिए
- Quantifiers greedy by default होते हैं (ज्यादा से ज्यादा match करते हैं)।

2. The Dot Character (.)

- Dot (.) एक special character है जो किसी भी single character को match करता है (उसके type की परवाह किये बिना)।
- यह letter, digit, symbol, space आदि सभी को match करता है।
- लेकिन यह newline character (\n) को match नहीं करता (by default)।
- Dot character का use तब करते हैं जब हमें किसी भी एक character की जगह match करना हो।

Note : अगर newline को भी match करना हो तो re.DOTALL flag का use किया जाता है।

Pattern	String	Match	Explanation
a.b	'acb'	acb	बीच का कोई भी character
a.b	'a b'	a b	space भी match होता है
a.b	'a#b'	a#b	symbol भी match होता है
a.b	'ab'	No match	बीच में character नहीं है
a.*b	'axxb'	axxb	.* के साथ 0 या अधिक character match होते हैं।

Common Use :

- file name matching → *.txt
- email pattern → a.b@gmail.com
- any character search → p.t (pat, pot, put...)

3. Greedy Matches (लालची मिलान)

- Python में quantifiers (*, +, ?, {n,m}) by default greedy होते हैं, जिसका मतलब वे सबसे लंबा (longest) possible match करते हैं।
- Greedy matching useful होता है, लेकिन कुछ cases में यह unwanted results दे सकता है।
- ऐसे situations में non-greedy (lazy) quantifiers का use किया जाता है, जिसमें quantifier के बाद ? लगाया जाता है (*?, +?, ??, {n,m}??)।

Pattern	String	Greedy Match	Non-greedy Match
<.*>	'<html>'	<html>	<html> (same)
<.*>	'<a> '	<a> 	<a> (with .*?)
a.*b	a123b456b'	a123b456b	a123b (with .*?)
".*"	"hello" "world"	"hello" "world"	"hello" (with .*?)

Key Points :

- Greedy match → सबसे लंबा match करता है।
- Non-greedy match → सबसे छोटा match करता है (quantifier के बाद ?)।
- Use non-greedy matching जब unwanted extra text भी match हो रहा हो।

Topic : Grouping , Matching at Beginning or End ,
Match Objects , Substituting

1. Grouping ()

- Parentheses () का use करके हम pattern के किसी part को group कर सकते हैं।
- Grouping से हम complex pattern को easily handle कर सकते हैं और उस group पर quantifiers apply कर सकते हैं।
- यह matching को organize करने के लिए useful होता है।
- Grouping की help से हम sub-patterns को capture भी कर सकते हैं (जिन्हें groups कहा जाता है)।

Pattern	Meaning	Example String	Match
$(ab)^+$	'ab' group एक या अधिक बार	ab, abab	ab, abab
$a b c$	'a' या 'b' के बाद 'c'	ac, bc	ac, bc
$(cat dog)$	'cat' या 'dog'	I have a dog	dog
$(\{3\}) - (\{2\})$	3 digits - 2 digits	123-45	123-45

Example :

```
import re
text = "cat and dog and cat"
pattern = r"(cat|dog)"
matches = re.findall(pattern, text)
print(matches)      # Output: ['cat', 'dog', 'cat']
```

3. Match Objects

- re module की functions (जैसे re.search(), re.match()) हमें एक Match Object return करती हैं।
- Match Object में matched text और उसके related information प्राप्त करने के लिए कई methods होती हैं।
- Match object का use तब किया जाता है जब हमें सिर्फ first match की information चाहिए।
- यदि match नहीं मिलता तो None return होता है।

Method	Description
group()	पूरा matched text return करता है।
start()	match की starting index देता है।
end()	match की ending index देता है।
span()	(start, end) tuple return करता है।
group(n)	किसी specific group का matched text देता है।

Example :

```
import re
text = "My roll number is 1023."
m = re.search(r"\d+", text)
if m:
    print("Matched text :", m.group()) # 1023
    print("Start index :", m.start())  # 19
    print("End index :", m.end())      # 23
    print("Span :", m.span())          # (19, 23)
```

2. Matching at Beginning or End

- $\wedge$ (caret) का use string के beginning में match करने के लिए होता है।
- $\$$ (dollar) का use string के end में match करने के लिए होता है।
- ये anchors पूरे string में position को check करते हैं।
- इनका use data validation (जैसे पूरा string match करना) के लिए किया जाता है।

Pattern	Meaning	Example String	Match
^abc	string की शुरुआत में 'abc'	abcdef	abc
xyz\$	string के अंत में 'xyz'	helloxyz	xyz
^\d+\$	पूरी string केवल digits	12345	12345
^[A-Z]	string की शुरुआत में uppercase letter	Python	P

Example :

```
import re
text = "hello world"
print(re.findall(r"^hello", text)) # ['hello']
print(re.findall(r"world$", text)) # ['world']
```

4. Substituting.

- `re module` में `re.sub()` function का use matched text को किसी दूसरे text से replace करने के लिए होता है।
- यह function सभी matches या specified number (count) तक के matches को replace कर सकता है।
- यह text को modify करने के लिए बहुत useful है, जैसे extra spaces हटाना, format बदलना, आदि।

Syntax :

re.sub(pattern, replacement, string, count=0)

↓ ↓ ↓ ↓

जिसे search जिससे replace जिस text में कितनी बार
करना है करना है search करना है replace करना है
(0 = सभी)

Example :

```
import re
text = "cat cat dog cat"
new_text = re.sub(r"cat", "pet", text)
print(new_text)      # pet pet dog pet

# सिर्फ first match replace करना
new_text2 = re.sub(r"cat", "pet", text, count=1)
print(new_text2)     # pet cat dog cat
```

Topic : Splitting a String , Compiling Regular Expressions , Flags

1. Splitting a String

- re module में re.split() function का use किसी string को split करने के लिए किया जाता है।
- यह specified pattern के आधार पर string को parts में divide करता है।
- Split करते समय delimiter (जैसे space, comma या कोई regex pattern) remove हो जाता है।
- यह function list return करता है जिसमें split हुए substrings होते हैं।
- यदि maxsplit दिया जाए तो केवल उतनी बार ही split किया जाता है।

Syntax :

```
re.split(pattern, string, maxsplit = 0, flags = 0)
```

- pattern - वह regex pattern जिसके आधार पर split करना है।
- string - वह input string जिसे split करना है।
- maxsplit - (optional) अधिकतम कितनी बार split करना है।
- flags - (optional) regex flags (जैसे re.IGNORECASE)।

Example 1 : (Basic Split)

```
import re
text = "apple,banana,mango,orange"
result = re.split(",", text)
print(result) # ['apple', 'banana', 'mango', 'orange']
```

Example 2 : (Split using multiple spaces)

```
import re
text = "This is a test"
result = re.split(r"\s+", text)
print(result) # ['This', 'is', 'a', 'test']
```

Example 3 : (Using maxsplit)

```
import re
text = "one-two-three-four"
result = re.split(r "-", text, maxsplit = 2)
print(result) # ['one', 'two', 'three-four']
```

Note : re.split() में delimiter result में include नहीं होता है, जबकि re.findall() match को return करता है।

Example : (IGNORECASE)

```
import re
text = "Python is Powerful"
pattern = re.compile(r"python", re.IGNORECASE)
match = pattern.search(text)
print(match.group()) # Python
```

Key Point : Flags regex को powerful और flexible बनाते हैं, जिससे अलग-अलग types के text patterns को आसानी से handle किया जा सकता है।

2. Compiling Regular Expressions

- Python में re.compile() function का use regex pattern को compile करने के लिए किया जाता है।
- Compiled pattern एक Pattern Object return करता है।
- इस Pattern Object का use बार-बार search, match, findall, split आदि operations करने के लिए किया जा सकता है।
- यदि हम एक ही pattern को कई बार use कर रहे हैं तो compile करना ज्यादा efficient होता है।
- यह code को readable भी बनाता है और performance भी improve करता है।

Syntax :

```
pattern = re.compile(pattern, flags = 0)
```

- pattern - regex pattern (string)
- flags - (optional) जैसे re.IGNORECASE, re.MULTILINE

Example :

```
import re
pattern = re.compile(r"\d+") # compile pattern
text = "Roll no: 101, 102, 205"
matches = pattern.findall(text)
print(matches) # ['101', '102', '205']
```

Note : Compiled pattern का use करने के लिए हम उस object के methods (search(), findall(), split()) आदि को call करते हैं।

3. Flags

- Flags का use regex pattern के behavior को modify करने के लिए किया जाता है।
- यह हमें कुछ special options provide करते हैं, जैसे case-insensitive search, multiline mode आदि।
- Flags को re.compile() या सीधे functions (जैसे re.search()) में use किया जा सकता है।

Flag	Name	Description
re.IGNORECASE	I	Case insensitive matching (uppercase/lowercase ignore)
re.MULTILINE	M	^ और \$ हर line पर work करते हैं।
re.DOTALL	S	Dot (.) newline (\n) को भी match करता है।
re.VERBOSE	X	Pattern को readable form में लिखने के लिए (spaces और comments allow)।

Example : (MULTILINE)

```
import re
text = "Hello\nWorld"
pattern = re.compile(r"^World", re.MULTILINE)
match = pattern.search(text)
print(match.group()) # World
```

Program : Validate Email Addresses using Regular Expression

1. Problem Statement

- एक Python program लिखिए जो किसी string में दिए गए email address को validate करे (check करे) कि वह valid email format में है या नहीं, using Regular Expression.

2. Approach

- Email का general format होता है :

username@domain.extension

- हम regular expression pattern का use करके email के format को match करेंगे।
- यदि string pattern से match करता है तो वह valid email है, अन्यथा invalid है।
- इसके लिए हम re module की re.match() function का use करेंगे।

3. Regular Expression Pattern

Pattern : `r"^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$"`

Pattern Explanation :

Part	Meaning
<code>^</code>	string की शुरुआत (start)
<code>[A-Za-z0-9._%+-]+</code>	username part (एक या अधिक valid characters)
<code>@</code>	'@' symbol (mandatory)
<code>[A-Za-z0-9.-]+</code>	domain name (जैसे gmail, yahoo)
<code>\.</code>	dot (.) before extension
<code>[A-Za-z]{2,}</code>	extension (जैसे .com, .in, .org)
<code>\$</code>	string का अंत (end)

4. Program Code (Python)

```
import re
# email validation के लिए regex pattern
pattern = r"^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$"
def check_email(email):
    if re.match(pattern, email):
        return "Valid Email"
    else:
        return "Invalid Email"
# Test cases
emails = [
    "student@gmail.com",
    "user.name@yahoo.in",
    "abc123@domain.co",
    "wrong@.com",
    "noatsymbol.com",
    "test@domain",
    "name+tag@university.ac.in"
]
for e in emails:
    print(f"{e} --> {check_email(e)}")
```

5. Output

```
student@gmail.com --> Valid Email
user.name@yahoo.in --> Valid Email
abc123@domain.co --> Valid Email
wrong@.com --> Invalid Email
noatsymbol.com --> Invalid Email
test@domain --> Invalid Email
name+tag@university.ac.in --> Valid Email
```

6. Explanation of Output

- पहले, दूसरे, तीसरे और आखिरी email का format regex pattern से match करता है, इसलिए यह Valid Email है।
- wrong@.com में domain name सही नहीं है (dot के बाद domain missing है), इसलिए Invalid है।
- noatsymbol.com में '@' symbol ही नहीं है, इसलिए Invalid है।
- test@domain में extension नहीं है, इसलिए Invalid है।

7. Key Points

- Regular Expression complex pattern matching के लिए बहुत useful होता है।
- Email validation में regex का use करके हम format को quickly check कर सकते हैं।
- re.match() function string की शुरुआत से match करता है।
- Pattern को और strict या flexible बनाया जा सकता है requirement के अनुसार।
- Regex validation 100% perfect नहीं होता, लेकिन basic format checking के लिए काफी अच्छा है।

8. Applications

- User registration form में email validation।
- Data cleaning और preprocessing के लिए।
- Web scraping में email extract करने के लिए।
- Any application जहाँ specific format validation की आवश्यकता हो।

Note :

Regular Expression का use करते समय pattern को समझना बहुत जरूरी है, क्योंकि छोटा सा change भी matching result को प्रभावित कर सकता है।